

CSD3009 - DATA STRUCTURES AND ANALYSIS OF ALGORITHMS

DR.S.DEVARAJU

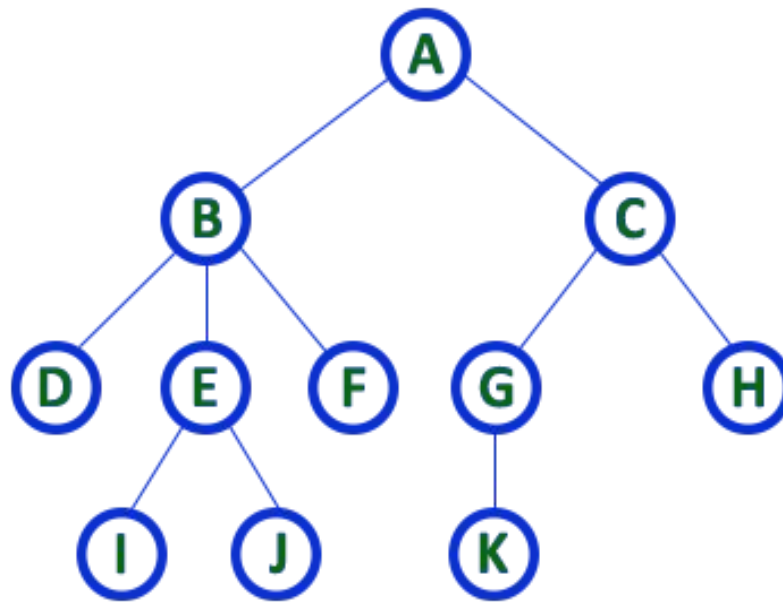
Module 3

Trees and Graphs: Trees: Binary tree, Binary search tree, threaded binary tree, Height balanced trees, Tries, Heaps, Hash tables. Graph traversals: Breadth-First Search, Depth First Search, Shortest path: Depth-first search in directed and undirected graphs. Union-find data structure and applications. Directed acyclic graphs; topological sort.

<https://www.javatpoint.com/data-structure-tutorial>

Tree Definition

Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition.



TREE with 11 nodes and 10 edges

- In any tree with '**N**' nodes there will be maximum of '**N-1**' edges
- In a tree every individual element is called as '**NODE**'

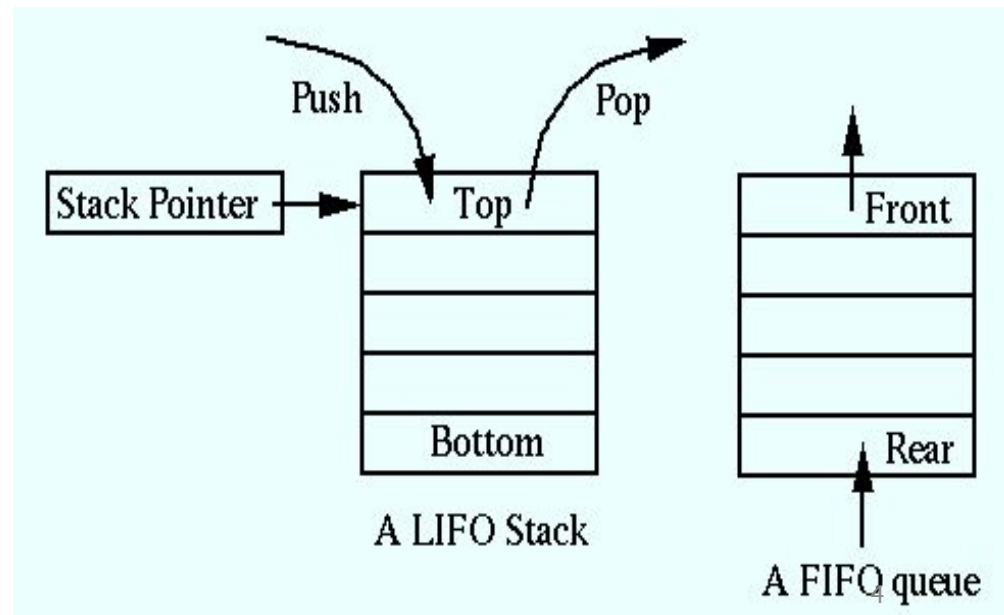
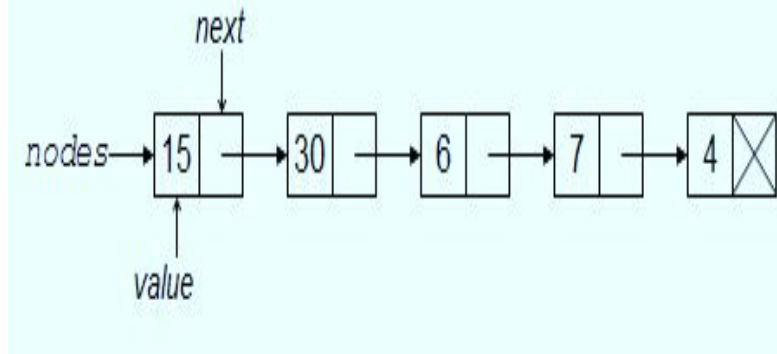
Data structure

Store and organize data in computer.

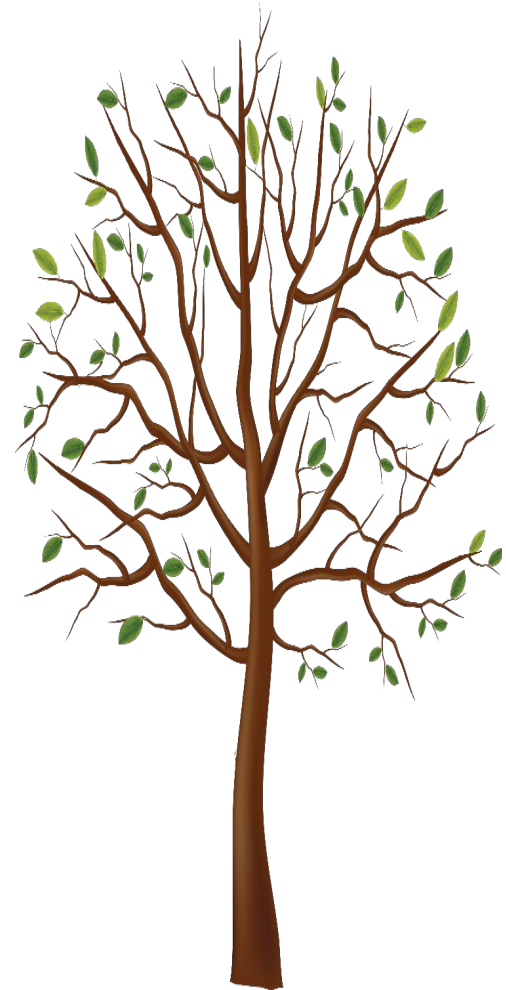
Linear Data Structure

- Arrays
- Linked List
- Stacks
- Queues

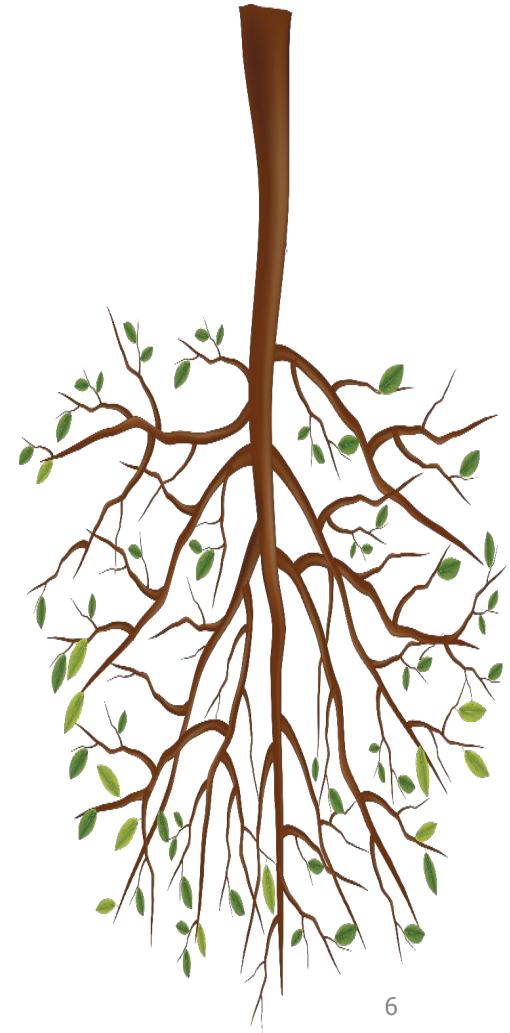
$a[i]$	$a[0]$	$a[1]$	$a[2]$	$a[n]$
element	5	7	2	12
Address	1000	1002	1004	n



Logic of Tree

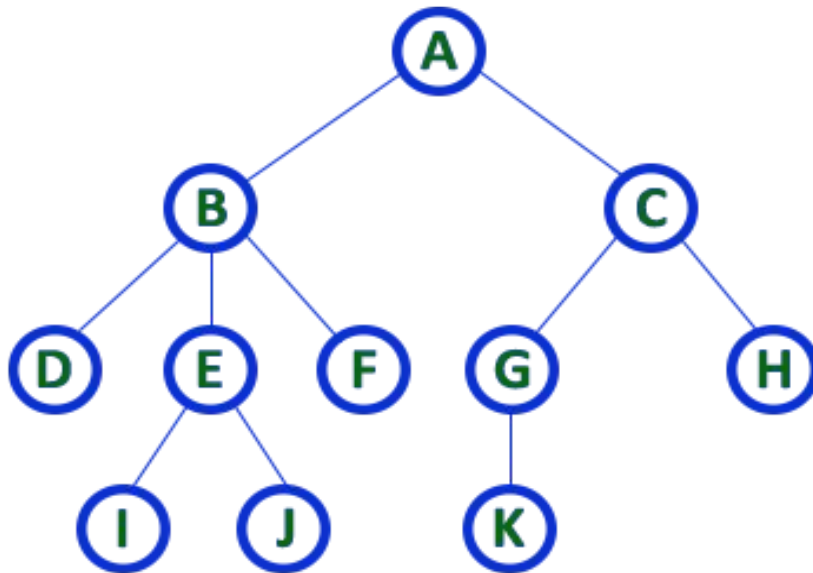


Logic of Tree



Tree Definition

Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition.



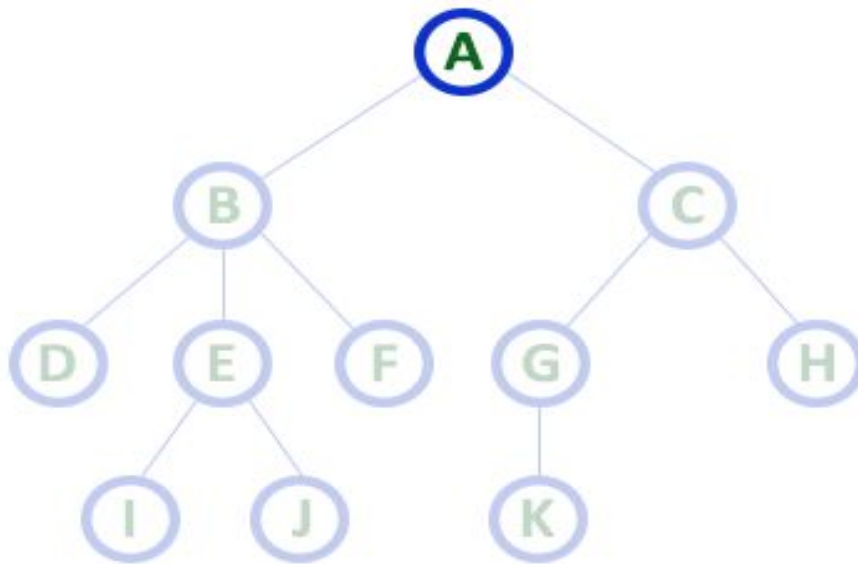
TREE with 11 nodes and 10 edges

- In any tree with '**N**' nodes there will be maximum of '**N-1**' edges
- In a tree every individual element is called as '**NODE**'

Tree terminology...

. 1.Root:

- The first node is called as Root Node.
- Every tree must have root node, there must be only one root node.
- Root node doesn't have any parent.

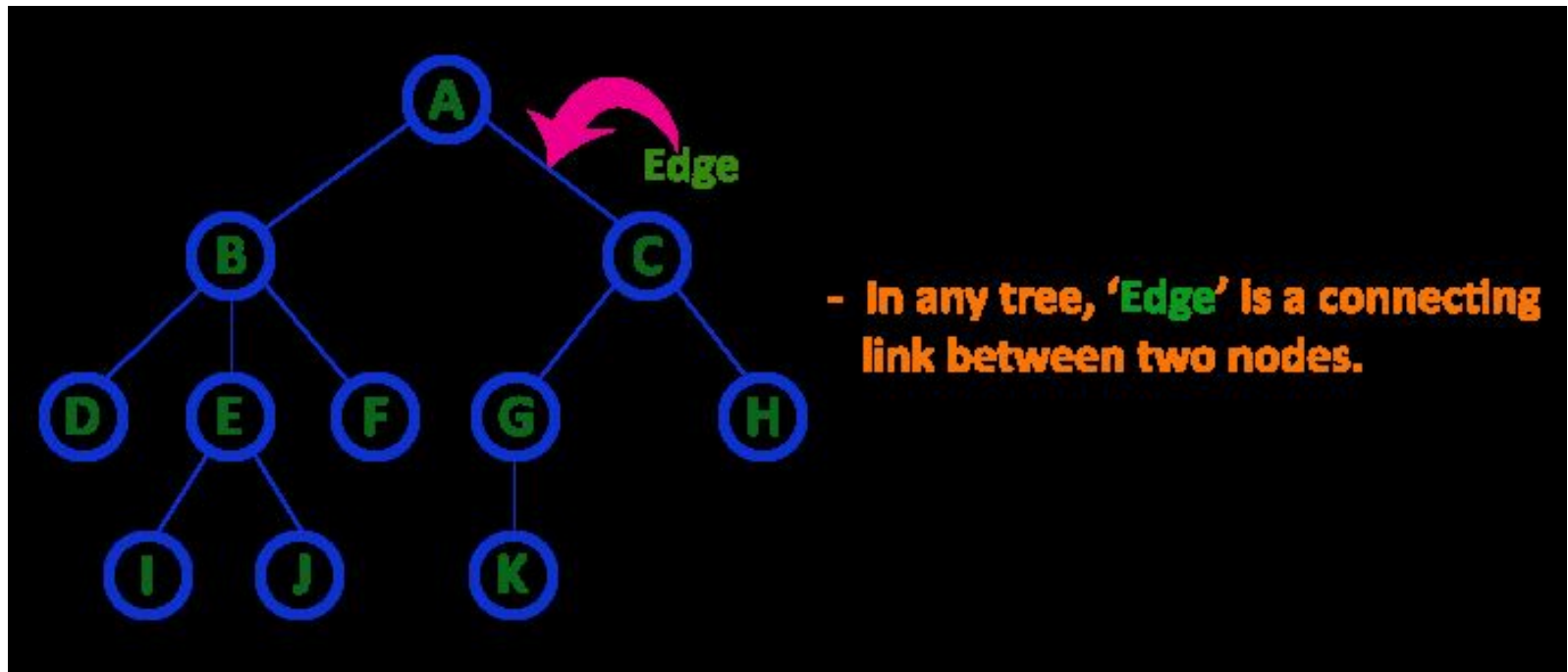


Here 'A' is the 'root' node

- In any tree the first node is called as ROOT node

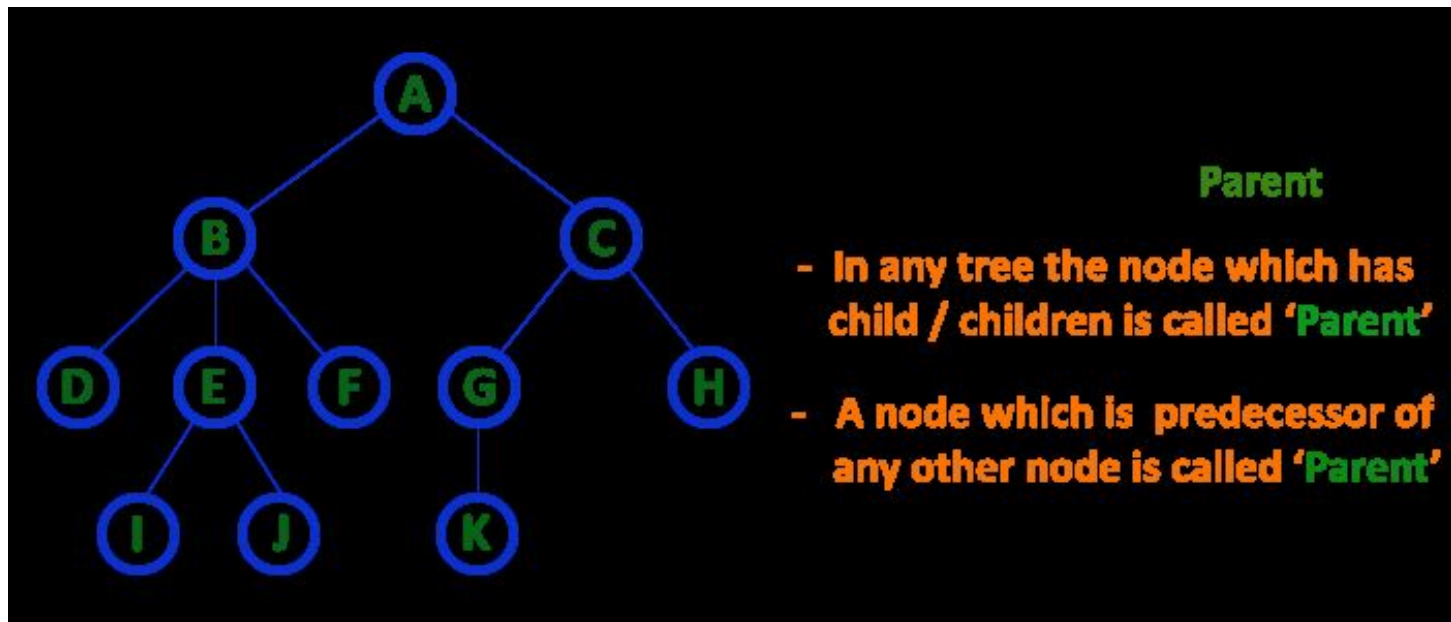
2. Edge

- In a tree data structure, the connecting link between any two nodes is called as EDGE. In a tree with 'N' number of nodes there will be a maximum of 'N-1' number of edges.



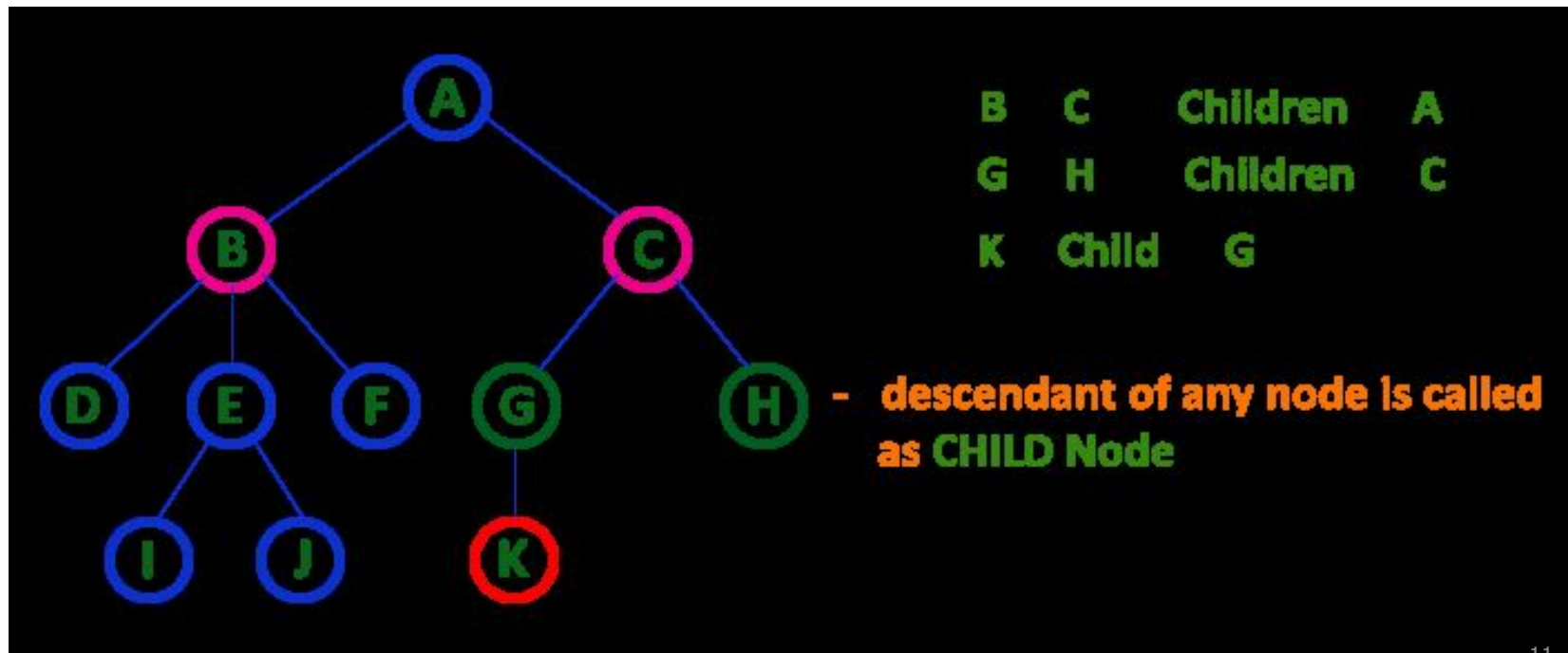
• 3. Parent

- In a tree data structure, the node which is predecessor of any node is called as PARENT NODE. In simple words, the node which has branch from it to any other node is called as parent node. Parent node can also be defined as "The node which has child / children".



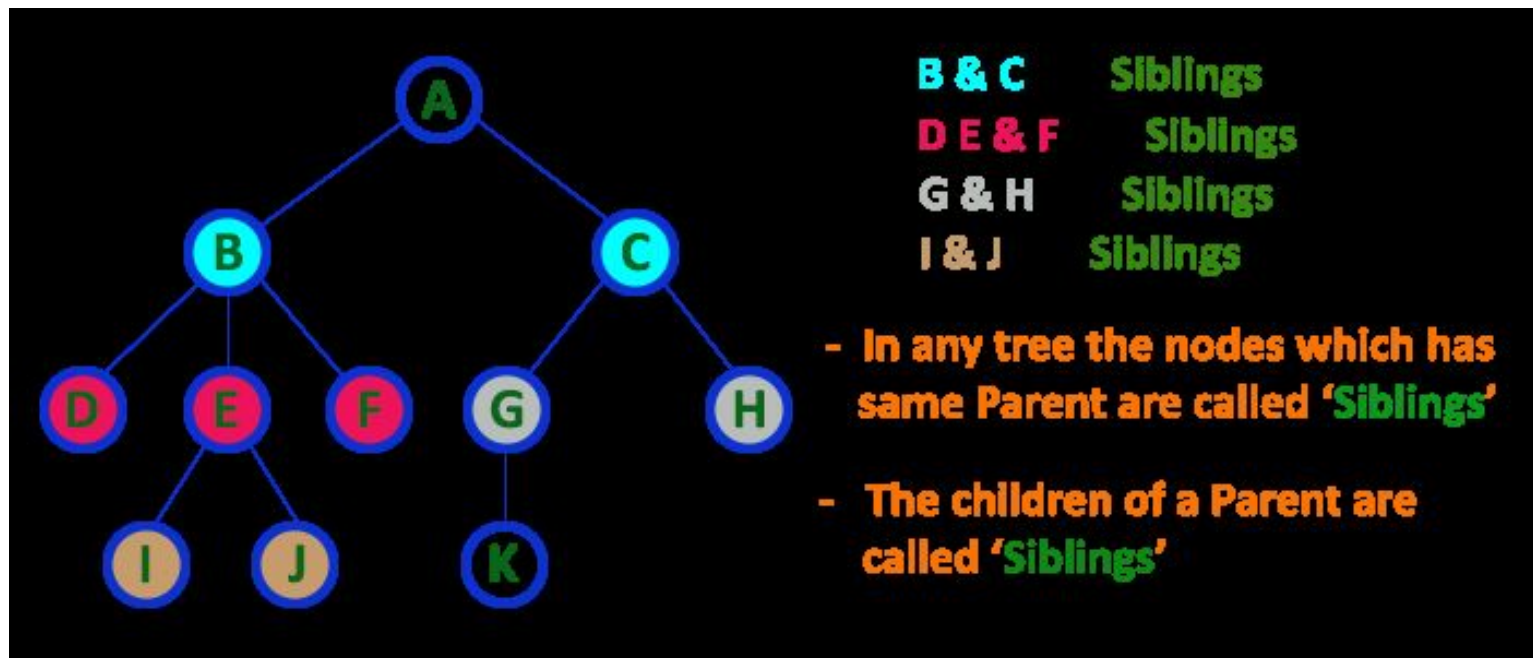
• 4. Child

- The node which has a link from its parent node is called as child node.
- In a tree, any parent node can have any number of child nodes.
- In a tree, all the nodes except root are child nodes.



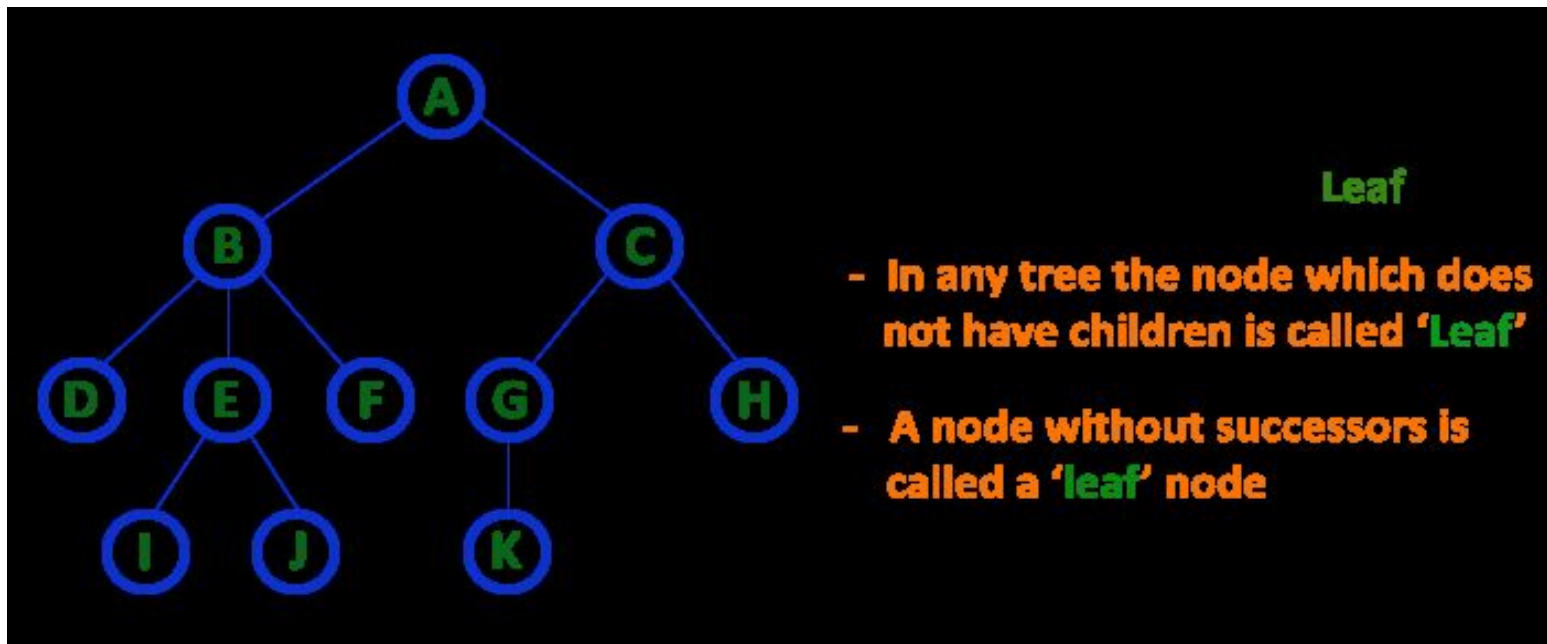
• 5. Siblings

- The nodes with same parent are called as Sibling nodes.



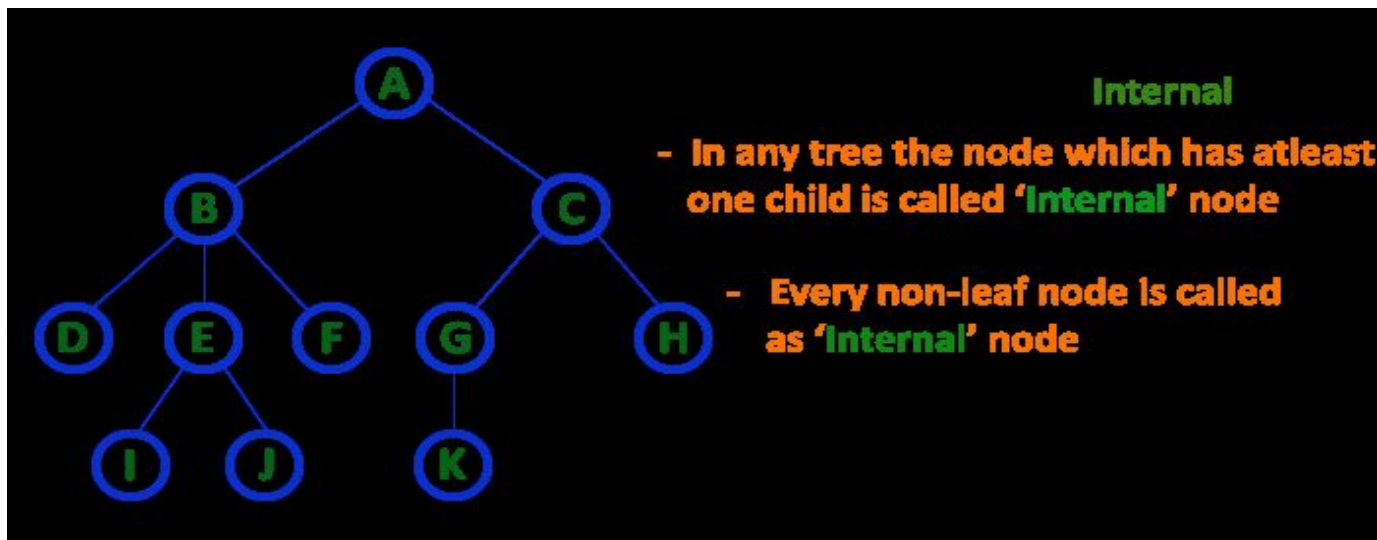
• 6. Leaf Node

- The node which does not have a child is called as LEAF Node.
- The leaf nodes are also called as External Nodes or 'Terminal' node.



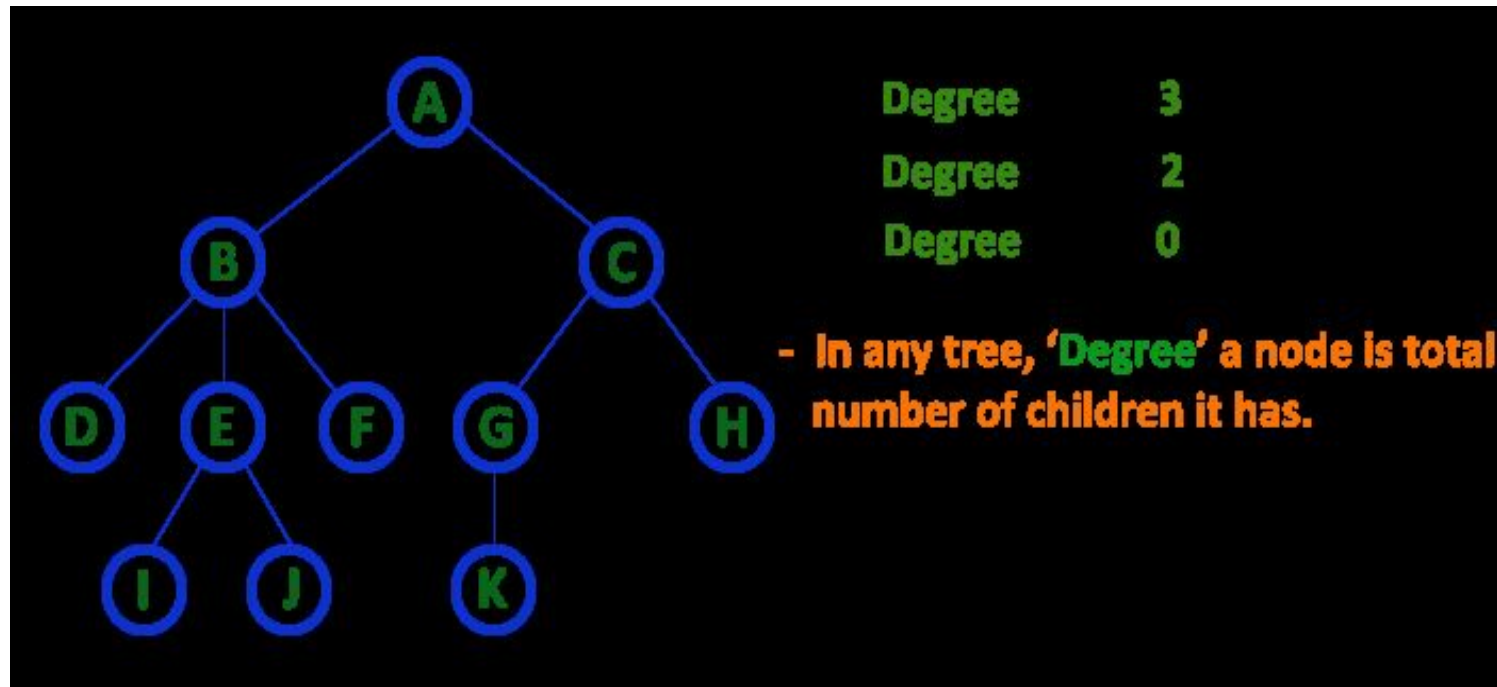
• 7. Internal Nodes

- An internal node is a node with atleast one child.
- Nodes other than leaf nodes are called as Internal Nodes.
- The root node is also said to be Internal Node if the tree has more than one node. Internal nodes are also called as 'Non-Terminal' nodes.



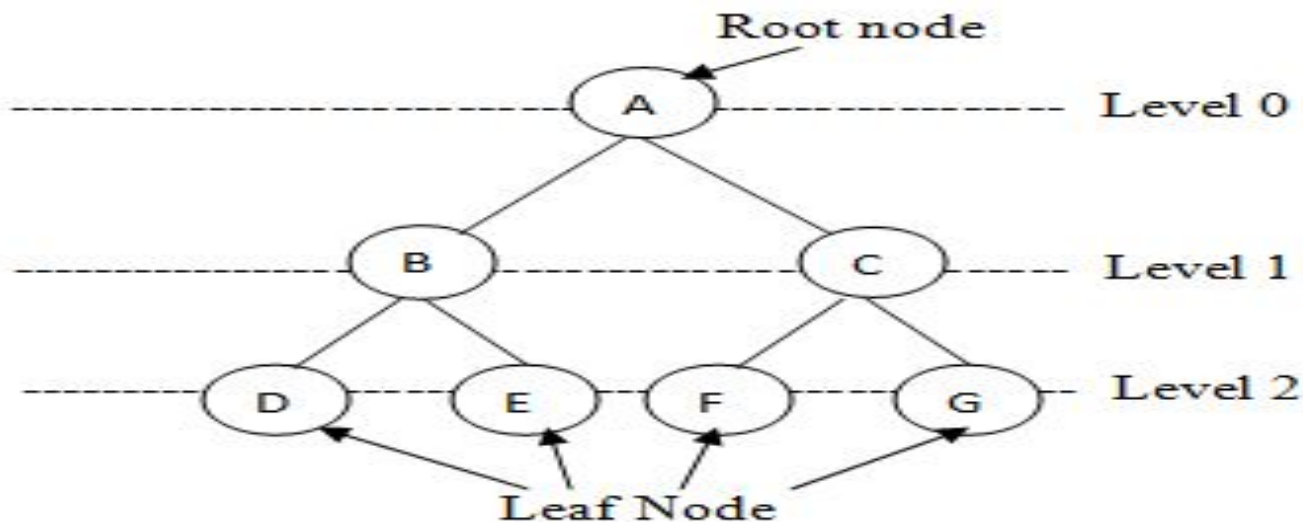
• 8. Degree

- In a tree data structure, the total number of children of a node is called as DEGREE of that Node.



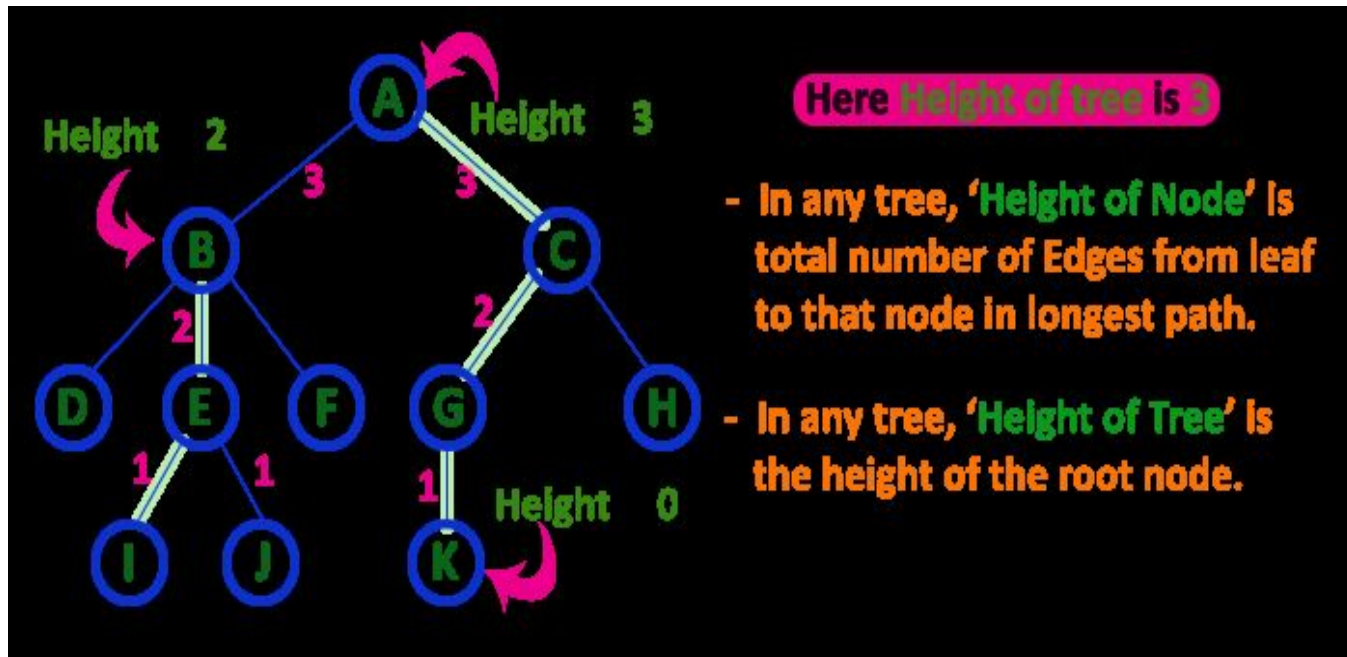
• 9. Level

- In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on...
- In a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step).



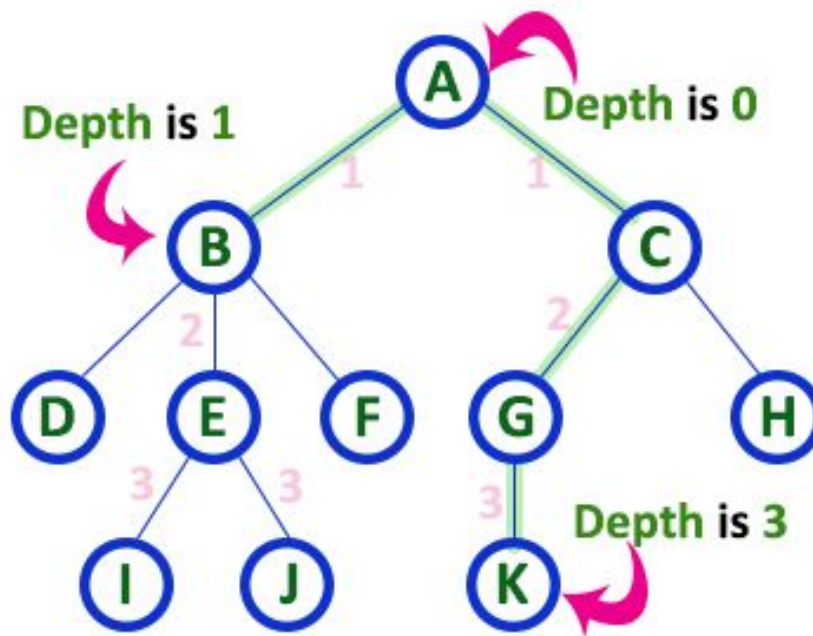
• 10. Height

- the total number of edges from leaf node to a particular node in the longest path is called as HEIGHT of that Node.
- In a tree, height of the root node is said to be height of the tree.



• 11. Depth

- In a tree data structure, the total number of edges from root node to a particular node is called as DEPTH of that Node.

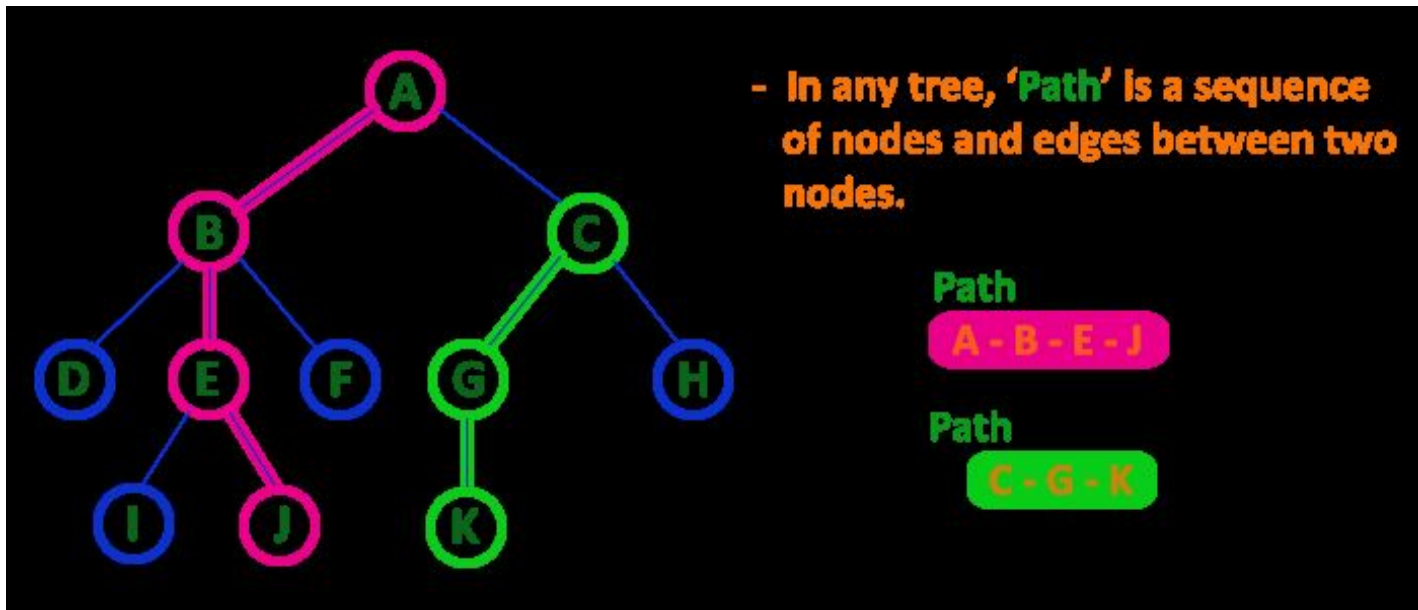


Here Depth of tree is 3

- In any tree, 'Depth of Node' is total number of Edges from root to that node.
- In any tree, 'Depth of Tree' is total number of edges from root to leaf in the longest path.

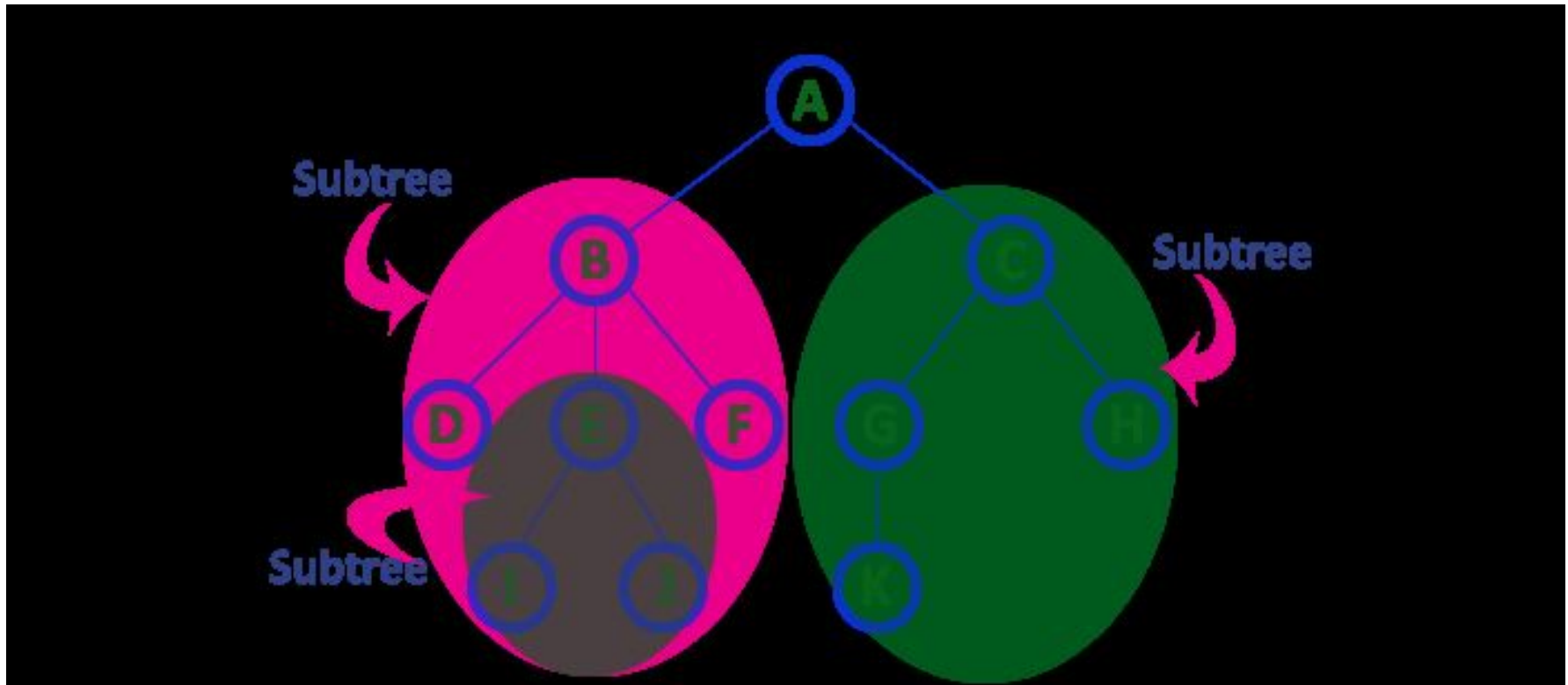
• 12. Path

- In a tree data structure, the sequence of Nodes and Edges from one node to another node is called as PATH between that two Nodes.
- Length of a Path is total number of nodes in that path. In below example the path A - B - E - J has length 4.



• 13. Sub Tree

- Every child node will form a subtree on its parent node.

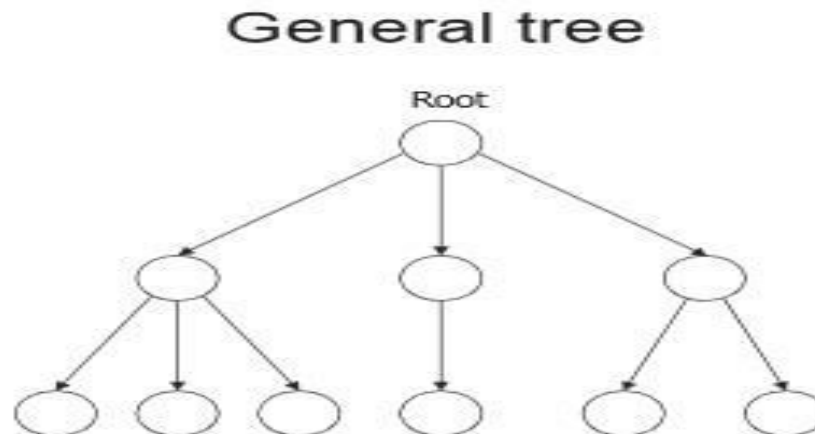


Type of Trees

- General tree
- Binary tree
- Binary Search Tree

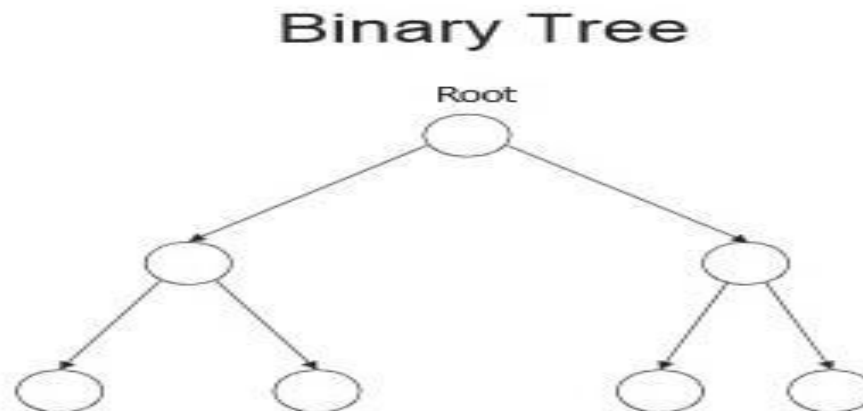
General Tree

- A general tree is a **data structure** in that each node can have infinite number of children .
- In general tree, root has **in-degree 0** and maximum **out-degree n**.
- **Height** of a general tree is the length of longest path from root to the leaf of tree. $\text{Height}(T) = \{\max(\text{height}(\text{child1}) , \text{height}(\text{child2}) , \dots \text{height}(\text{child-n})) + 1\}$



Binary tree

- A Binary tree is a data structure in that each node has at most **two nodes** left and right
- In binary tree, root has **in-degree 0** and maximum **out-degree 2**.
- In binary tree, each node have in-degree **one** and maximum out-degree **2**.
- Height of a binary tree is : $\text{Height}(T) = \{ \max(\text{Height}(\text{Left Child}), \text{Height}(\text{Right Child}) + 1) \}$



Representation of Binary Tree

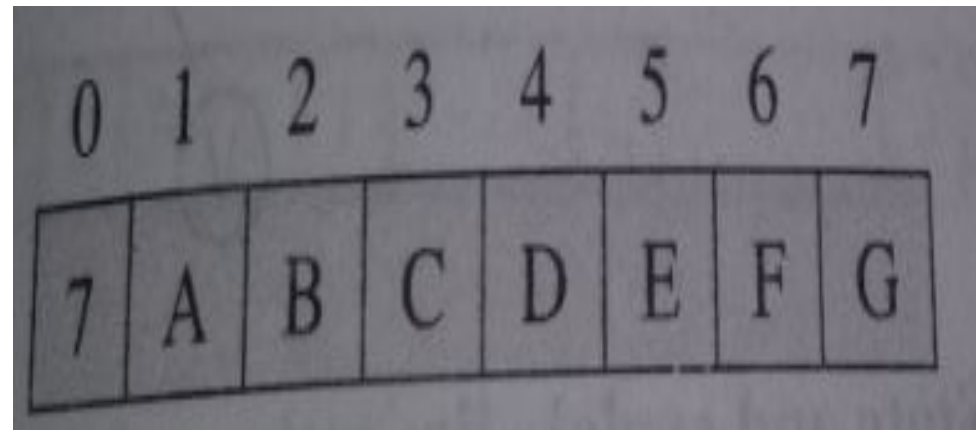
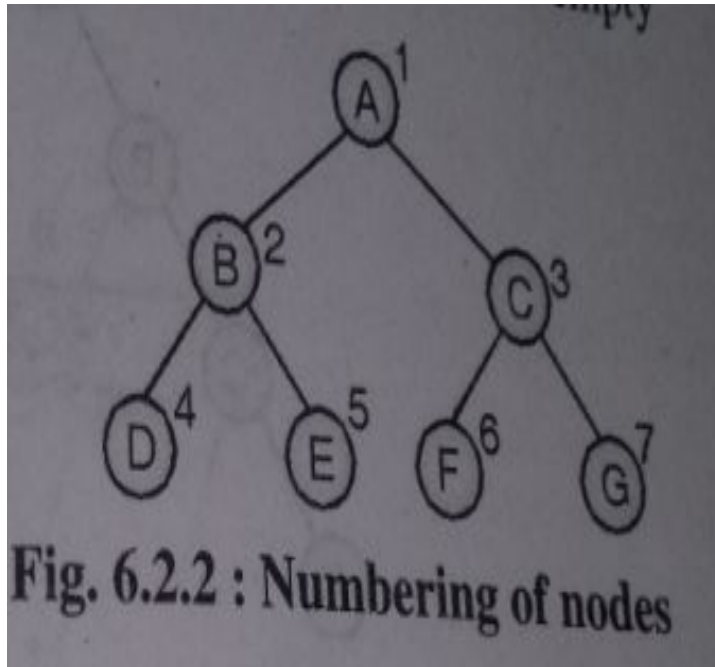
1. Array Representation
2. Linked List Representation.

Representation of Binary Tree

```
Struct node  
{  
  
    int data;  
    struct node * left, right;  
};
```

Array Representation

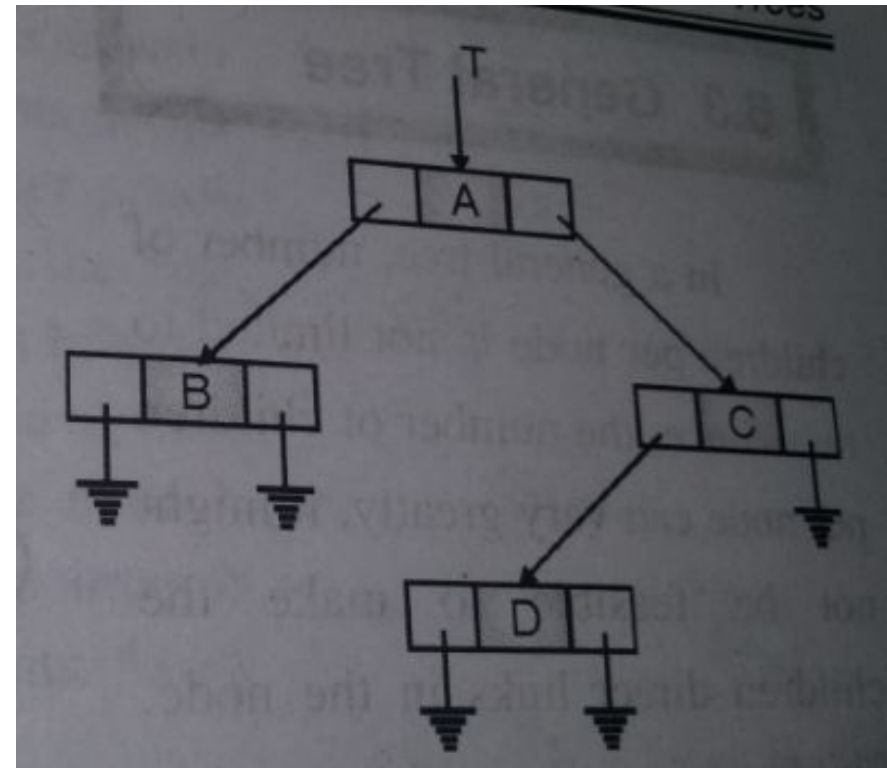
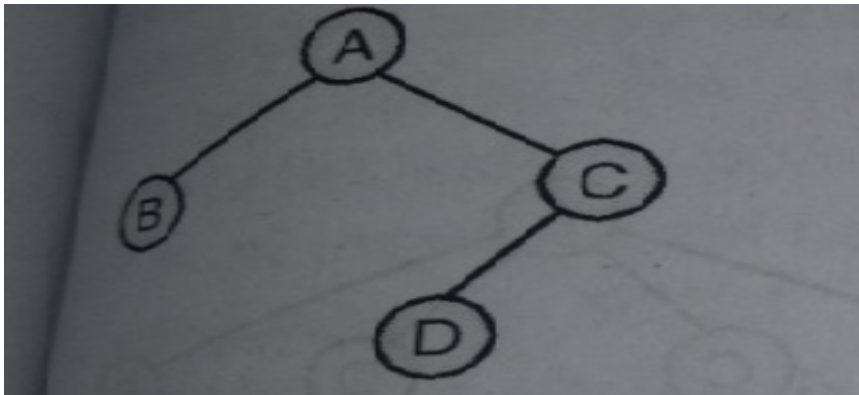
1. To represent a tree in one dimensional array nodes are marked sequentially from left to right start with root node.
2. First array location can be used to store no of nodes in a tree.



Linked Representation

1. This type of representation is more efficient as compared to array.
2. Left and right are pointer type fields left holds address of left child and right holds address of right child.
3. *Struct node*
{

```
int data;  
struct node * left,*right;  
};
```

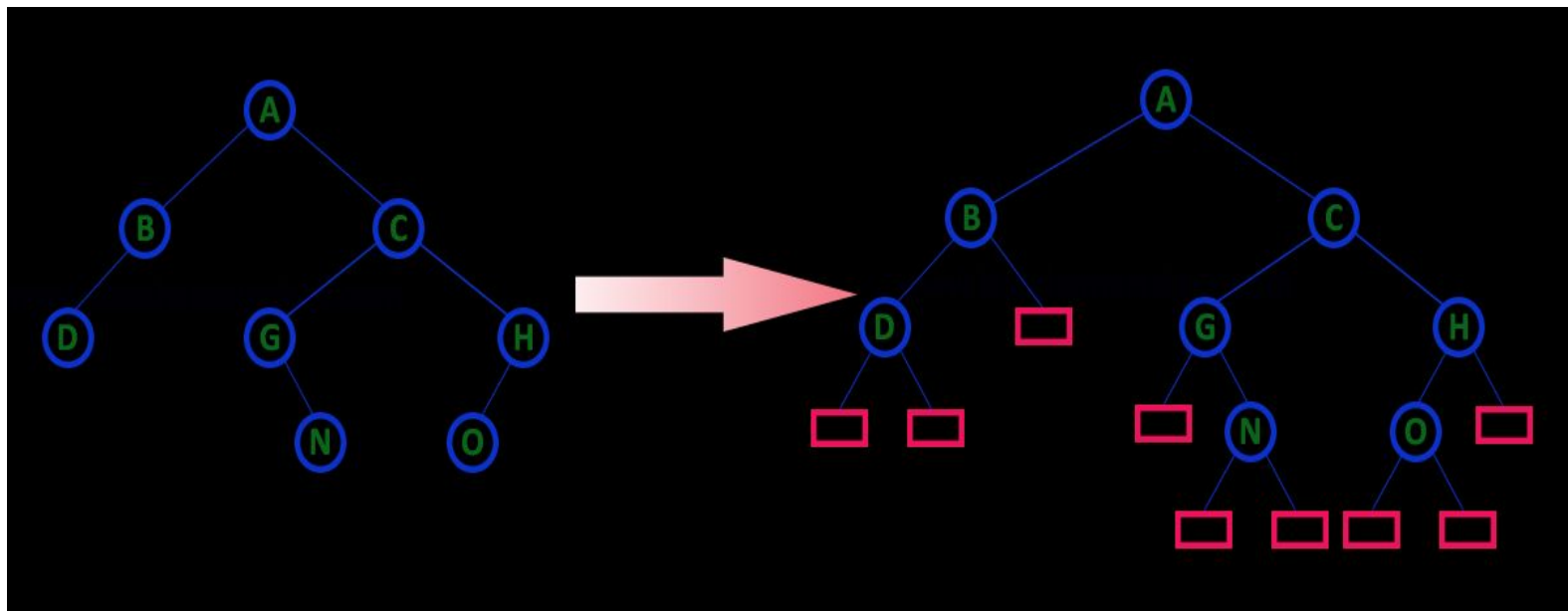


Binary Tree Types

1. Extended Binary Tree
2. Complete Binary Tree
3. Full Binary Tree
4. Skewed Binary Tree
5. Strictly Binary Tree
6. Expression Binary tree

1. Extended Binary Tree

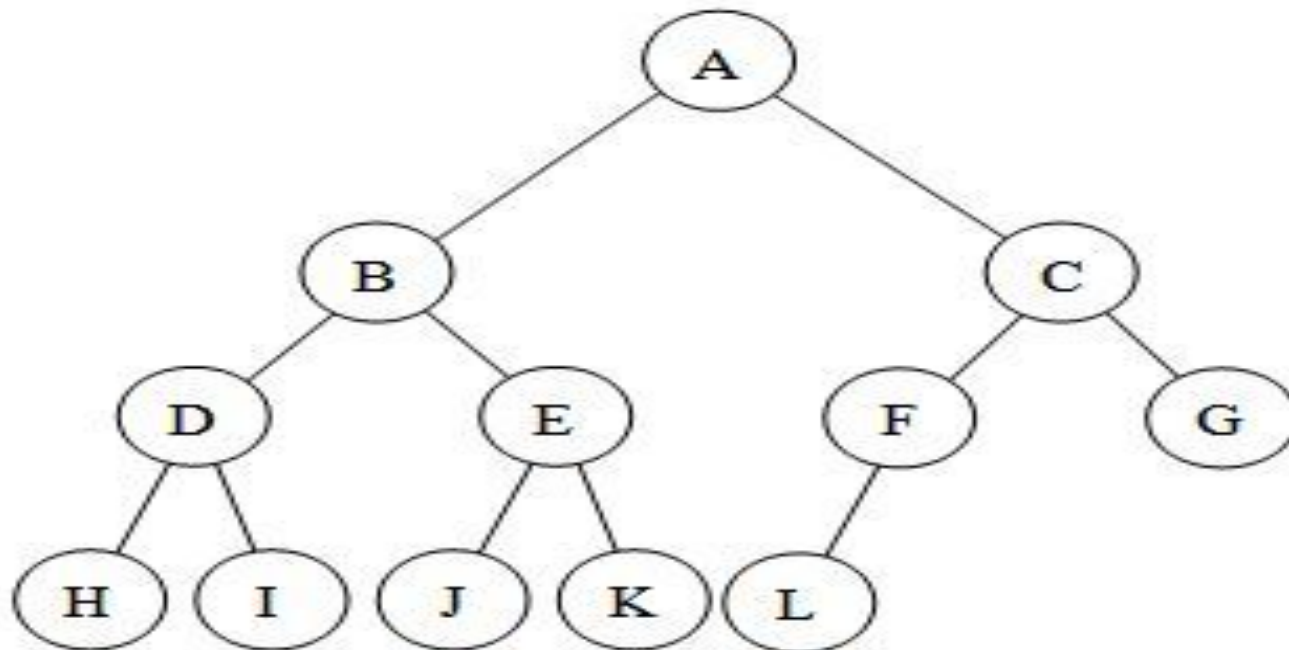
An **extended binary tree** is a transformation of any **binary tree** into a complete **binary tree**. This transformation consists of replacing every null subtree of the original **tree** with “special nodes” or “failure nodes”. The nodes from the original **tree** are then internal nodes, while the “special nodes” are external nodes.



Complete Binary Tree

A **complete binary tree** is a **tree** in which

1. All leaf nodes are at n or $n-1$ level
2. Levels are filled from left to right

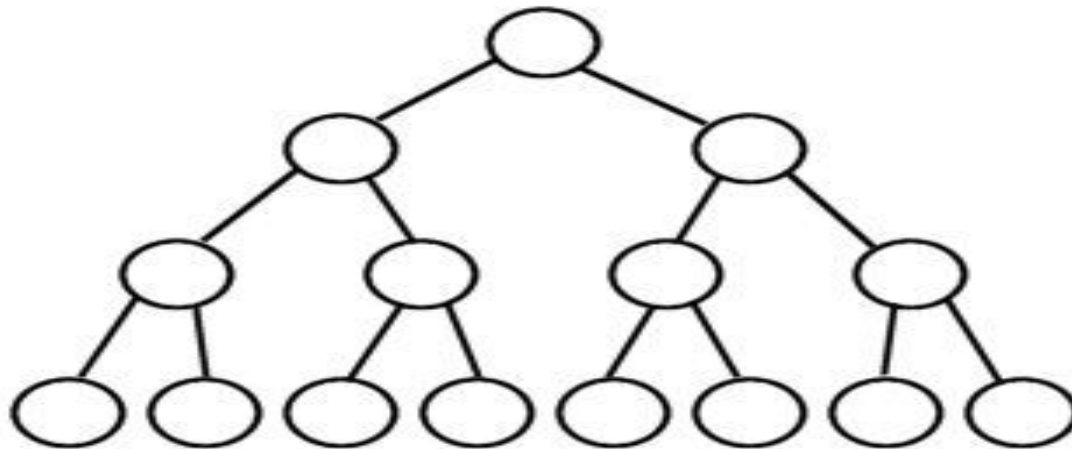


Full Binary Tree

A **full binary tree** (sometimes proper **binary tree** or **2-tree**) is a **tree** in which every node other than the leaves has two children or no children. Every level is completely filled up.

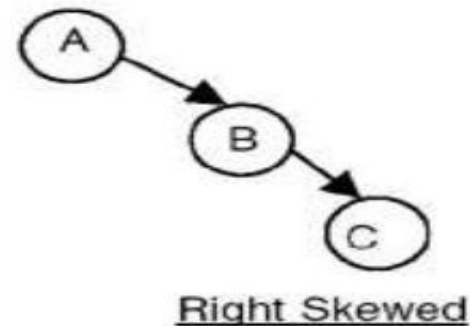
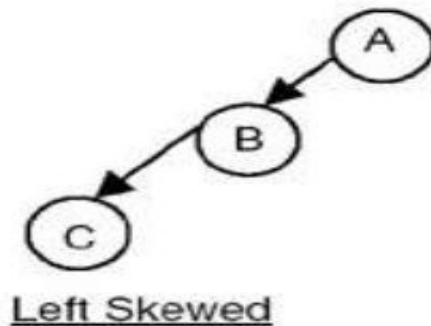
No of nodes = $2^{h+1} - 1$

Full Binary Tree



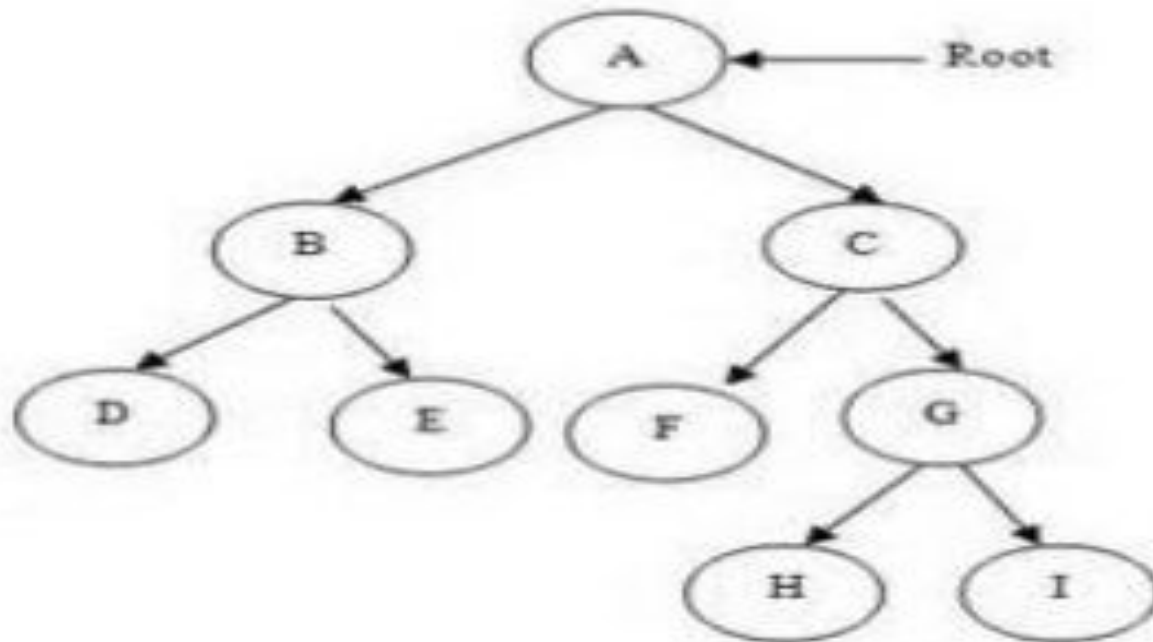
Skewed Binary Tree

A binary tree is said to be *Skewed Binary Tree* if every node in the tree contains either only left or only right sub tree. If the node contains only left sub tree then it is called *left-skewed binary tree* and if the tree contains only right sub tree then it is called *right-skewed binary tree*.



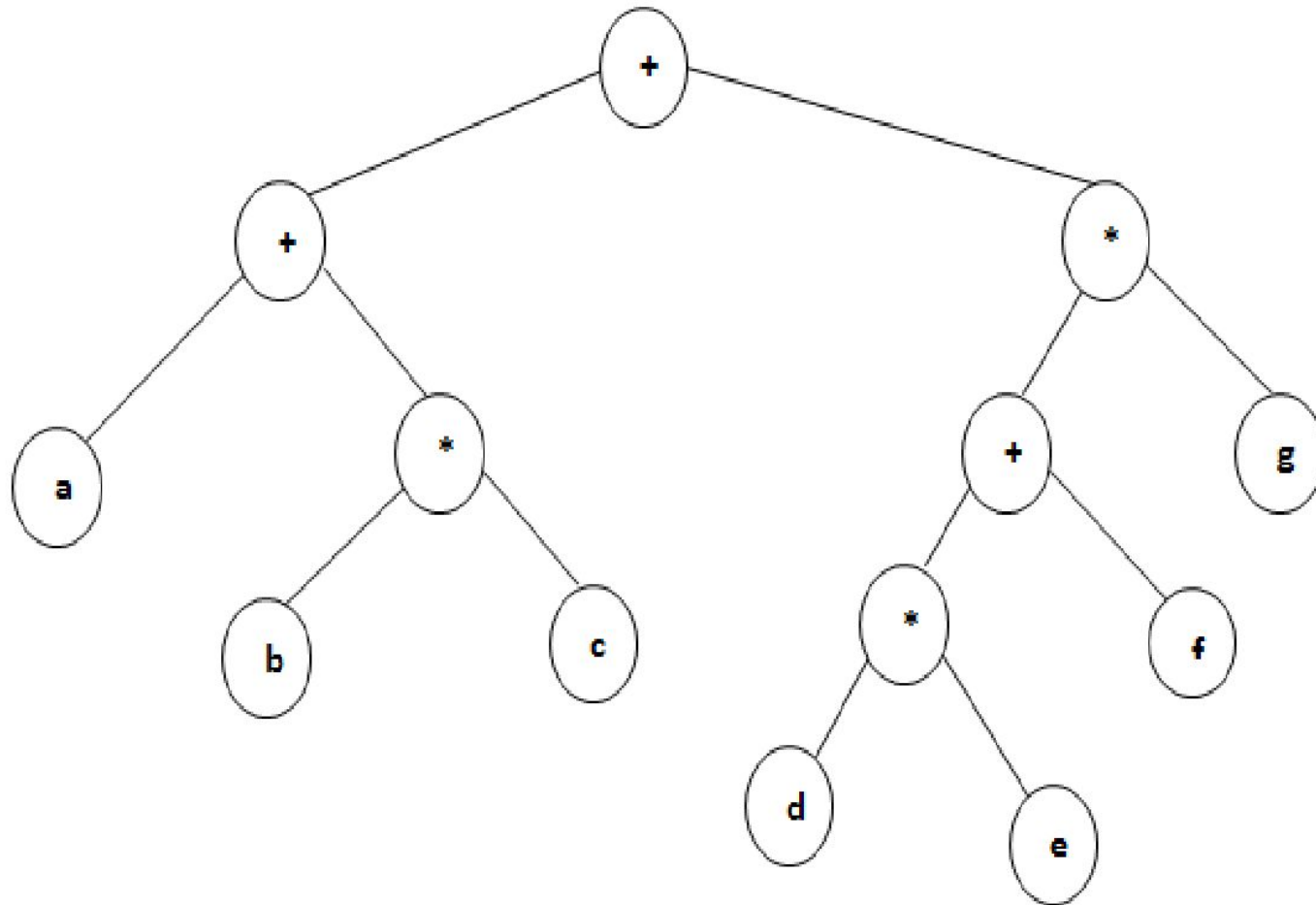
Strictly Binary Tree

A node will have either two children or no child at all.



Expression Binary tree

- Expression trees are a special kind of binary tree used to evaluate certain expressions.
- Two common types of expressions that a binary expression tree can represent are algebraic and boolean.
- These trees can represent expressions that contain both unary and binary operators.
- The leaves of a binary expression tree are operands, such as constants or variable names, and the other nodes contain operators.
- Expression tree are used in most compilers.



<https://www.youtube.com/watch?v=NGEKaXpGAT8>

Binary Tree Traversal

1.Preorder traversal:-

In this traversal method first process root element, then left sub tree and then right sub tree.

Procedure:-

Step 1: Visit root node

Step 2: Visit left sub tree in preorder

Step 3: Visit right sub tree in preorder

2. Inorder traversal:-

In this traversal method first process left element, then root element and then the right element.

Procedure:-

Step 1: Visit left sub tree in inorder

Step 2: Visit root node

Step 3: Visit right sub tree in inorder

3. Postorder traversal:-

In this traversal first visit / process left sub tree, then right sub tree and then the root element.

Procedure:-

Step 1: Visit left sub tree in postorder

Step 2: Visit right sub tree in postorder

Step 3: Visit root node

Binary Search Tree

Part-1/2 - <https://www.youtube.com/watch?v=KGcrQj0pqMo>

Part-2/2 - <https://www.youtube.com/watch?v=vzmmOL4Bx4Y>

expression tree: <https://www.youtube.com/watch?v=NGEKaXpGAT8>

Binary Search Tree Traversal:

<https://www.youtube.com/watch?v=UjIiNK6OVDU>

Searching in Binary Search Tree:

<https://www.youtube.com/watch?v=Y5rCIBa5edg>

Insertion in Binary Search Tree:

<https://www.youtube.com/watch?v=BzOgSh7YxuM>

Deletion in Binary Search Tree:

https://www.youtube.com/watch?v=9kF8il8_7ag

Binary Search Tree

A **binary search tree (BST)** or "ordered **binary tree**" is a empty or in which each node contains a key that satisfies following conditions:

1. All keys are distinct.'
2. all elements in its left subtree are less to the node ($<$), **and** all the elements in its right subtree are greater than the node ($>$).

Part-1/2 - <https://www.youtube.com/watch?v=KGcrQj0pqMo>

Part-2/2 - <https://www.youtube.com/watch?v=vzmmOL4Bx4Y>

expression tree: <https://www.youtube.com/watch?v=NGEKaXpGAT8>

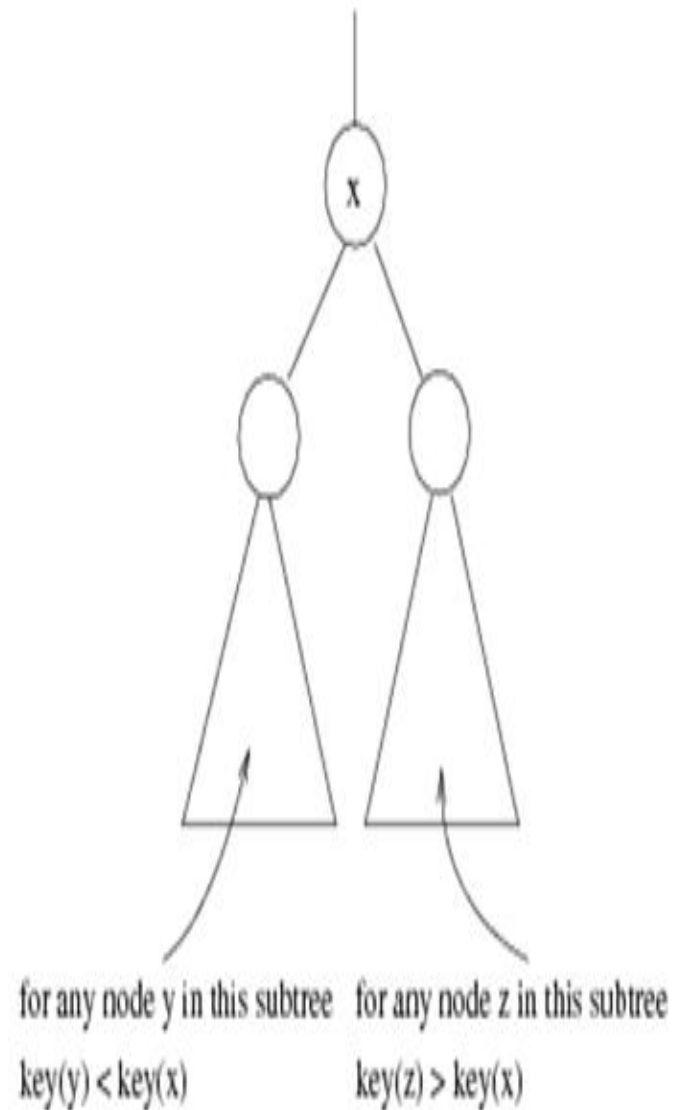
Binary Search Tree Traversal:

<https://www.youtube.com/watch?v=UjIiNK6OVDU>

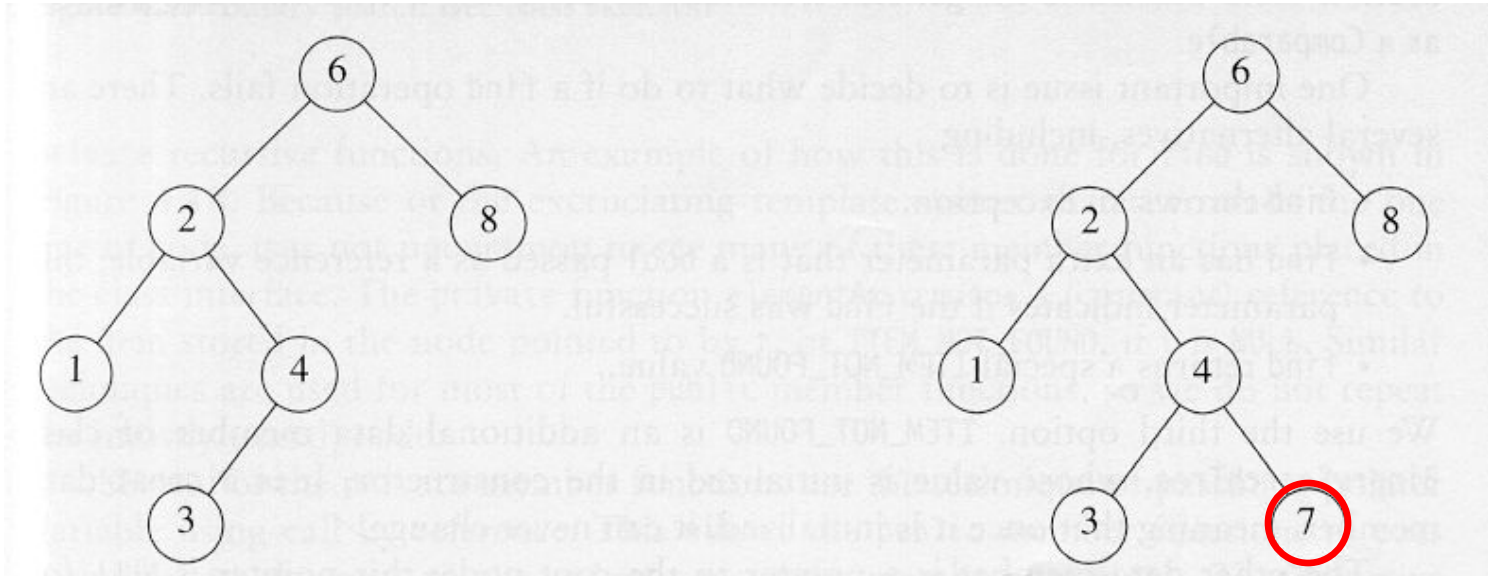
Binary search tree

Binary search tree property

- For every node X
- All the keys in its left subtree are smaller than the key value in X
- All the keys in its right subtree are larger than the key value in X



Binary Search Trees



A binary search tree

Not a binary search tree

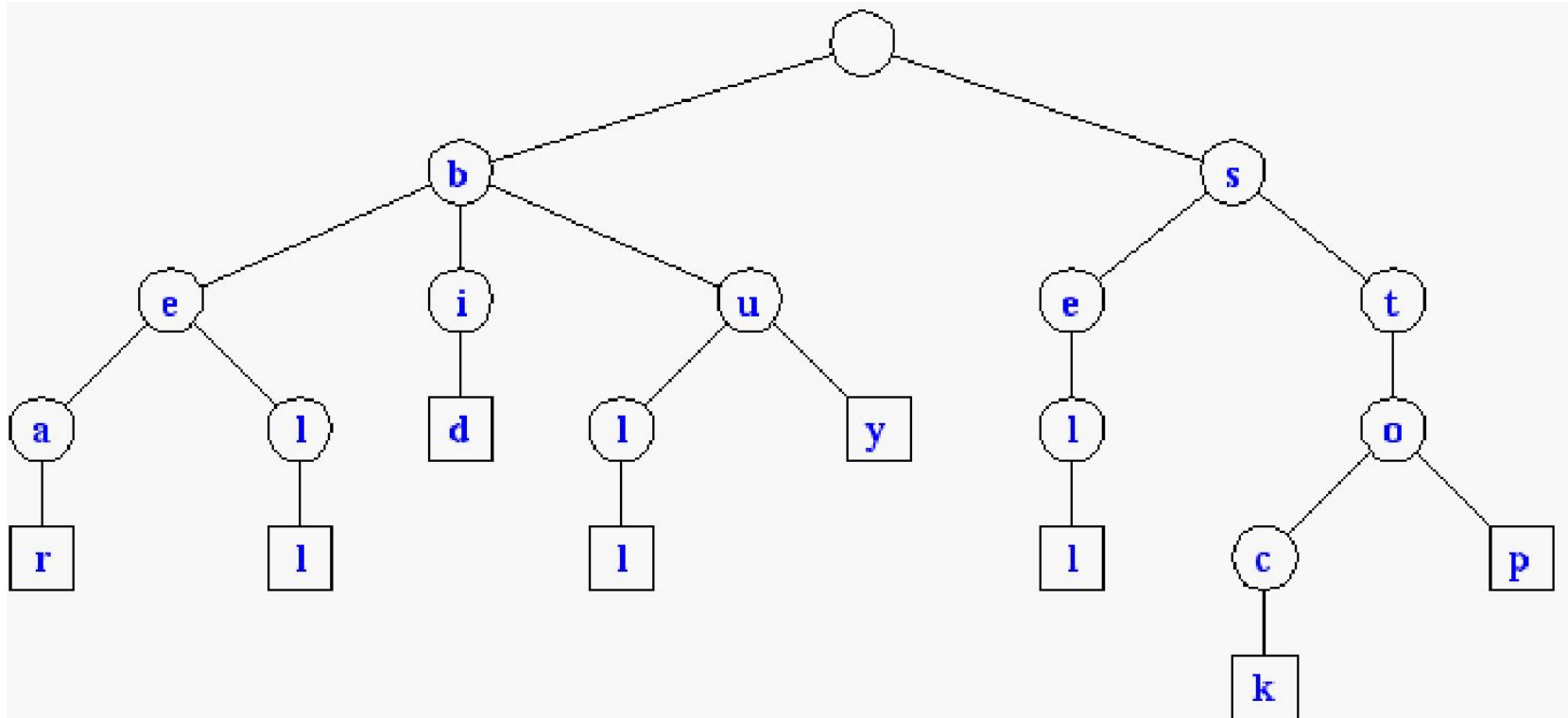
Tries

Definition

- Trie data structure is defined as a Tree based data structure that is used for storing some collection of strings and performing efficient search operations on them.
- The word Trie is derived from reTRIEval, which means finding something or obtaining it.

Tries

- Standard Tries
- Compressed Tries
- Suffix Tries



Text Processing

- We have seen that preprocessing the pattern speeds up pattern matching queries
- After preprocessing the pattern in time proportional to the pattern length, the Boyer-Moore algorithm searches an arbitrary English text in (average) time **proportional to the text length**
- If the text is large, immutable and searched for often (e.g., works by Shakespeare), we may want to preprocess the text instead of the pattern in order to perform pattern matching queries in time **proportional to the pattern length**.
- Tradeoffs in text searching

	Preprocess Pattern	Preprocess Text	Space	Search Time
Brute Force			$O(1)$	$O(mn)$
Boyer Moore	$O(m+d)$		$O(d)$	$O(n)$ *
Suffix Trie		$O(n)$	$O(n)$	$O(m)$

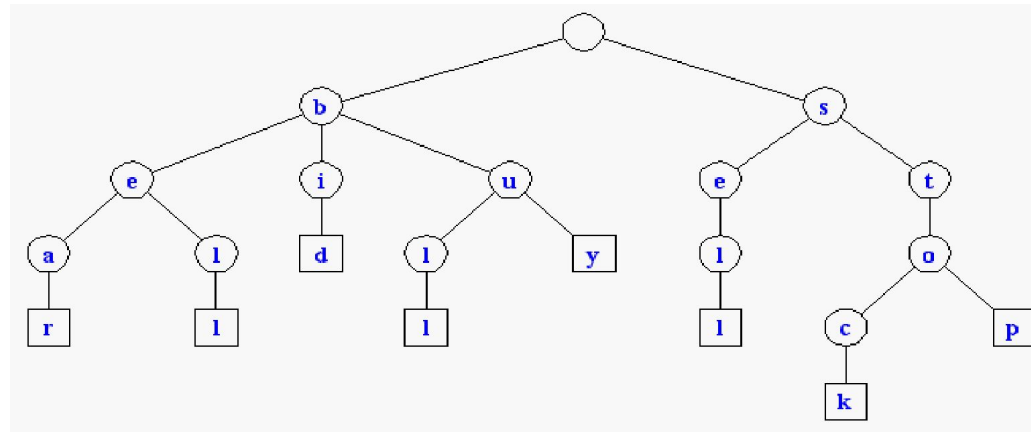
n = text size

m = pattern size

* on average

Standard Tries

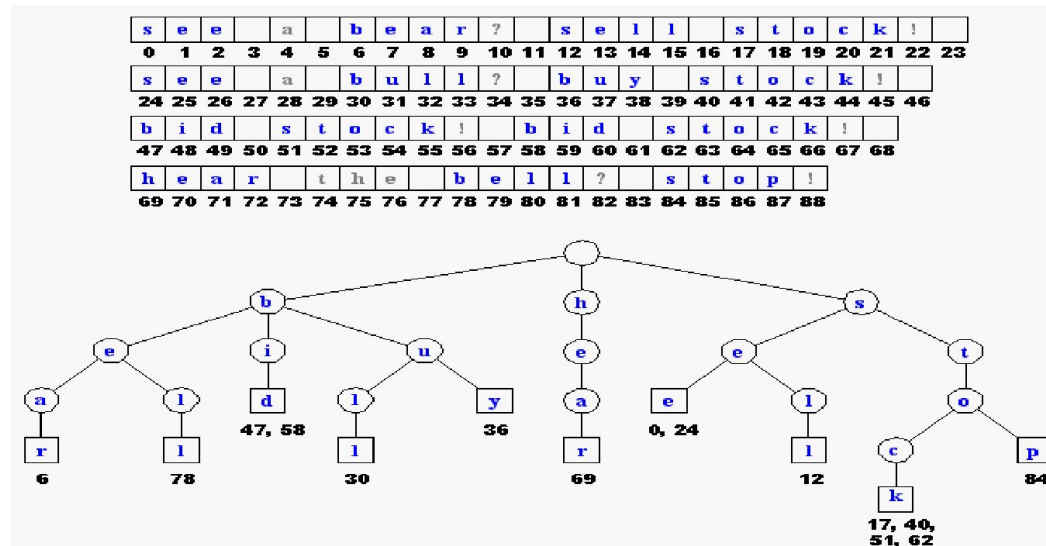
- The **standard trie** for a set of strings S is an ordered tree such that:
 - each node but the root is labeled with a character
 - the children of a node are alphabetically ordered
 - the paths from the external nodes to the root yield the strings of S
- Example: standard trie for the set of strings
 $S = \{ \text{bear, bell, bid, bull, buy, sell, stock, stop} \}$



- A standard trie uses $O(n)$ space. Operations (find, insert, remove) take time $O(dm)$ each, where:
 - n = total size of the strings in S ,
 - m = size of the string parameter of the operation
 - d = alphabet size,

Applications of Tries

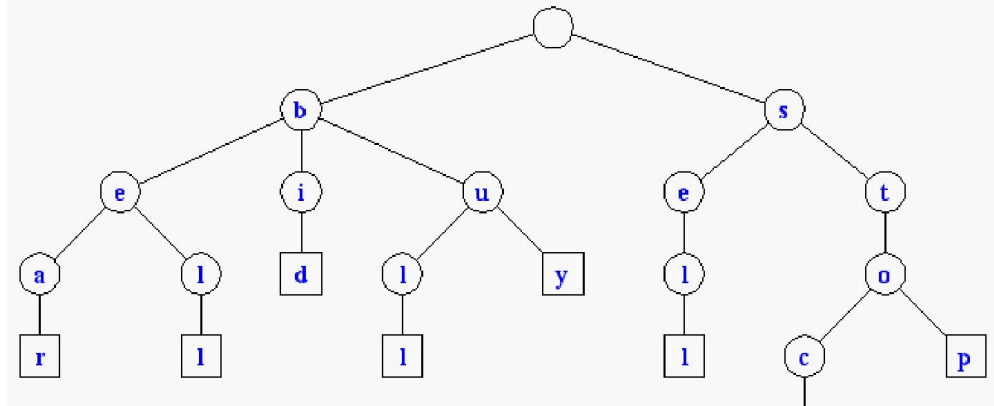
- A standard trie supports the following operations on a preprocessed text in time $O(m)$, where $m = |X|$
 - **word matching**: find the first occurrence of word X in the text
 - **prefix matching**: find the first occurrence of the longest prefix of word X in the text
- Each operation is performed by tracing a path in the trie starting at the root



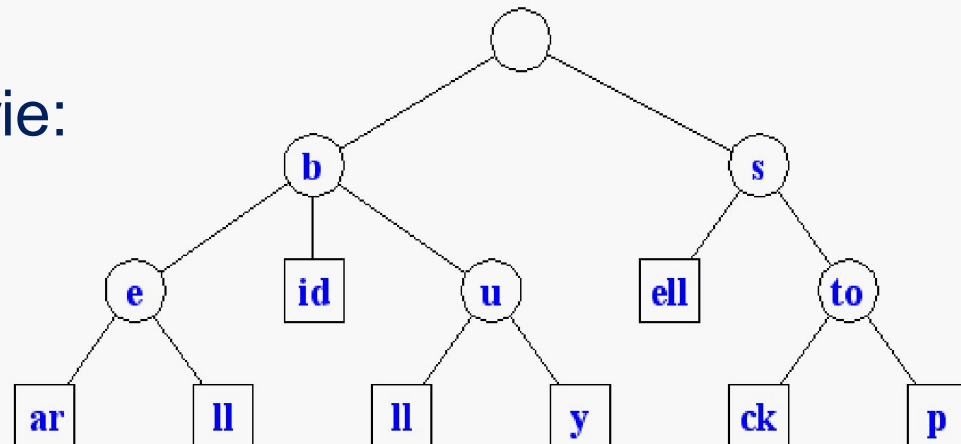
Compressed Tries

- Trie with nodes of degree at least 2
- Obtained from standard trie by compressing chains of *redundant nodes*

Standard Trie:

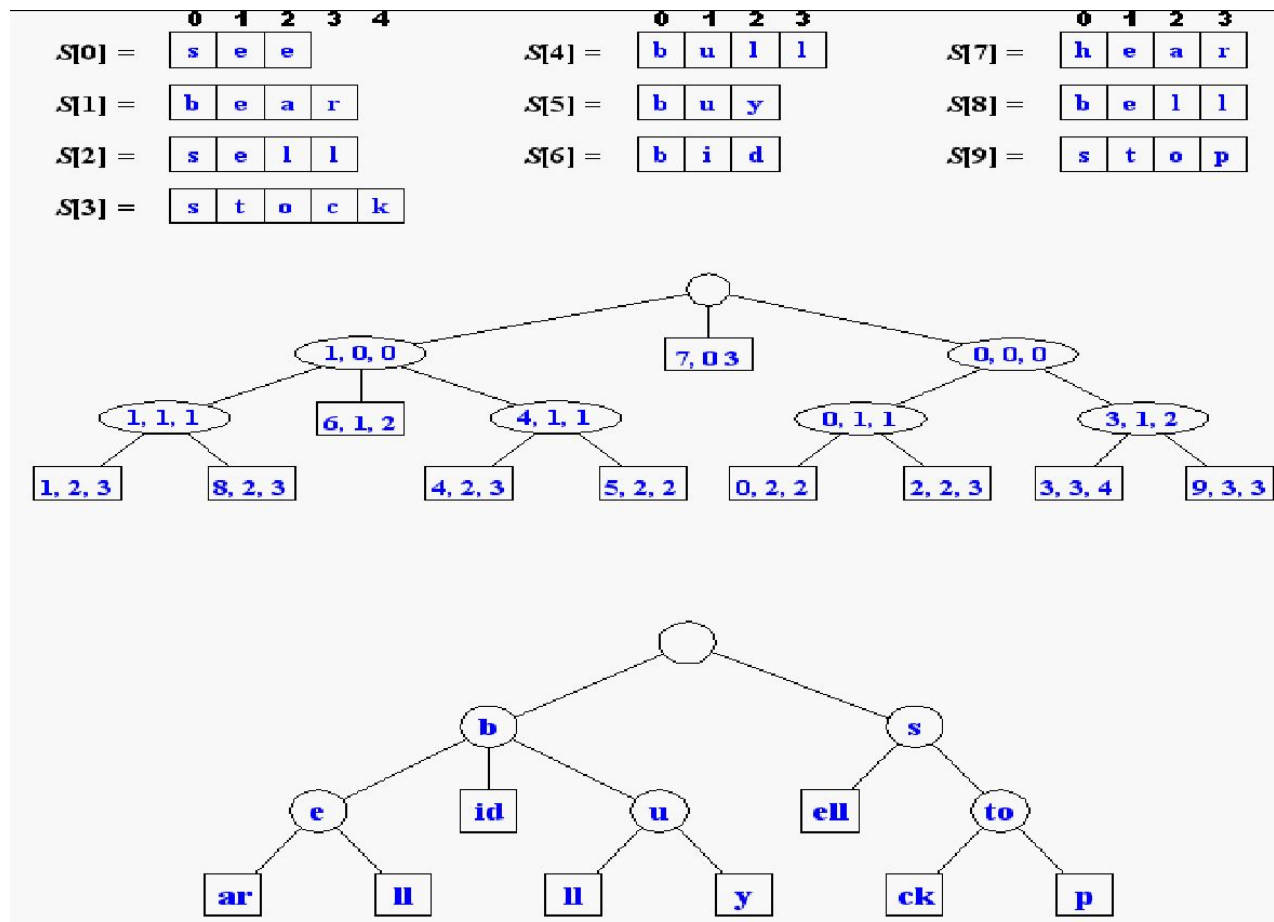


Compressed Trie:

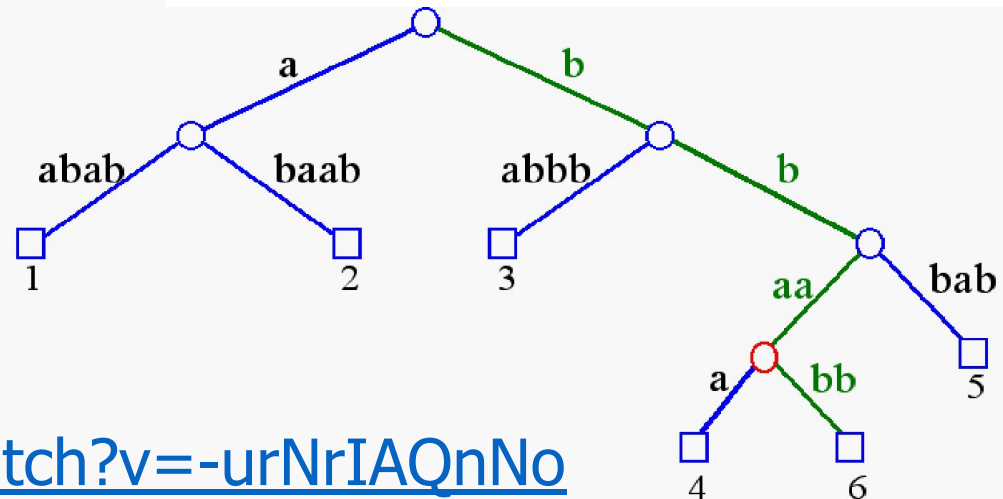
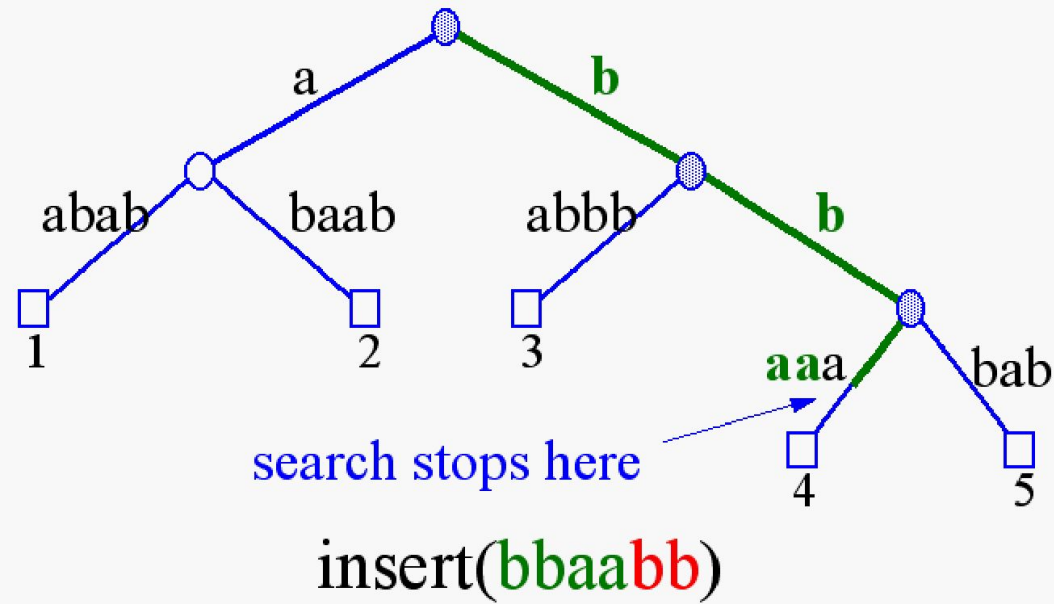


Compact Storage of Compressed Tries

- A compressed trie can be stored in space $O(s)$, where $s = |S|$, by using $O(1)$ space *index ranges* at the nodes



Insertion and Deletion into/from a Compressed Trie

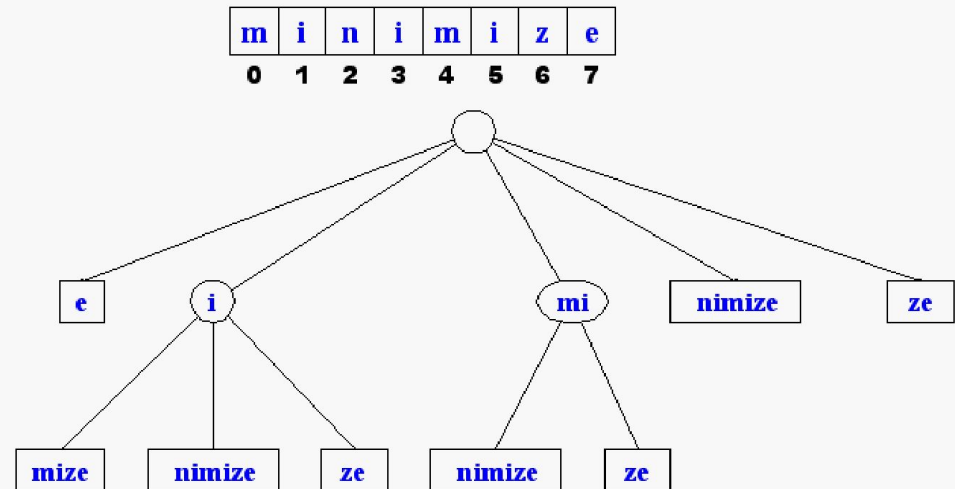


<https://www.youtube.com/watch?v=-urNrIAQnNo>

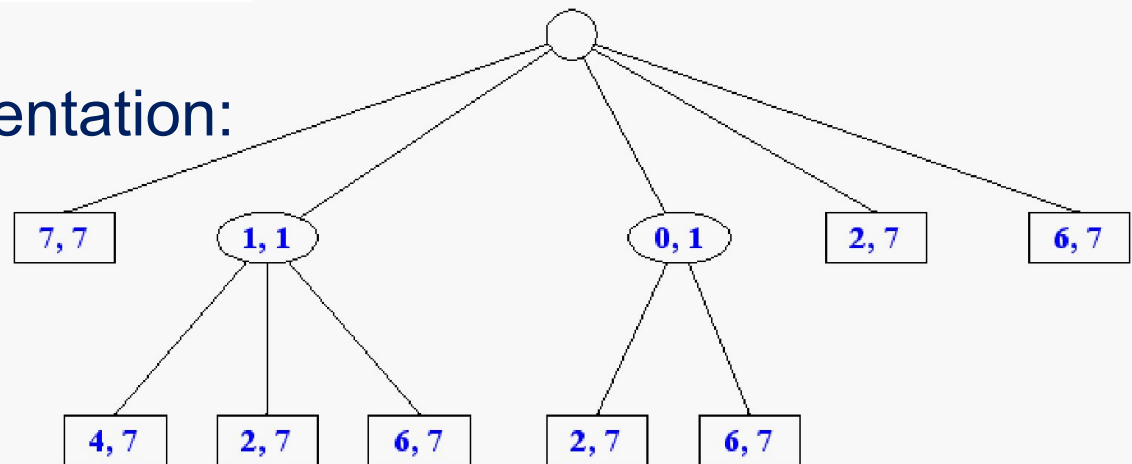
Suffix Tries

- A *suffix trie* is a compressed trie for all the suffixes of a text

Example:

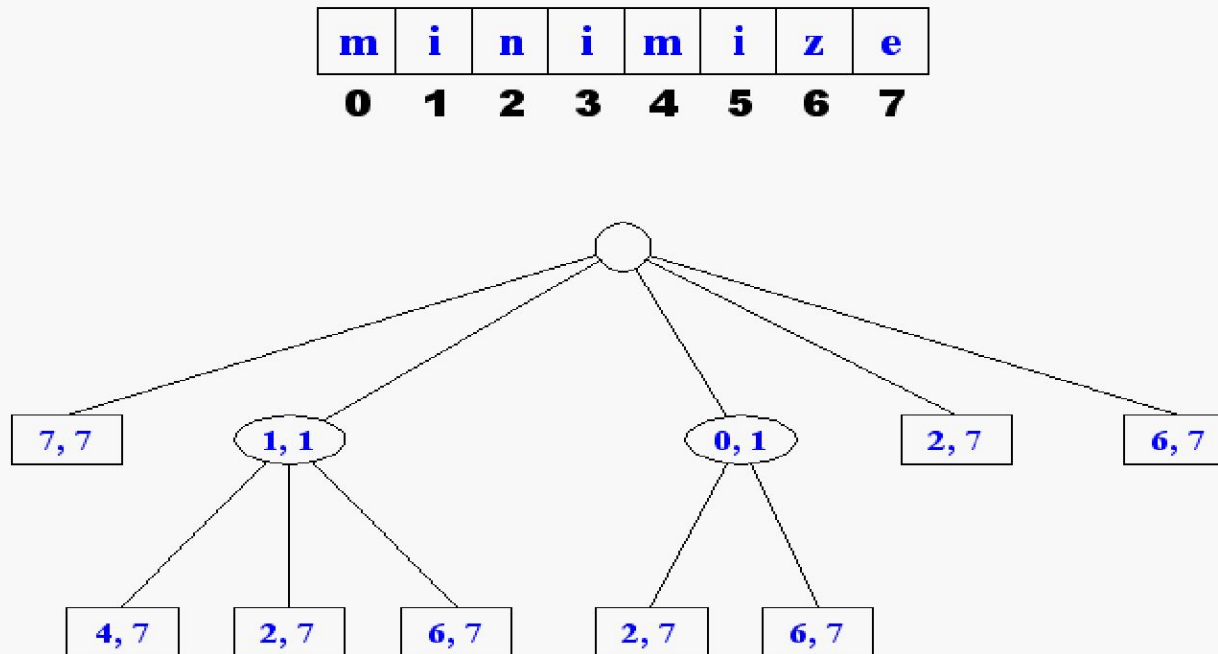


Compact representation:

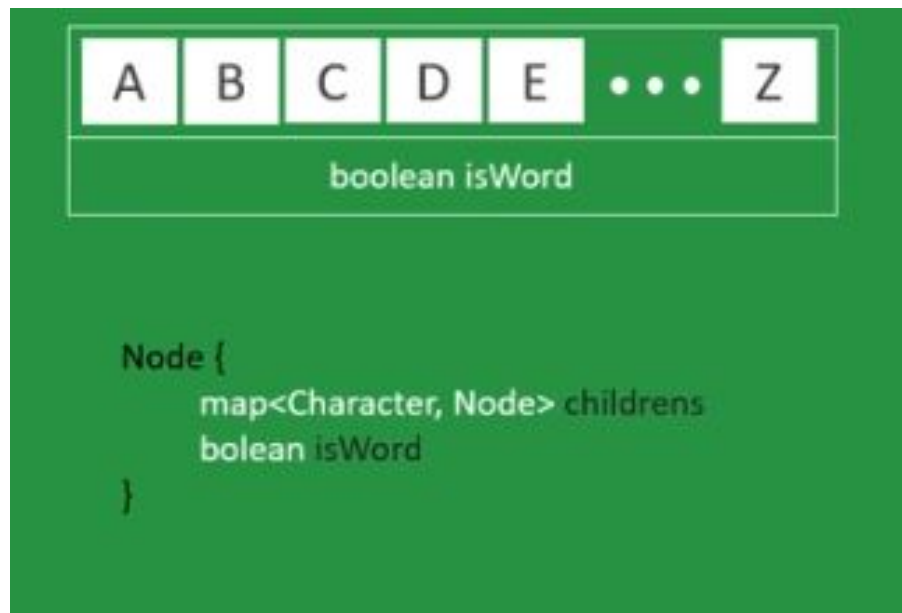
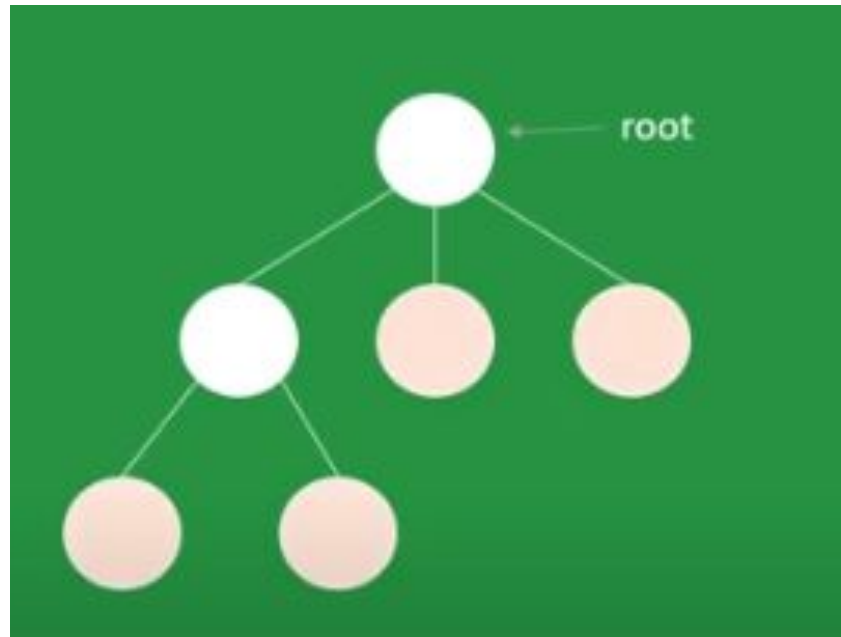


Properties of Suffix Tries

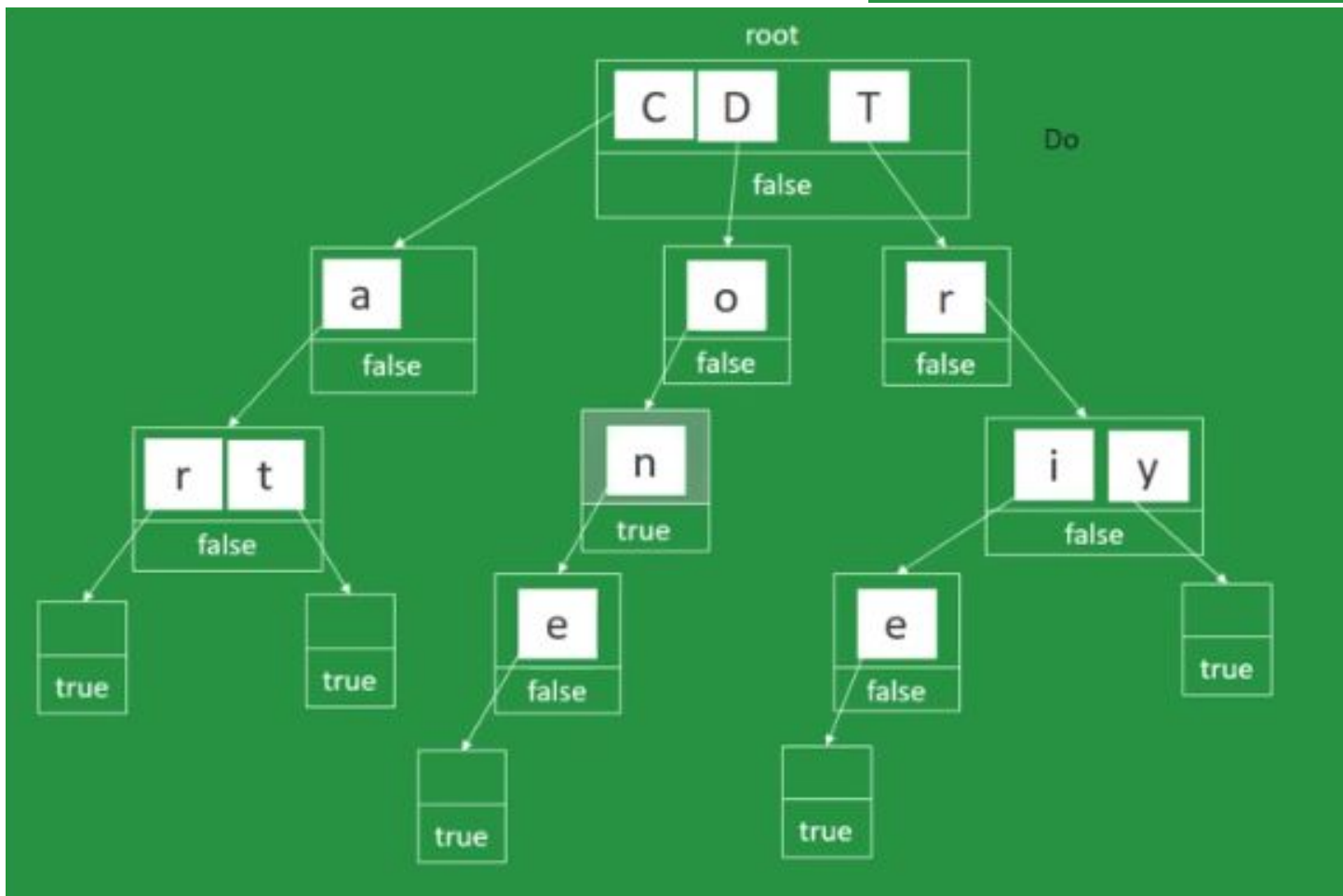
- The **suffix trie** for a text X of size n from an alphabet of size d
 - stores all the $n(n-1)/2$ **suffixes** of X in $O(n)$ space
 - supports arbitrary **pattern matching** and prefix matching queries in **$O(dm)$ time**, where m is the length of the pattern
 - can be constructed in **$O(dn)$ time**



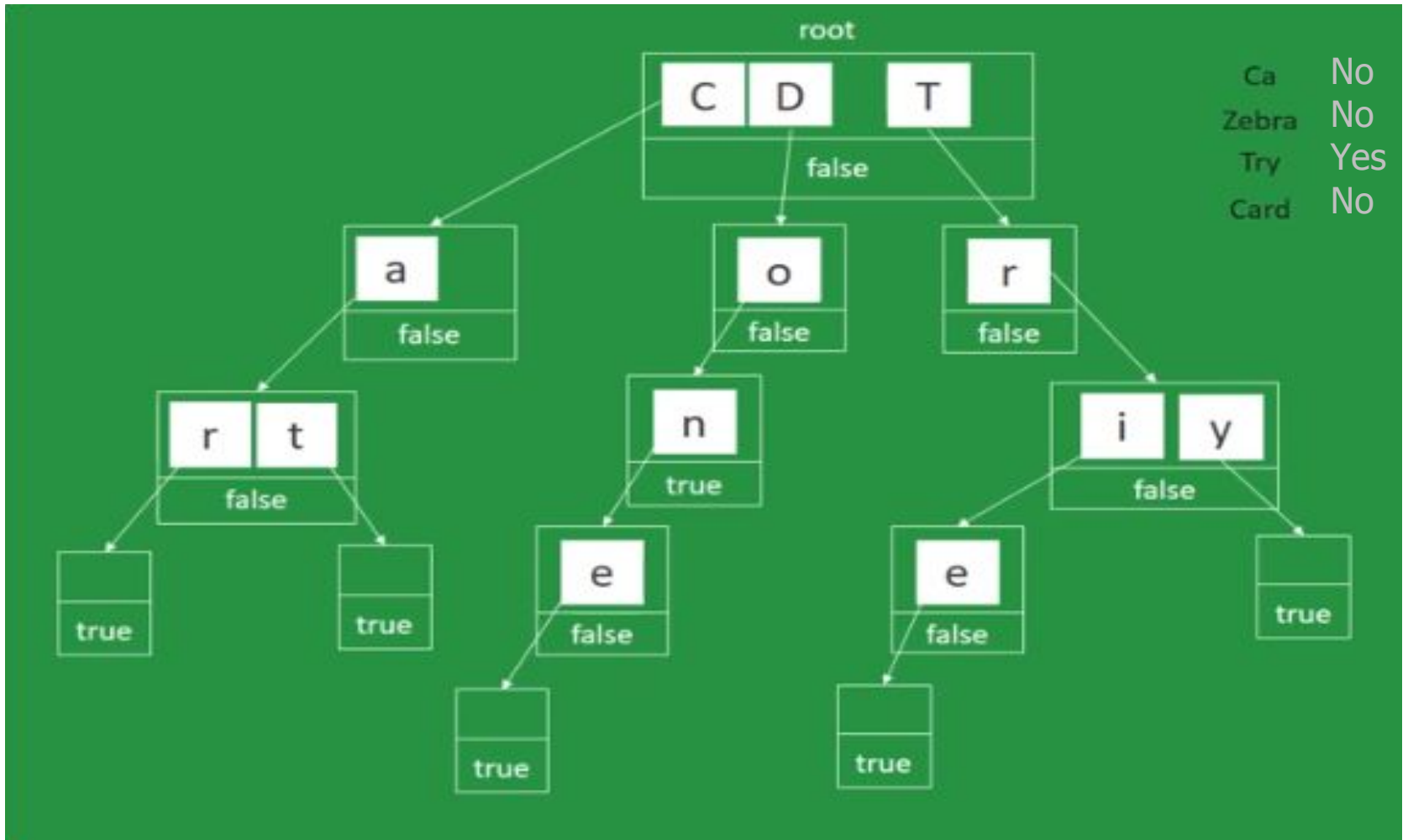
Example



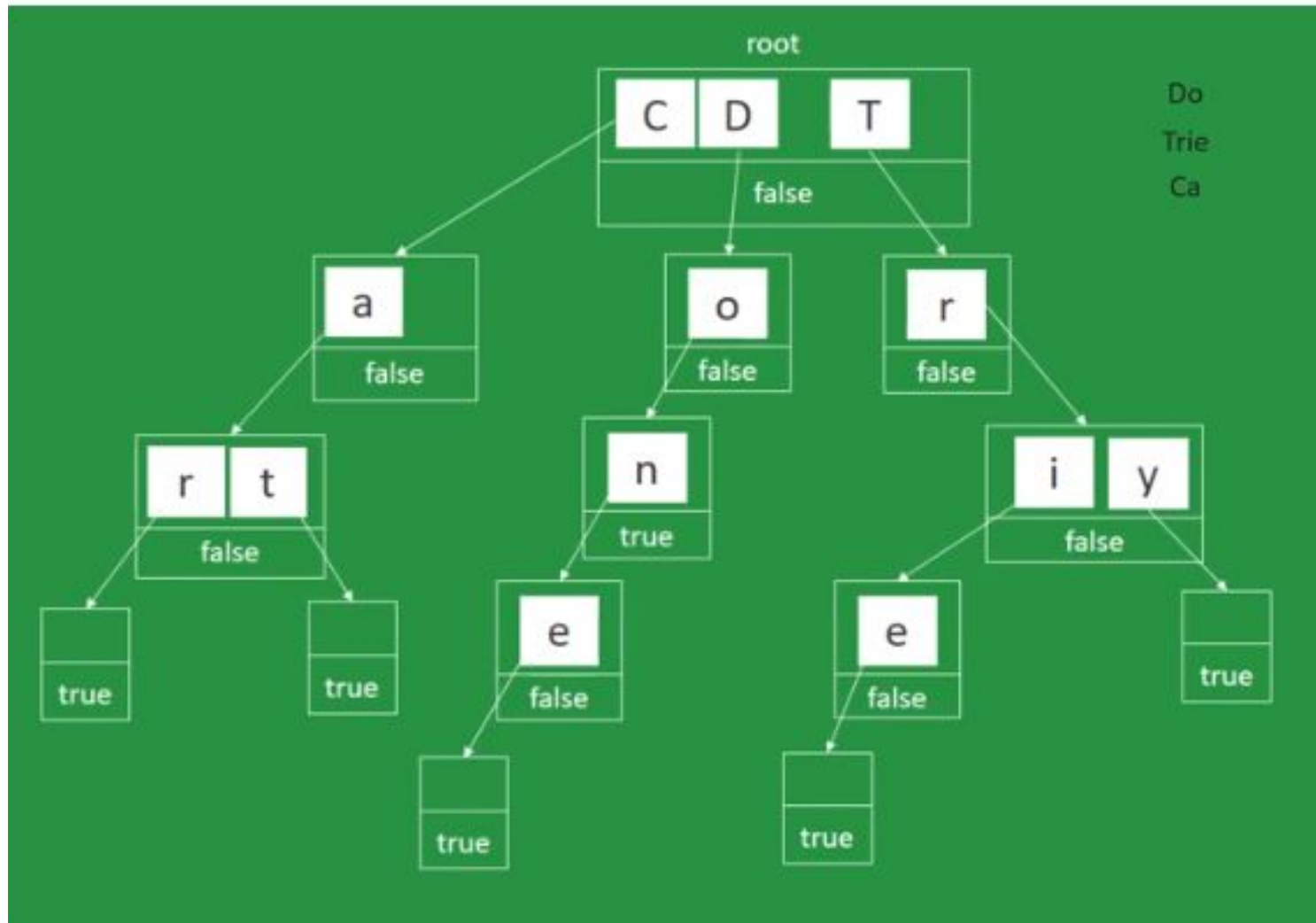
Insertion



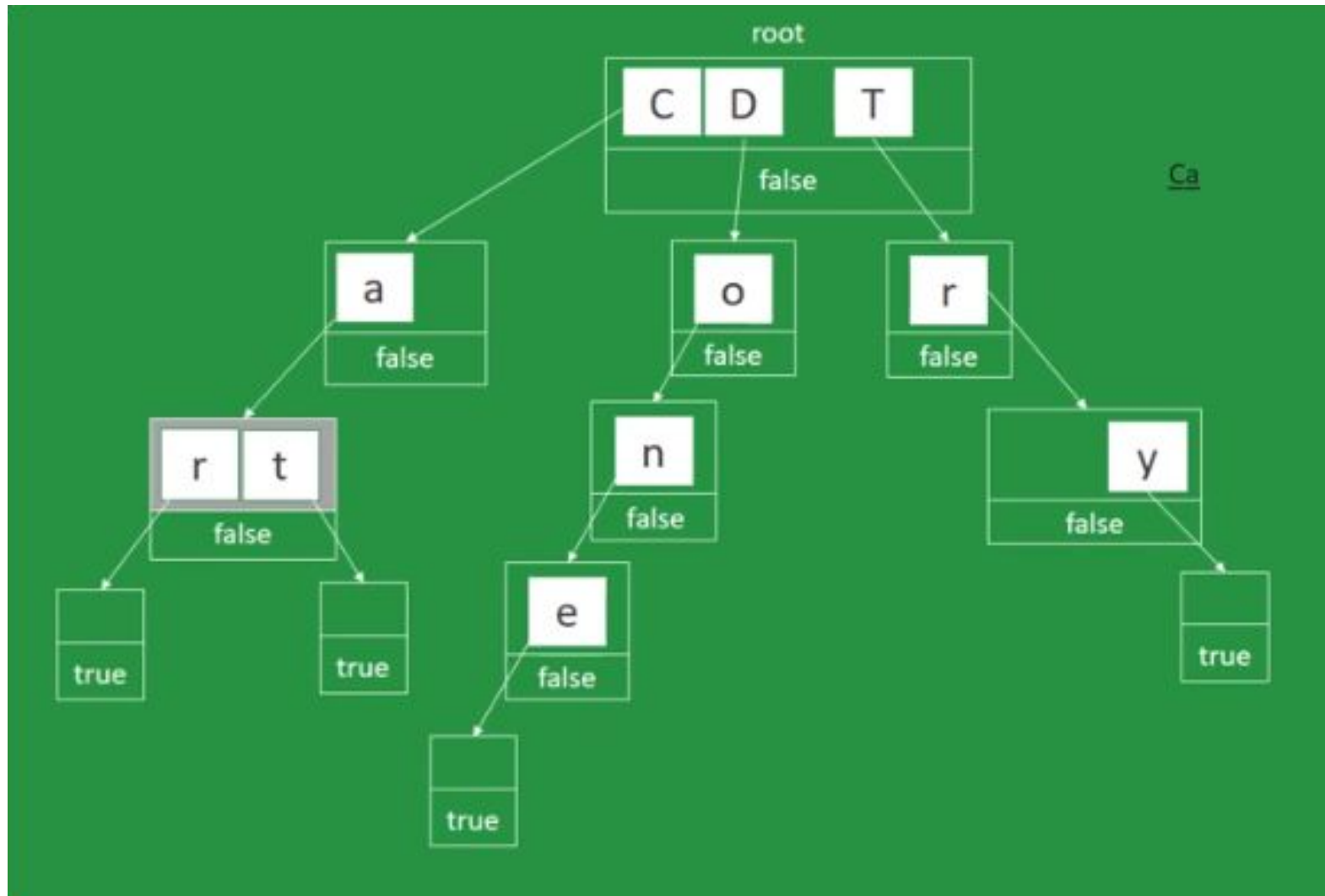
Search



Deletion (before)



Deletion (after)



Tries and Web Search Engines

- The ***index of a search engine*** (collection of all searchable words) is stored into a compressed trie
- Each leaf of the trie is associated with a word and has a list of pages (URLs) containing that word, called ***occurrence list***
- The trie is kept in internal memory
- The occurrence lists are kept in external memory and are ranked by relevance
- Boolean queries for sets of words (e.g., Java and coffee) correspond to set operations (e.g., intersection) on the occurrence lists
- Additional ***information retrieval*** techniques are used, such as
 - stopword elimination (e.g., ignore “the” “a” “is”)
 - stemming (e.g., identify “add” “adding” “added”)
 - link analysis (recognize authoritative pages)

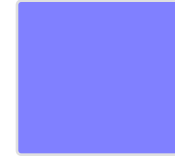
Tries and Internet Routers

- Computers on the internet (hosts) are identified by a unique 32-bit IP (*internet protocol*) address, usually written in “dotted-quad-decimal” notation
- E.g., `www.cs.brown.edu` is `128.148.32.110`
- Use `nslookup` on Unix to find out IP addresses
- An organization uses a subset of IP addresses with the same prefix, e.g., Brown uses `128.148.*.*`, Yale uses `130.132.*.*`
- Data is sent to a host by fragmenting it into packets. Each packet carries the IP address of its destination.
- The internet whose nodes are ***routers***, and whose edges are communication links.
- A router forwards packets to its neighbors using IP ***prefix matching*** rules. E.g., a packet with IP prefix `128.148.` should be forwarded to the Brown gateway router.
- Routers use tries on the alphabet `0,1` to do prefix matching.

Heaps

Heaps

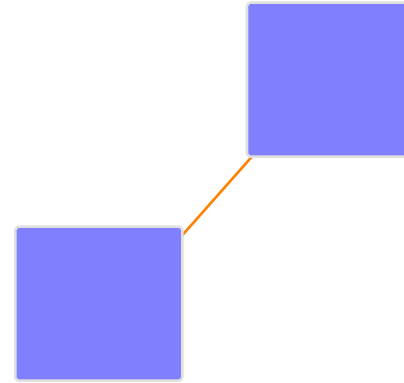
A heap is a certain kind of complete binary tree.



When a complete binary tree is built, its first node must be the root.

Heaps

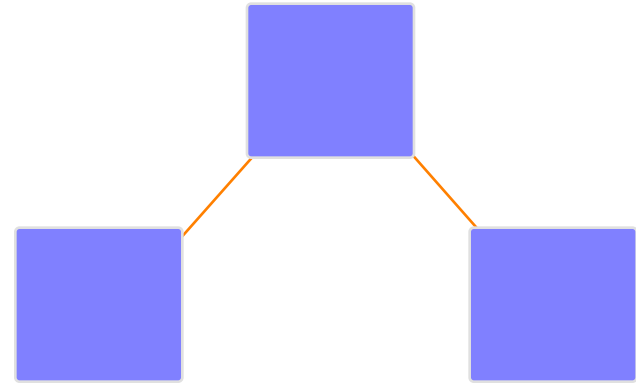
Complete binary tree.



The second node is
always the left child
of the root.

Heaps

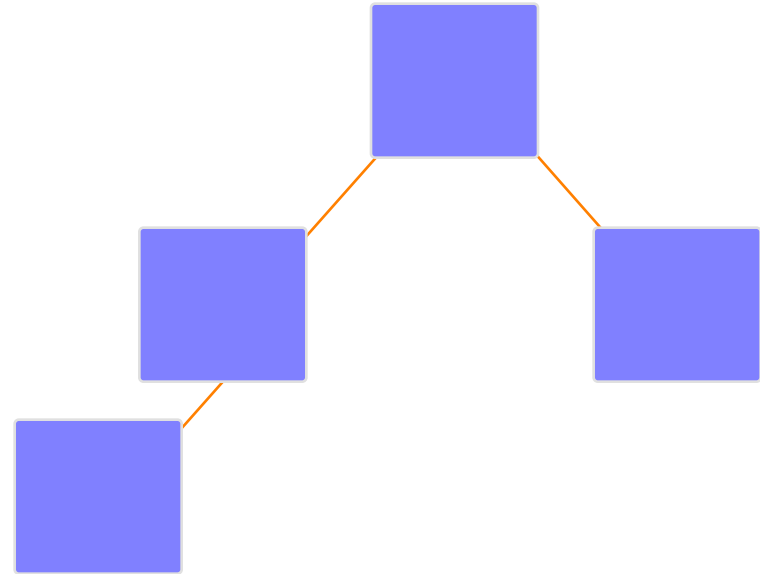
Complete binary tree.



The third node is
always the right child
of the root.

Heaps

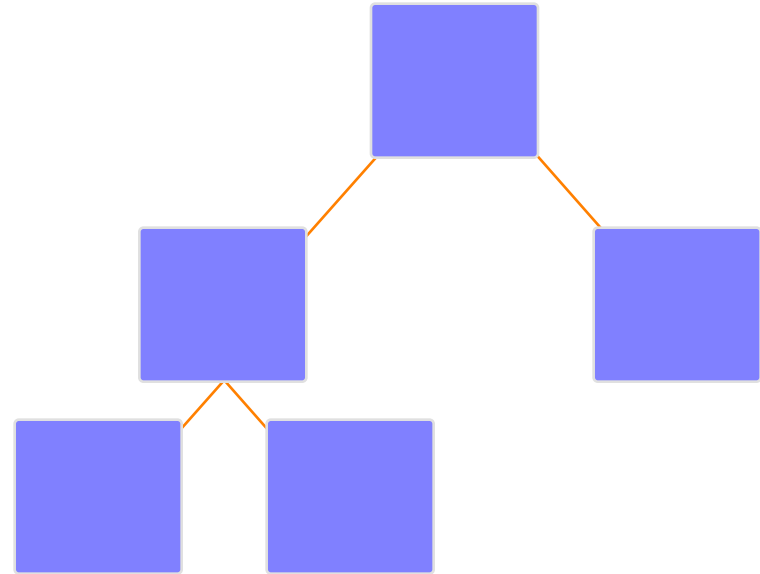
Complete binary tree.



The next nodes
always fill the next
level from left-to-
right.

Heaps

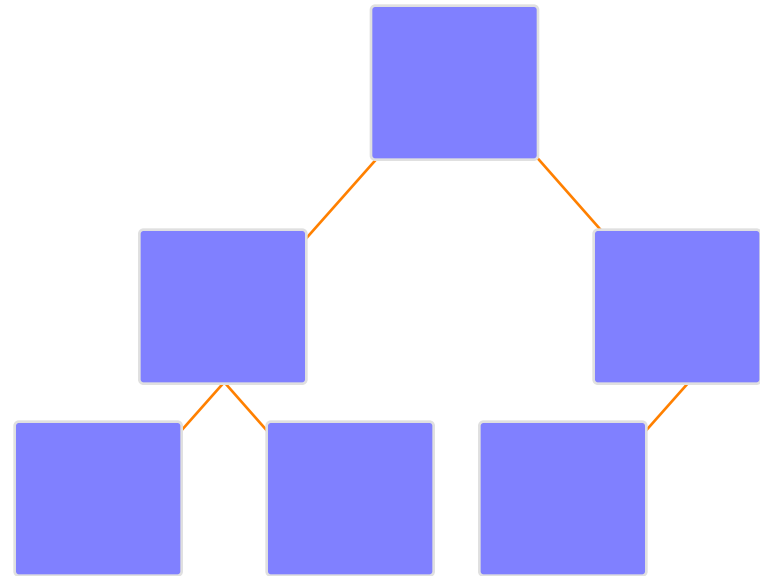
Complete binary tree.



The next nodes
always fill the next
level from left-to-
right.

Heaps

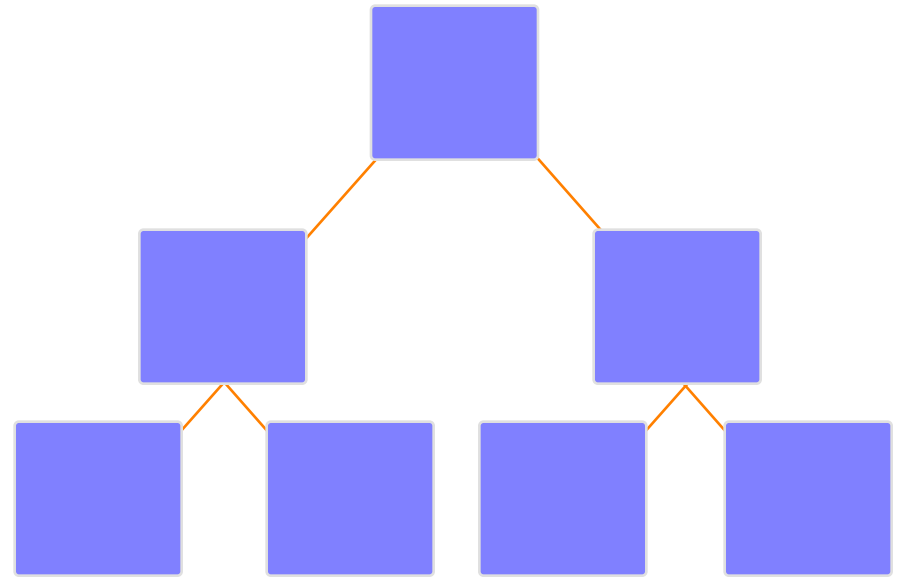
Complete binary tree.



The next nodes
always fill the next
level from left-to-
right.

Heaps

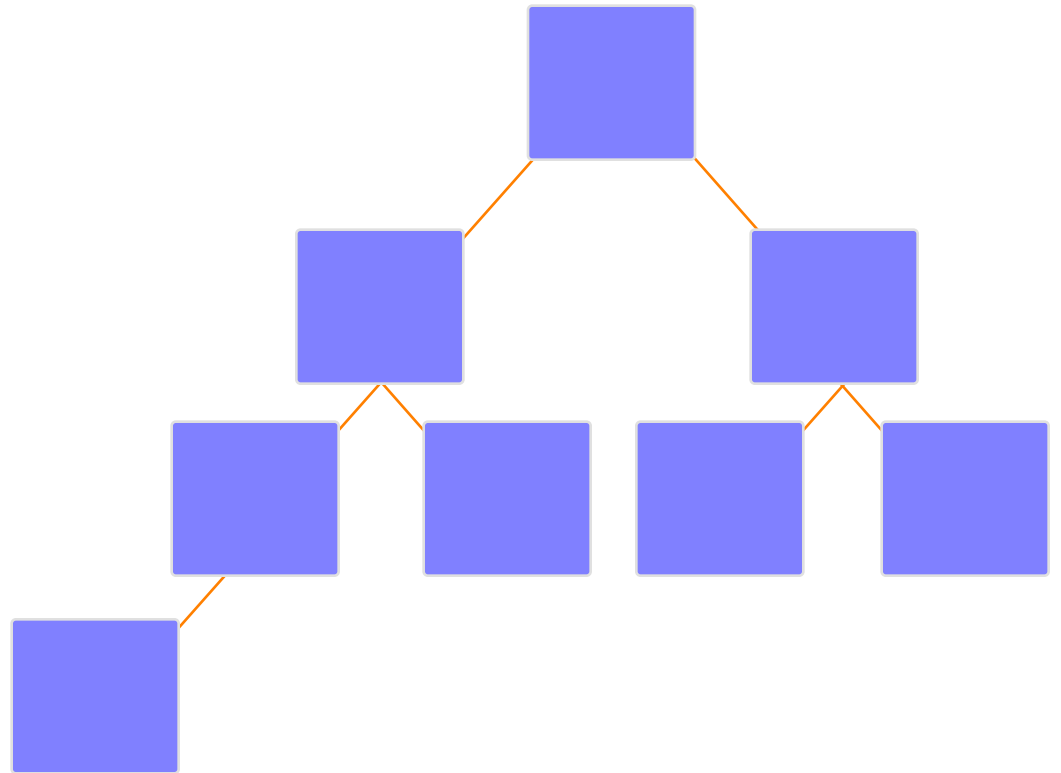
Complete binary tree.



The next nodes
always fill the next
level from left-to-
right.

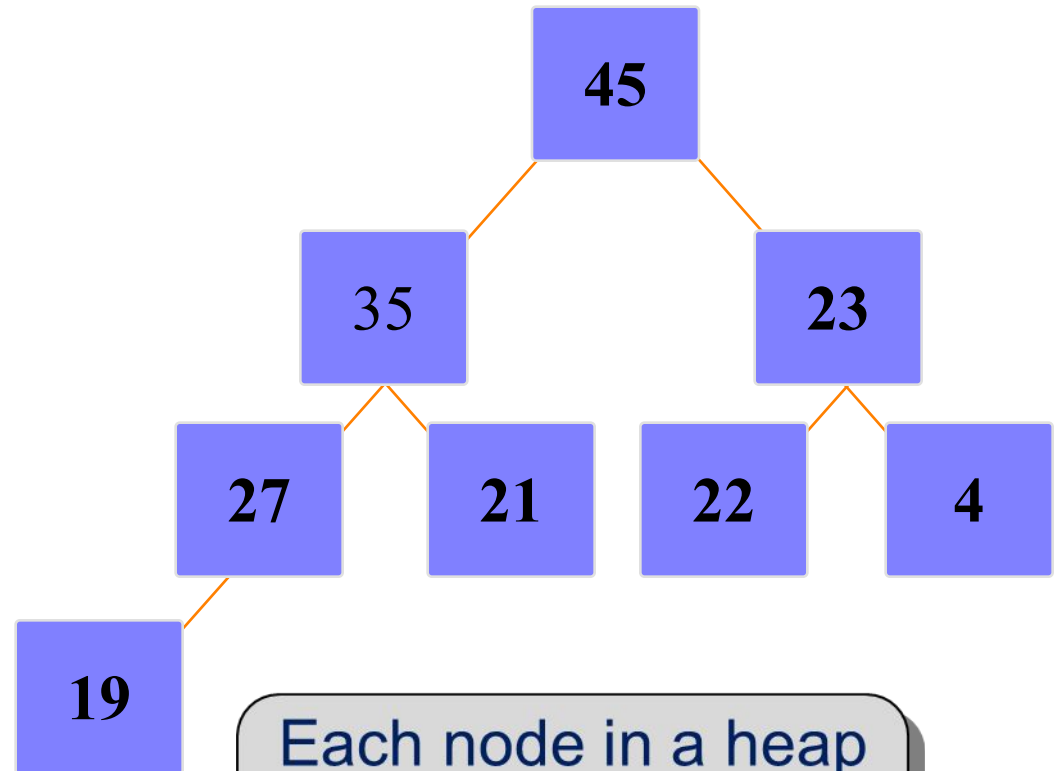
Heaps

Complete binary tree.



Heaps

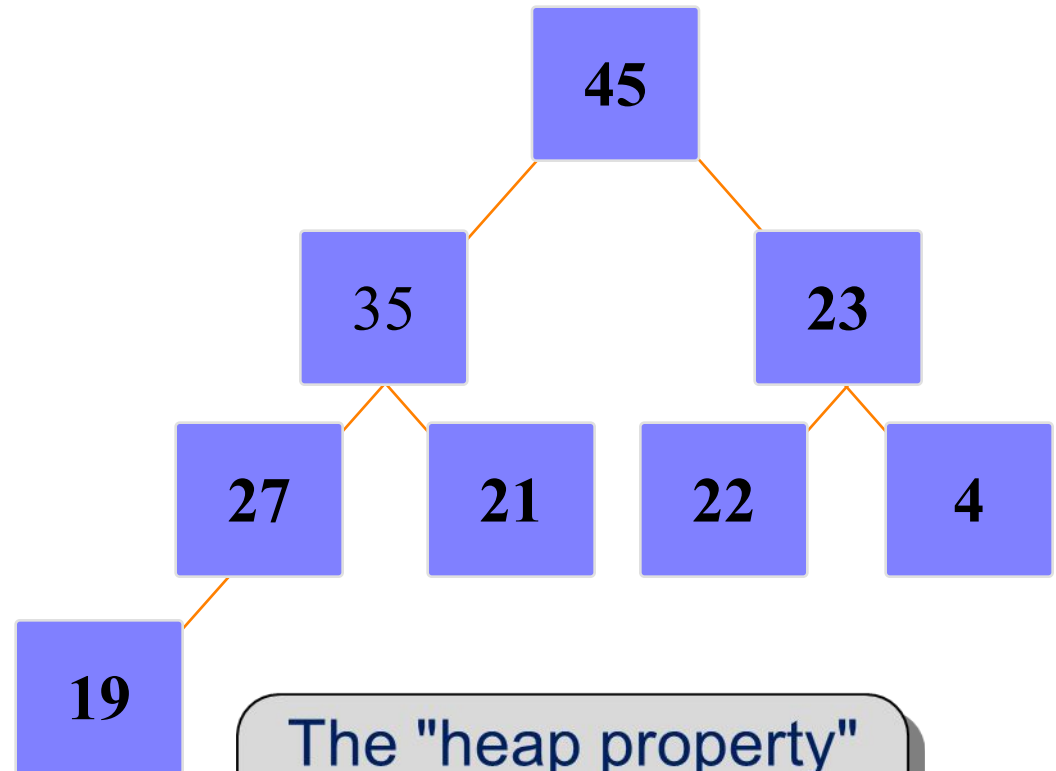
A heap is a **certain** kind of complete binary tree.



Each node in a heap contains a key that can be compared to other nodes' keys.

Heaps

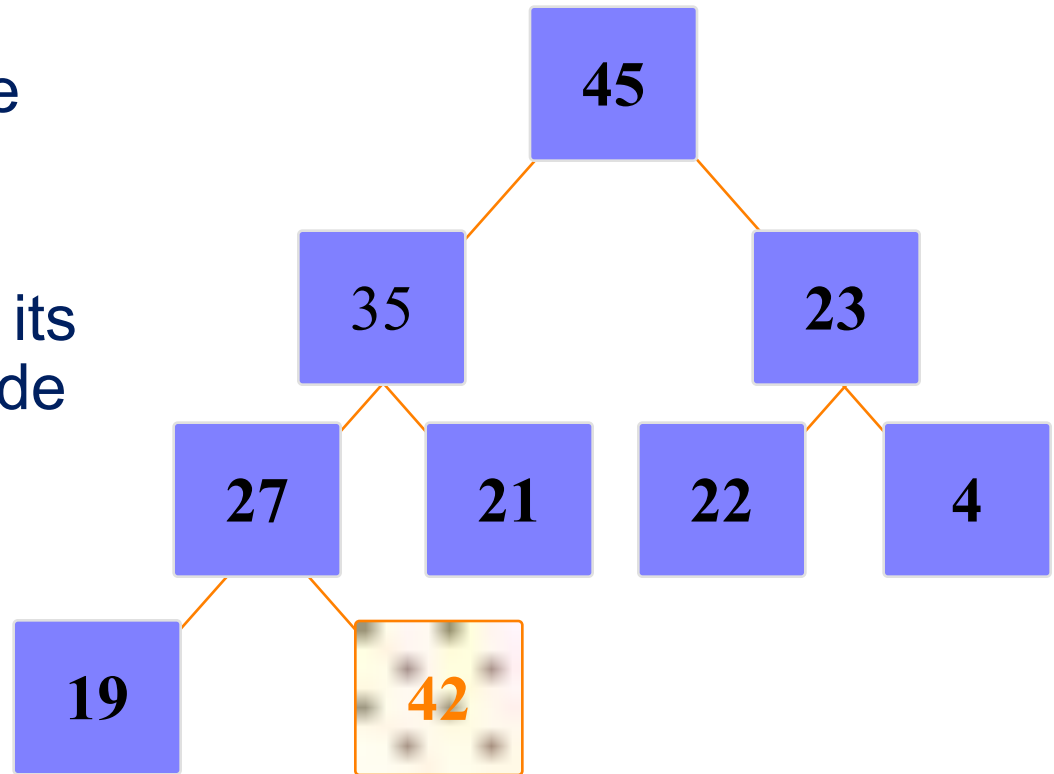
A heap is a certain kind of complete binary tree.



The "heap property" requires that each node's key is $\geq$ the keys of its children

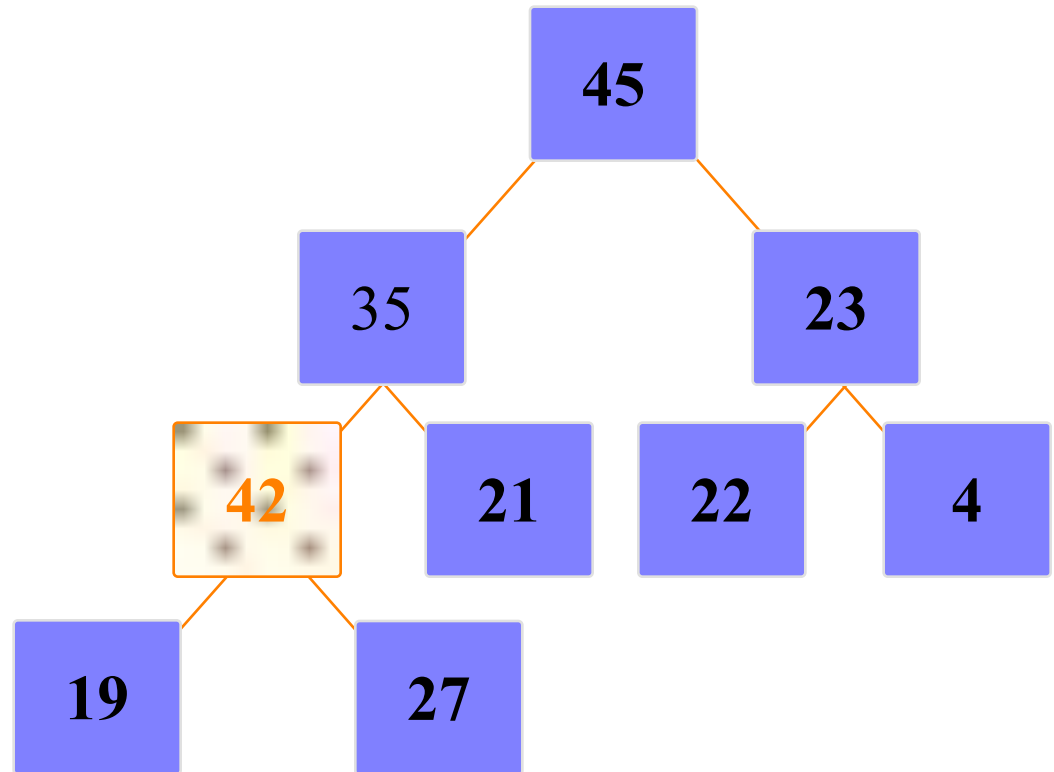
Adding a Node to a Heap

- ❑ Put the new node in the next available spot.
- ❑ Push the new node upward, swapping with its parent until the new node reaches an acceptable location.



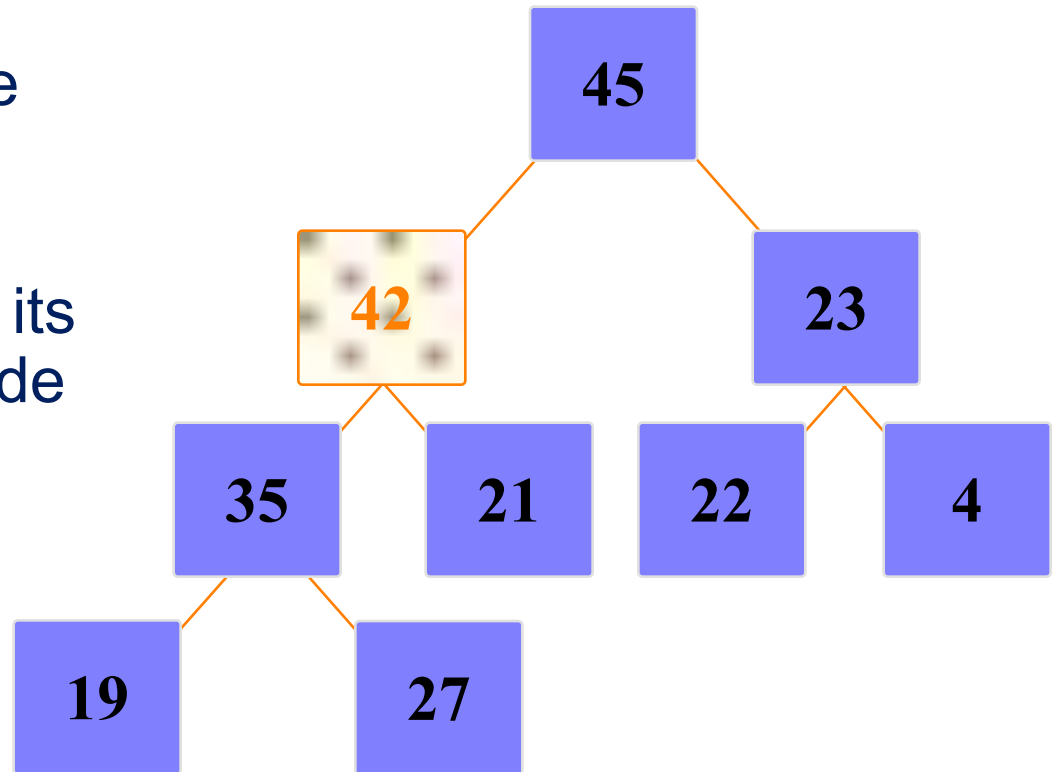
Adding a Node to a Heap

- ❑ Put the new node in the next available spot.
- ❑ Push the new node upward, swapping with its parent until the new node reaches an acceptable location.



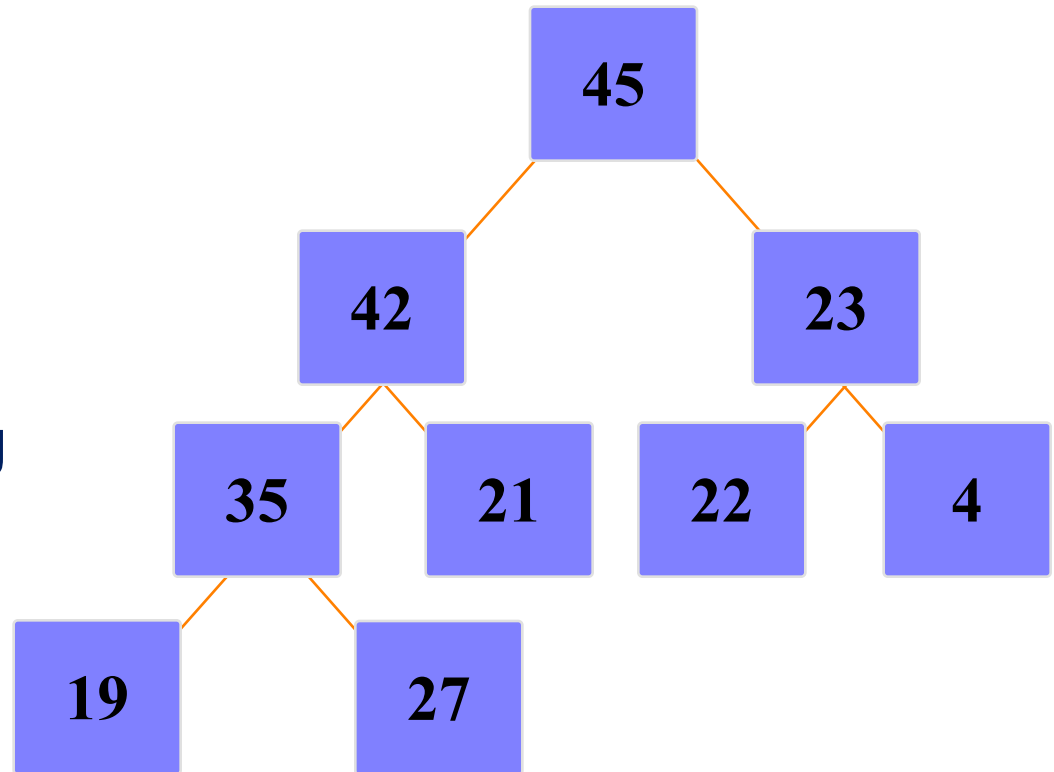
Adding a Node to a Heap

- ❑ Put the new node in the next available spot.
- ❑ Push the new node upward, swapping with its parent until the new node reaches an acceptable location.



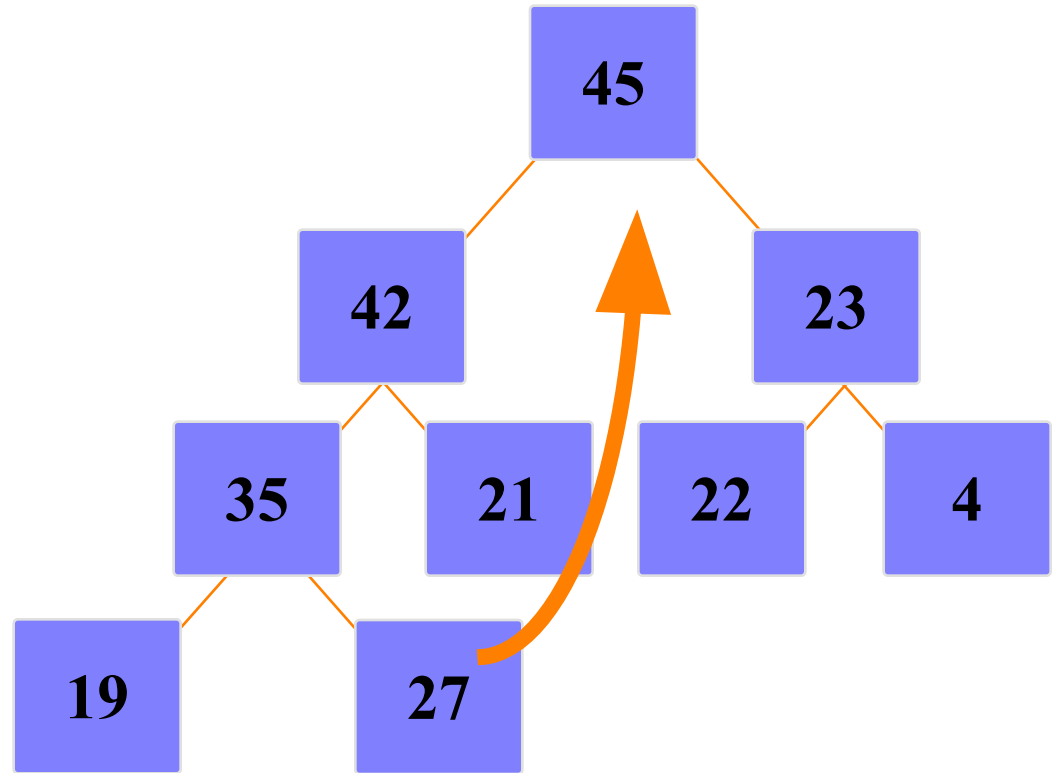
Adding a Node to a Heap

- ❑ The parent has a key that is $\geq$ new node, or
- ❑ The node reaches the root.
- ❑ The process of pushing the new node upward is called reheapification upward.



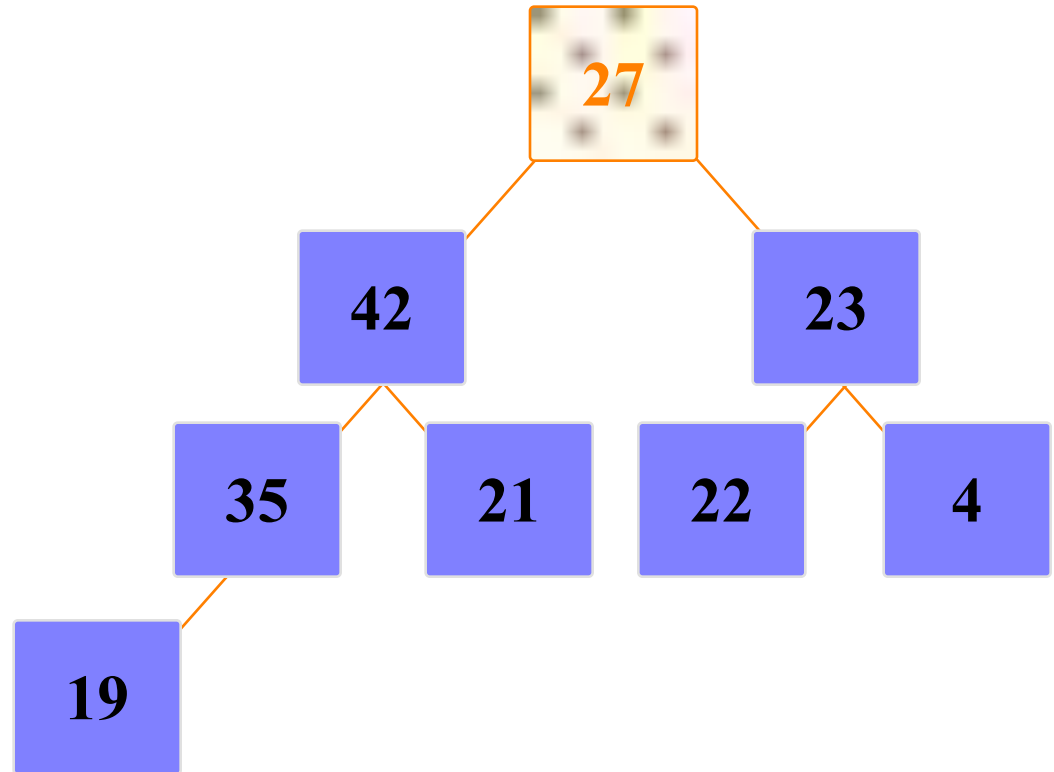
Removing the Top of a Heap

- Move the last node onto the root.



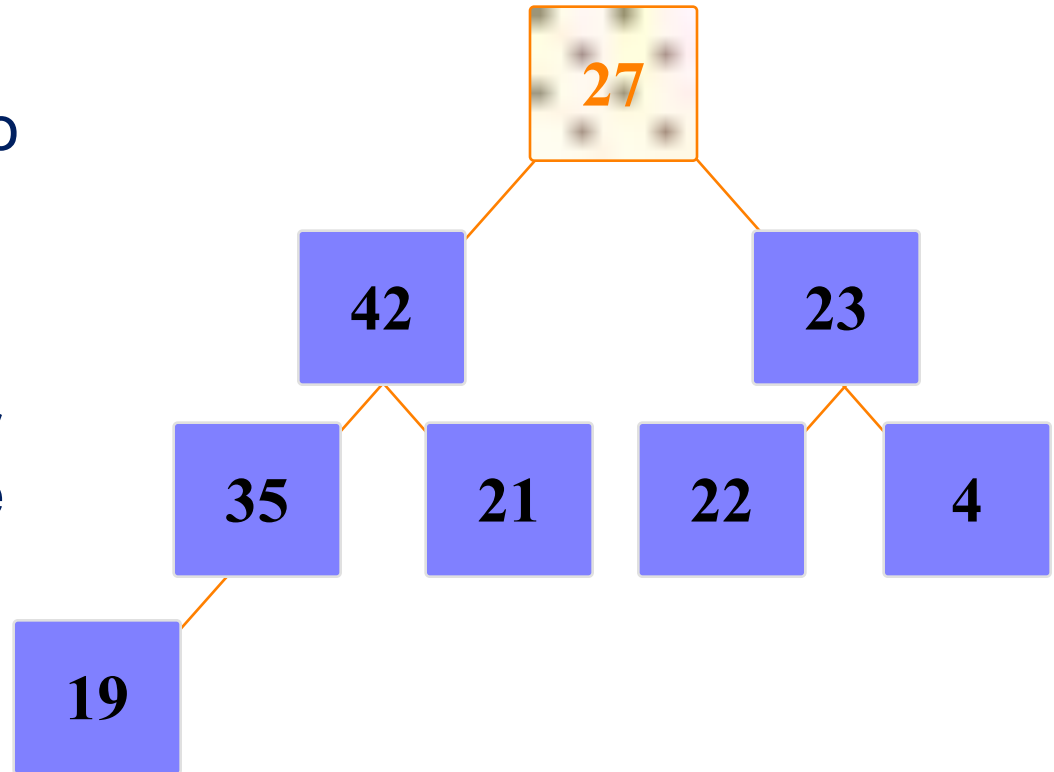
Removing the Top of a Heap

- ❑ Move the last node onto the root.



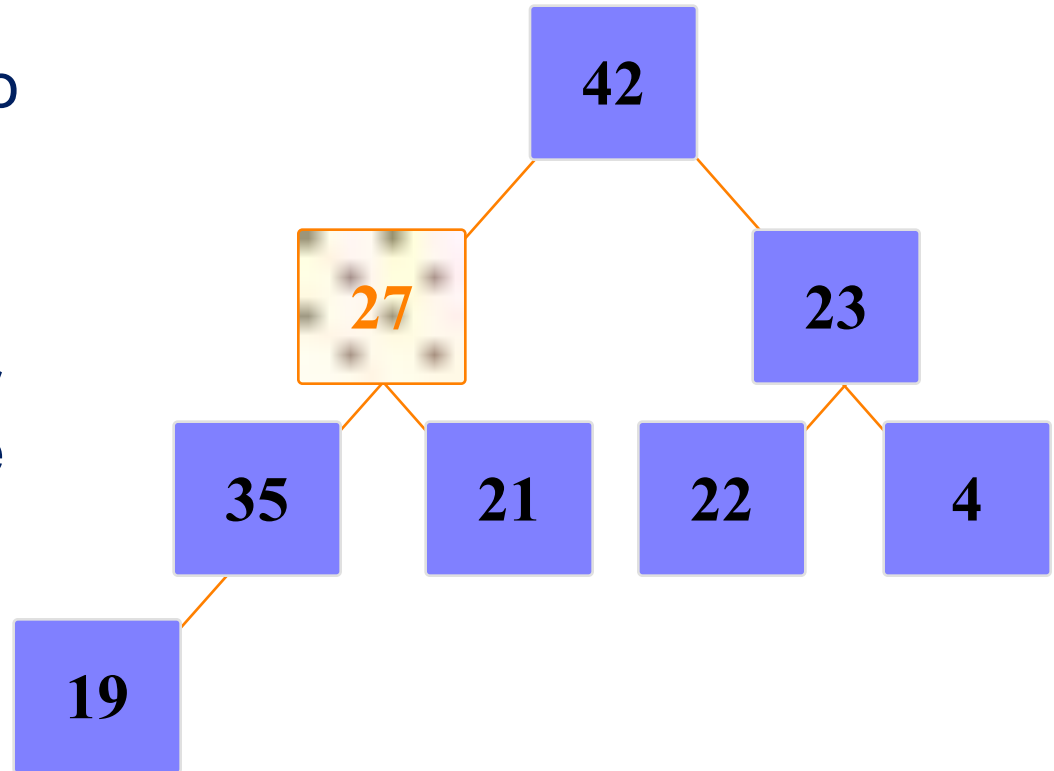
Removing the Top of a Heap

- ❑ Move the last node onto the root.
- ❑ Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.



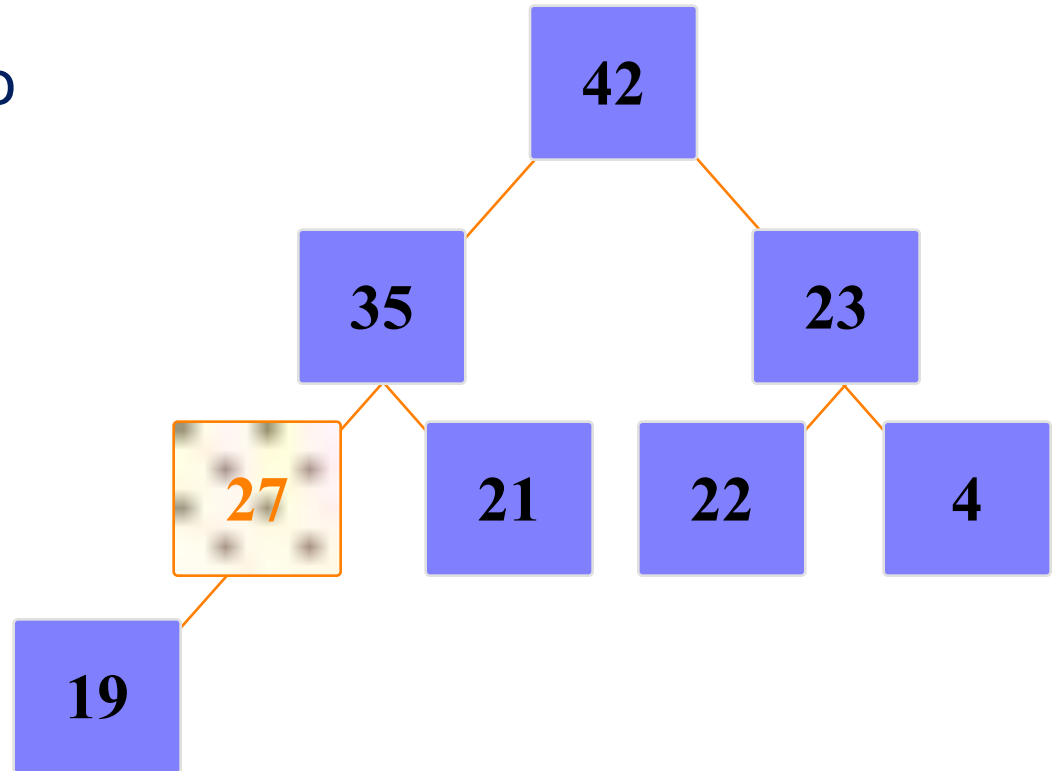
Removing the Top of a Heap

- ❑ Move the last node onto the root.
- ❑ Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.



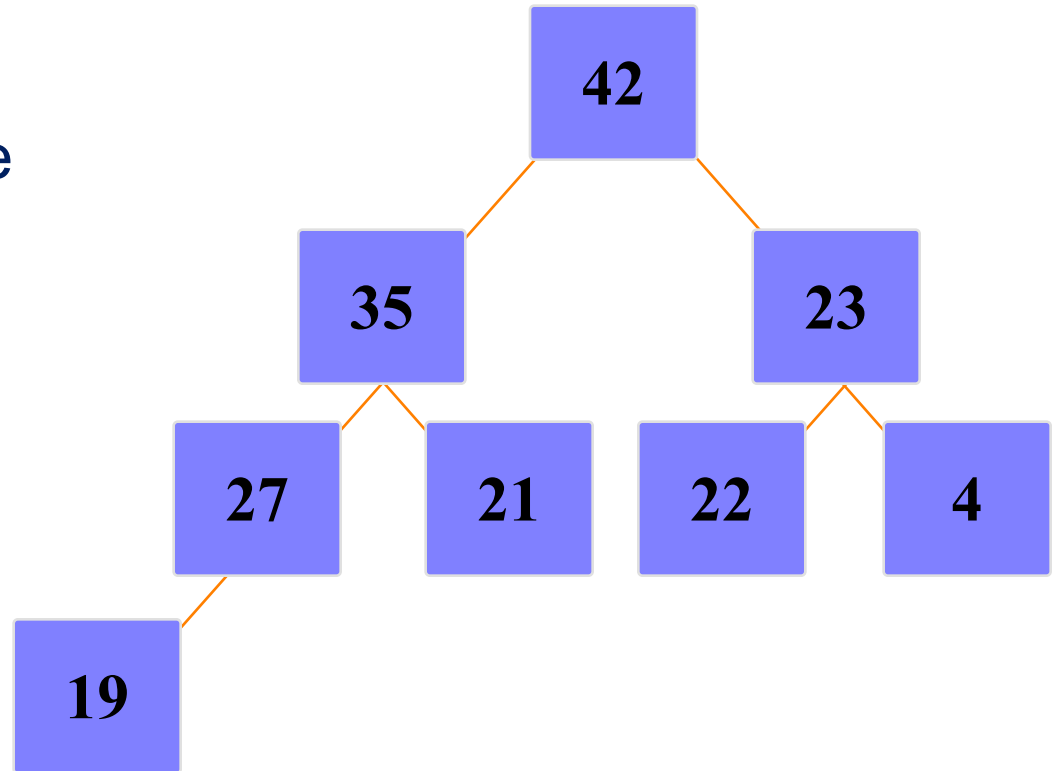
Removing the Top of a Heap

- ❑ Move the last node onto the root.
- ❑ Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.



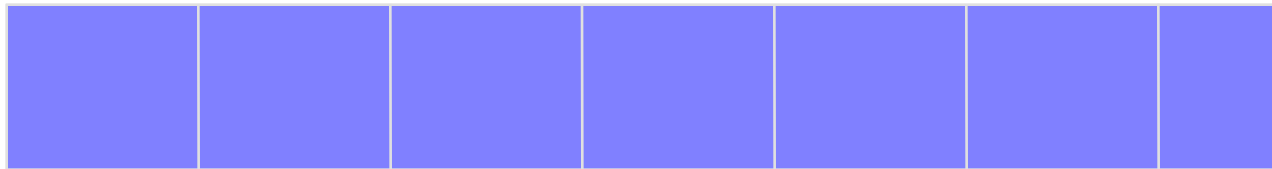
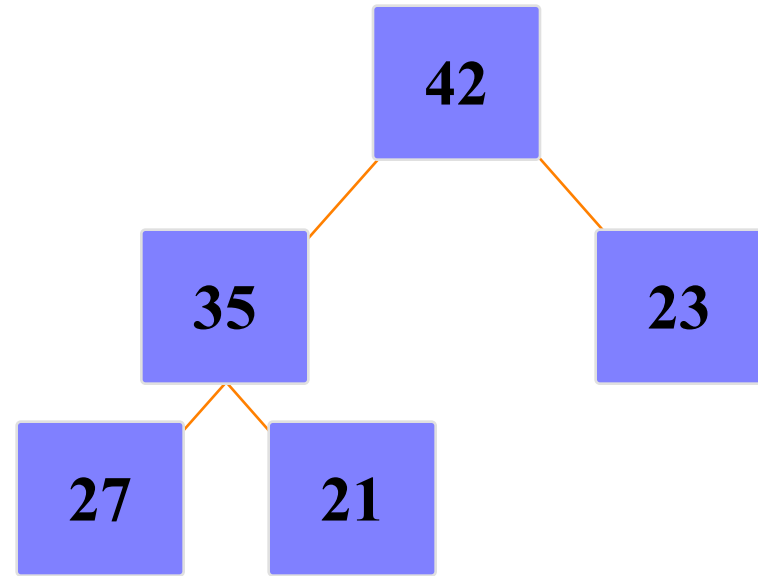
Removing the Top of a Heap

- ❑ The children all have keys $\leq$ the out-of-place node, or
- ❑ The node reaches the leaf.
- ❑ The process of pushing the new node downward is called **reheapification downward**.



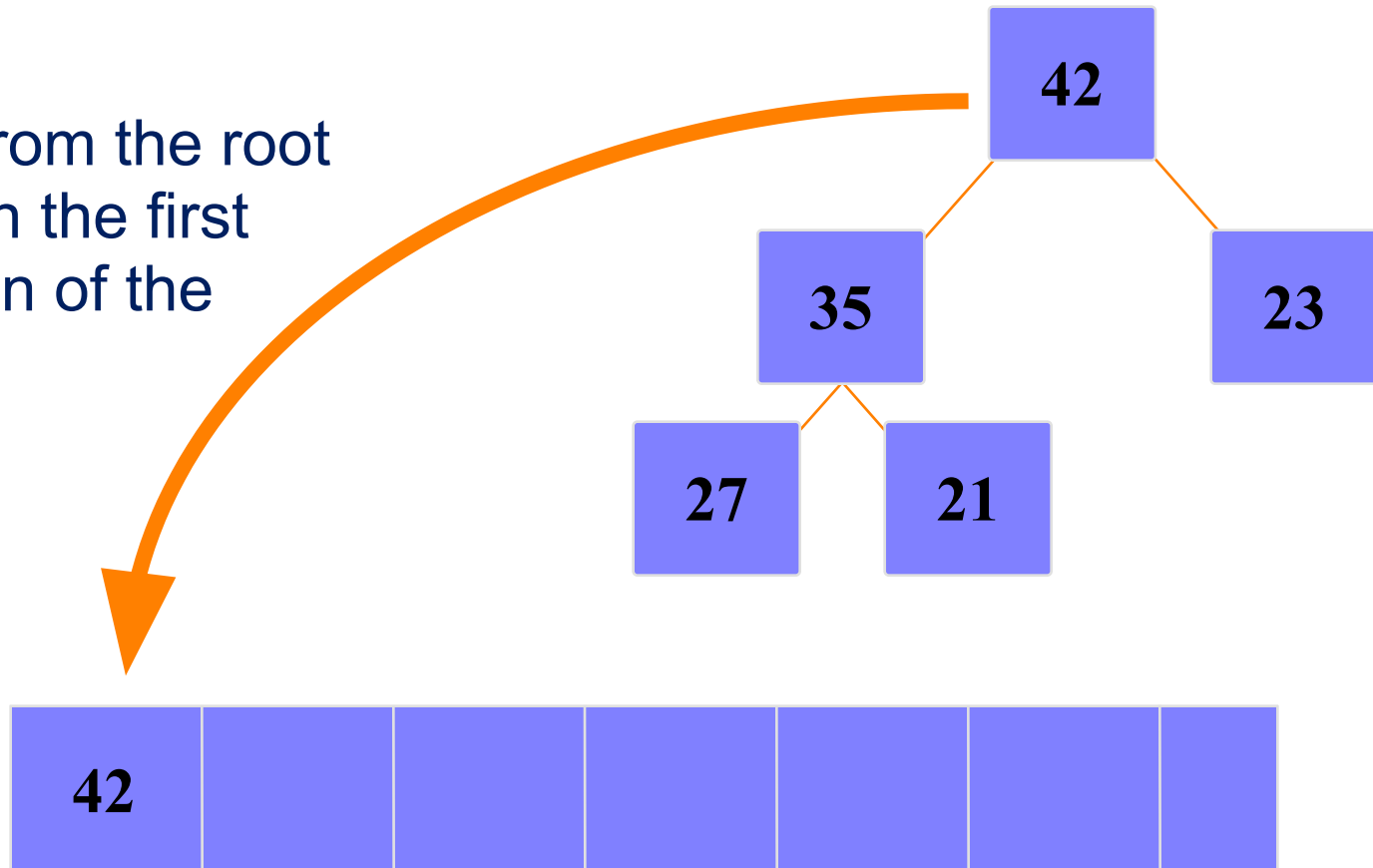
Implementing a Heap

- We will store the data from the nodes in a partially-filled array.



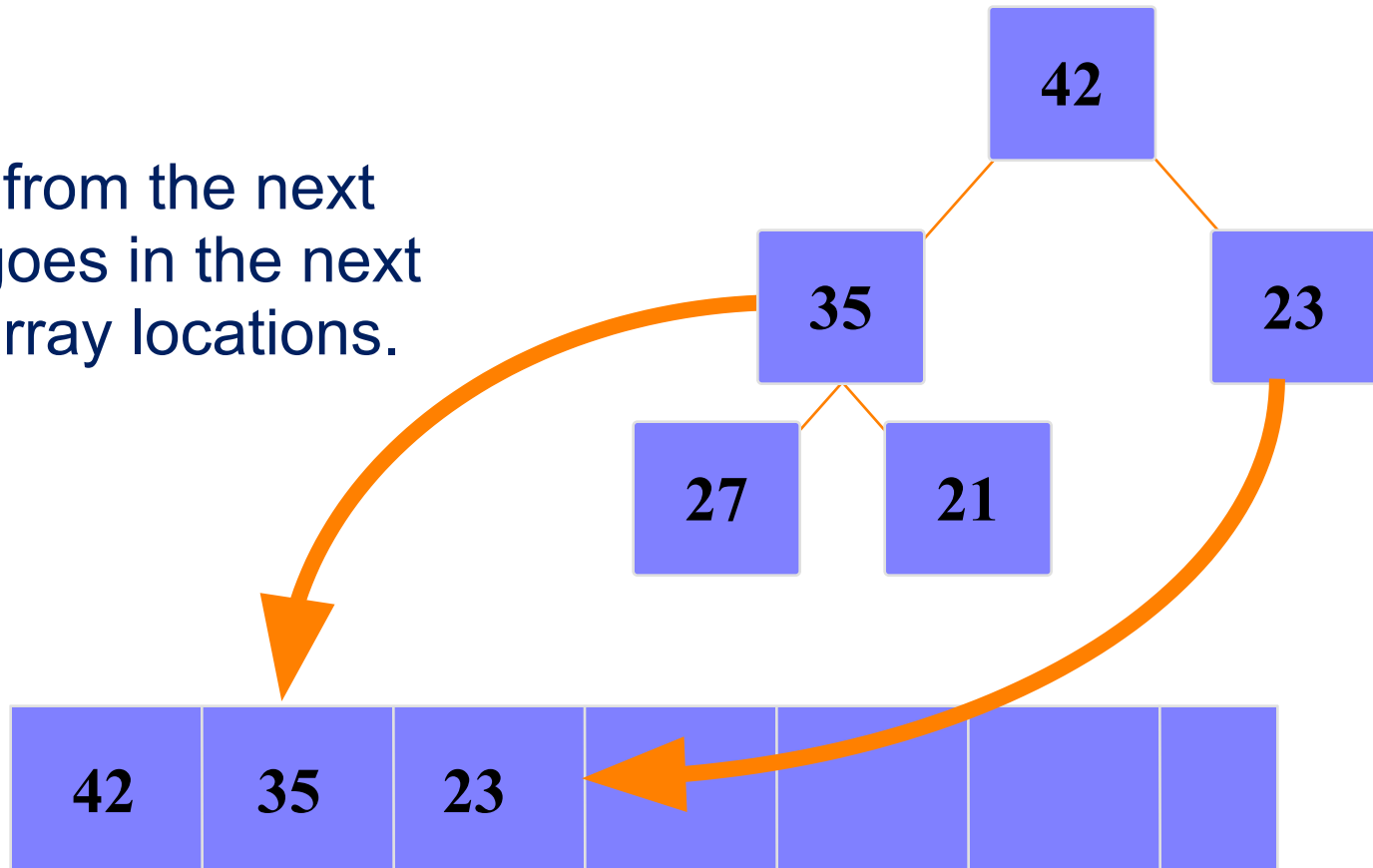
Implementing a Heap

- Data from the root goes in the first location of the array.



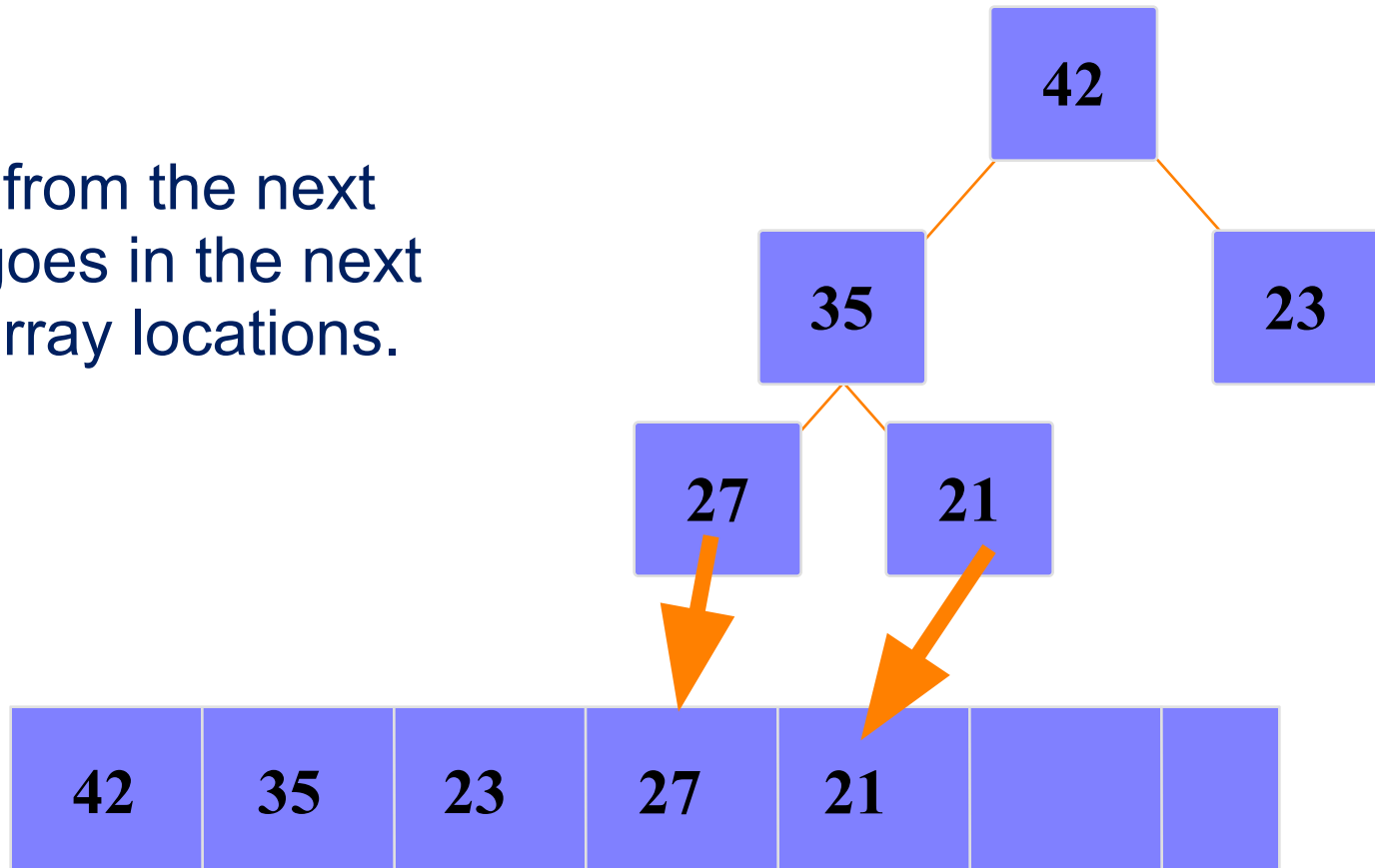
Implementing a Heap

- Data from the next row goes in the next two array locations.



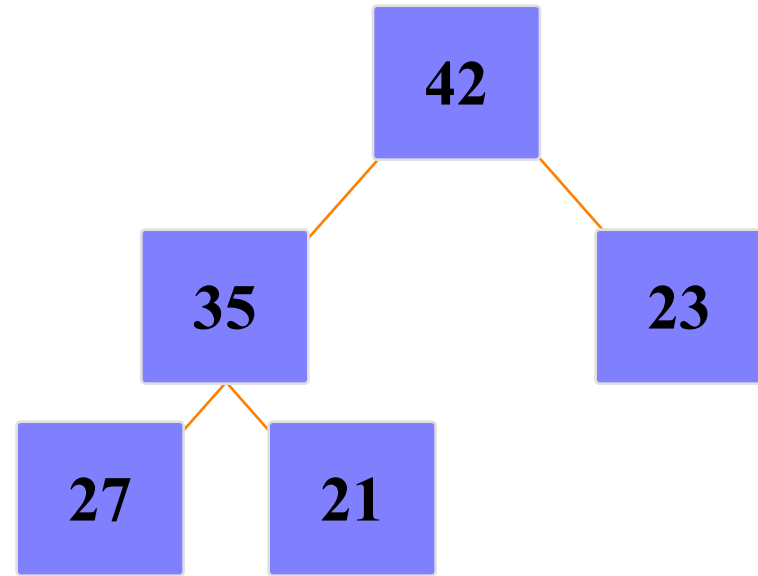
Implementing a Heap

- Data from the next row goes in the next two array locations.



Implementing a Heap

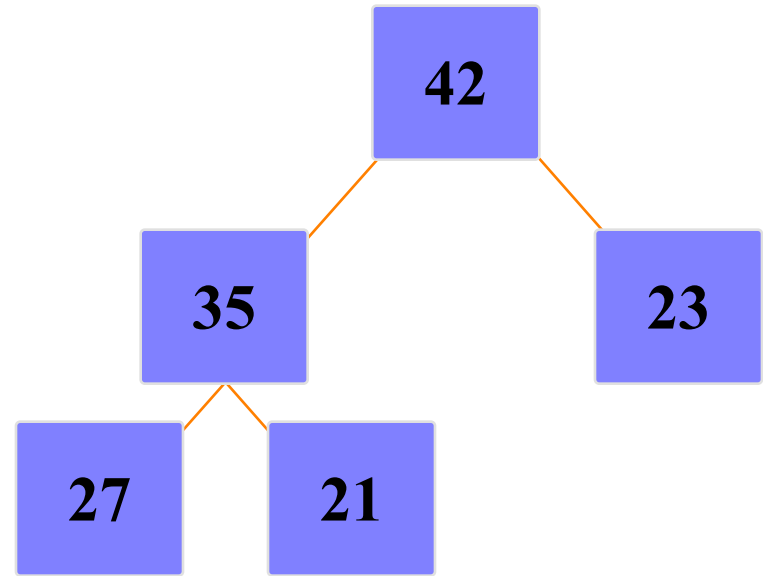
- Data from the next row goes in the next two array locations.



We don't care what's in
this part of the array.

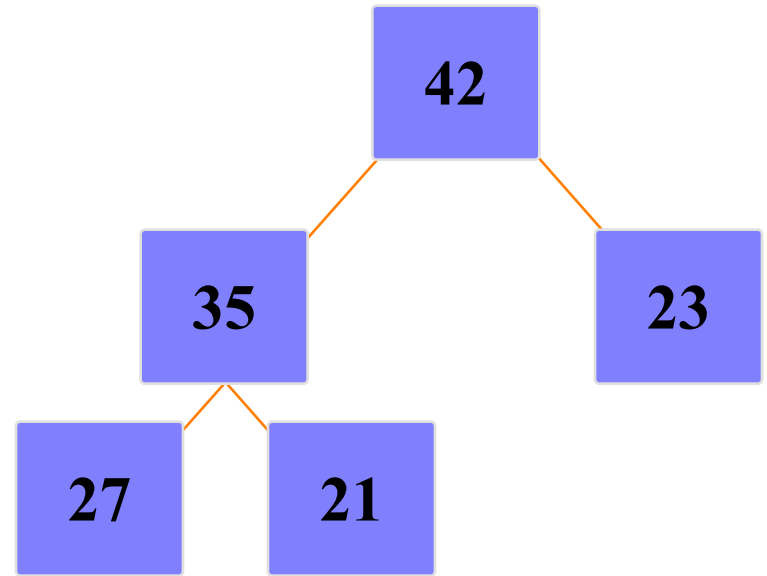
Important Points about the Implementation

- The links between the tree's nodes are not actually stored as pointers, or in any other way.
- The only way we "know" that "the array is a tree" is from the way we manipulate the data.



Important Points about the Implementation

- If you know the index of a node, then it is easy to figure out the indexes of that node's parent and children.



Summary

- A heap is a complete binary tree, where the entry at each node is greater than or equal to the entries in its children.
- To add an entry to a heap, place the new entry at the next available spot, and perform a reheapification upward.
- To remove the biggest entry, move the last node onto the root, and perform a reheapification downward.

Hash Table

What is a Hash Table ?

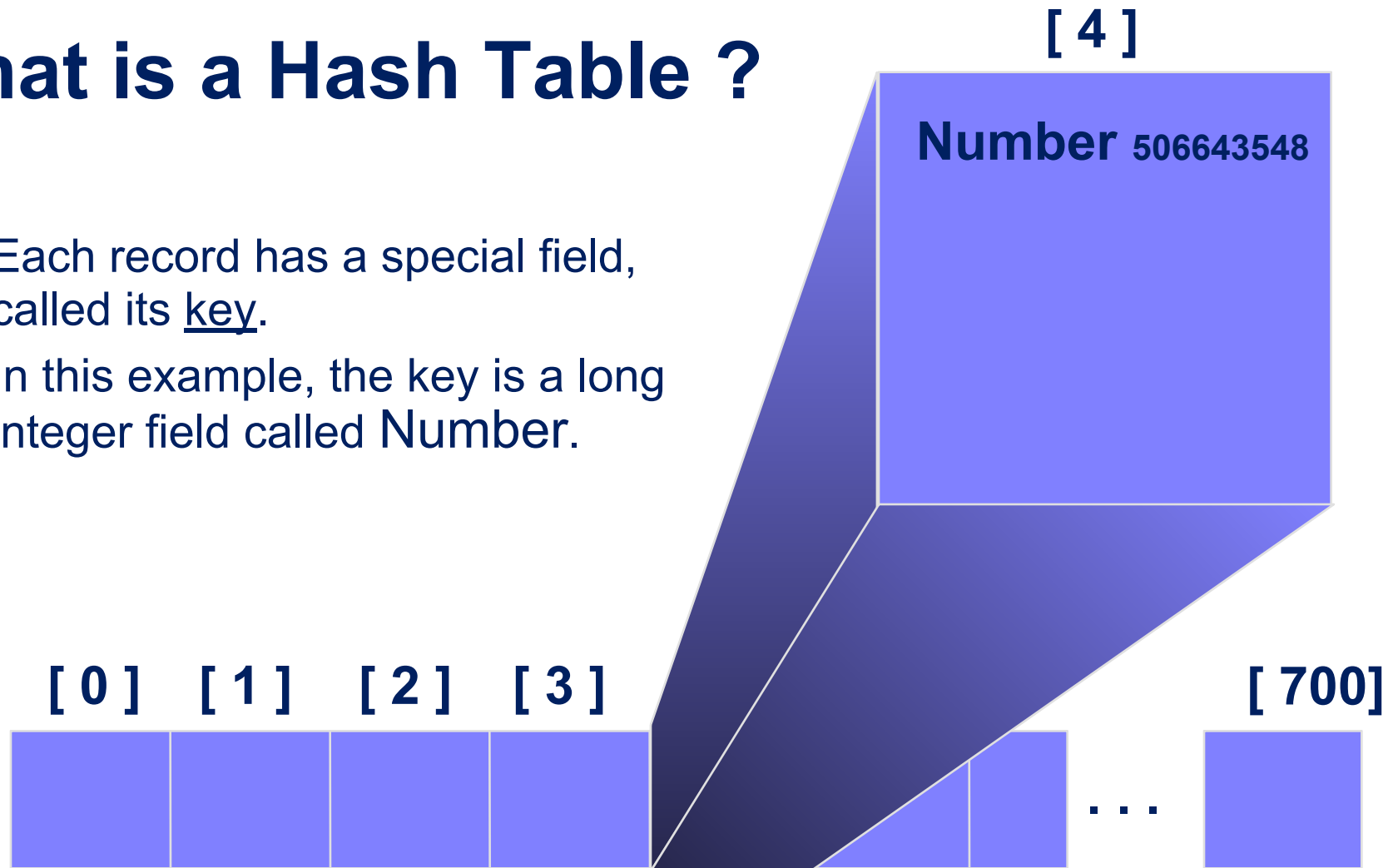
- The simplest kind of hash table is an array of records.
- This example has 701 records.



An array of records

What is a Hash Table ?

- Each record has a special field, called its key.
- In this example, the key is a long integer field called Number.



What is a Hash Table ?

- The number might be a person's identification number, and the rest of the record has information about the person.



What is a Hash Table ?

- When a hash table is in use, some spots contain valid records, and other spots are "empty".



Inserting a New Record

- In order to insert a new record, the **key** must somehow be **converted to** an array **index**.
- The index is called the **hash value** of the key.



Inserting a New Record

- Typical way create a hash value:



What is $(580625685 \bmod 701)$?



Inserting a New Record

- Typical way to create a hash value:

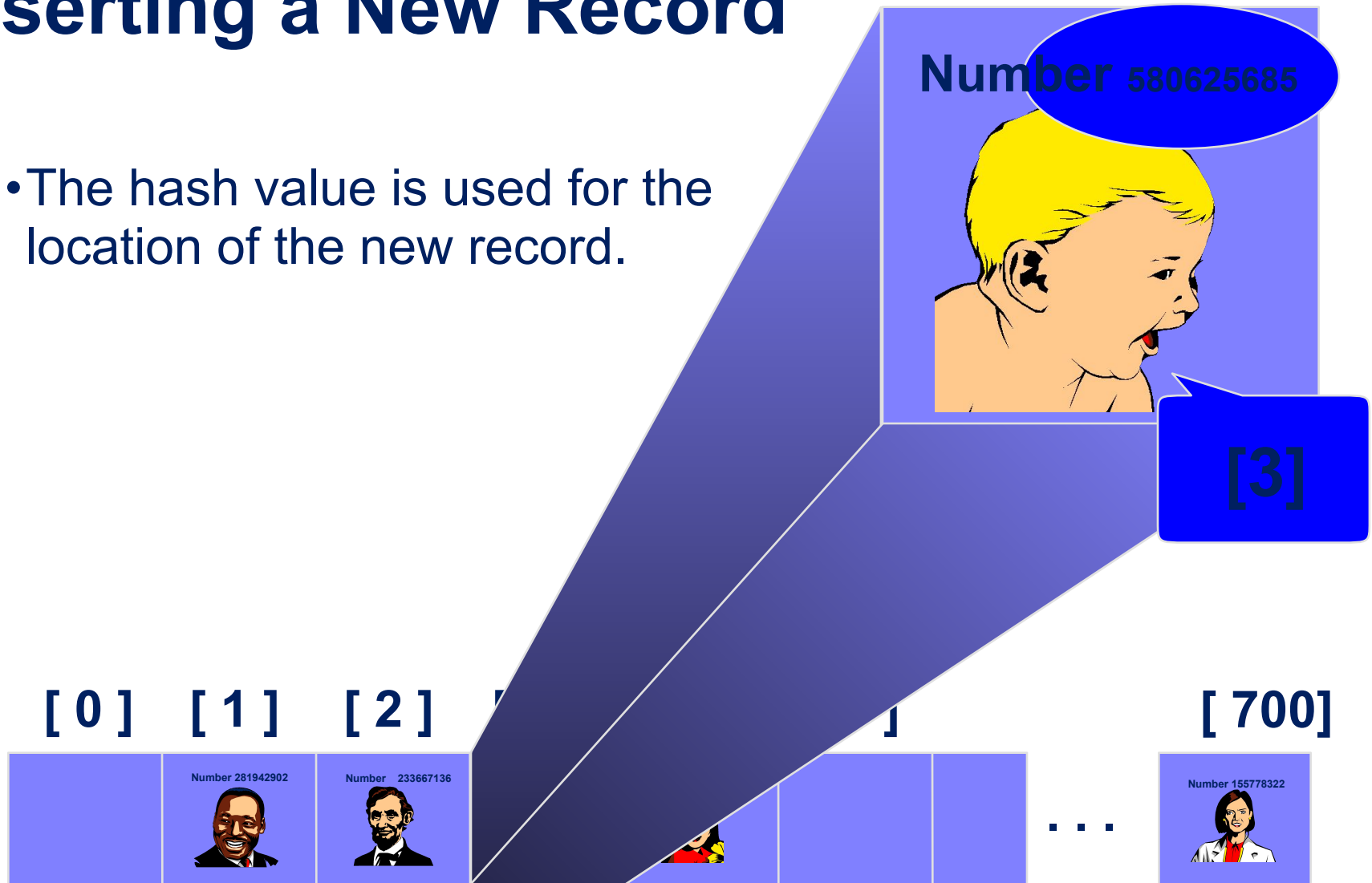
(Number mod 701)

What is $(580625685 \bmod 701)$?



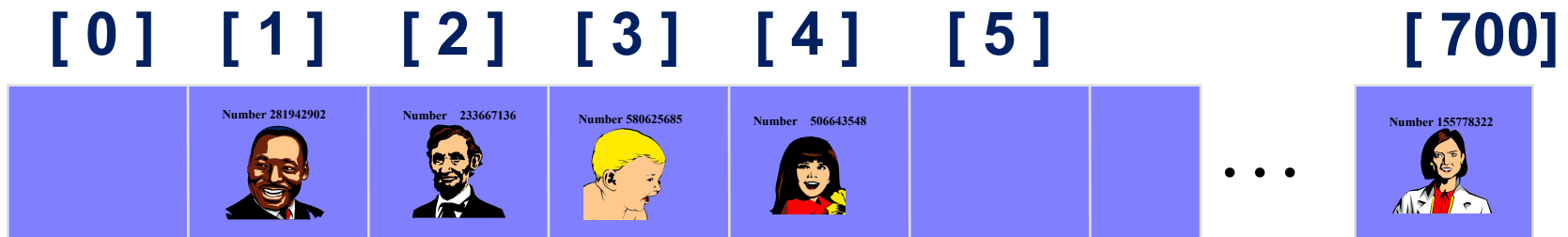
Inserting a New Record

- The hash value is used for the location of the new record.



Inserting a New Record

- The hash value is used for the location of the new record.

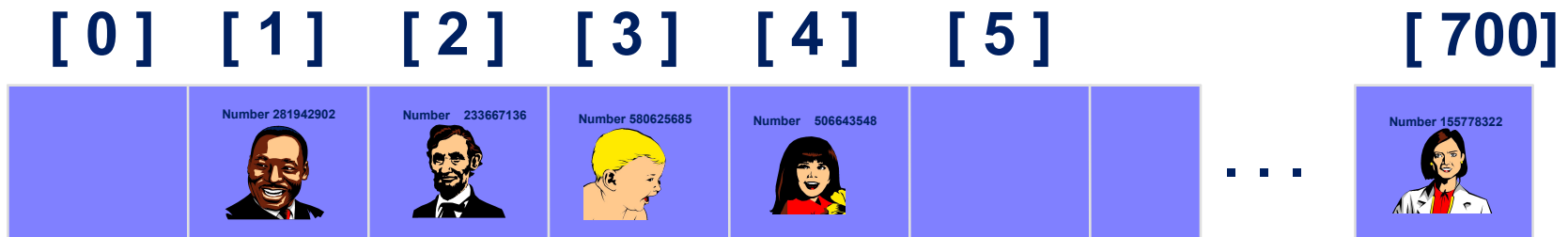
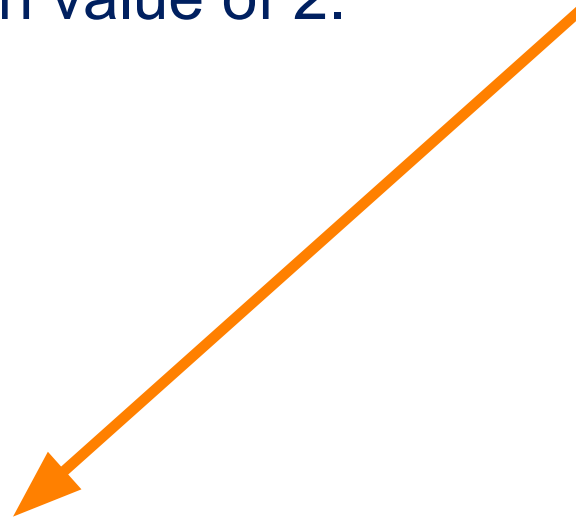


Collisions

- Here is another new record to insert, with a hash value of 2.



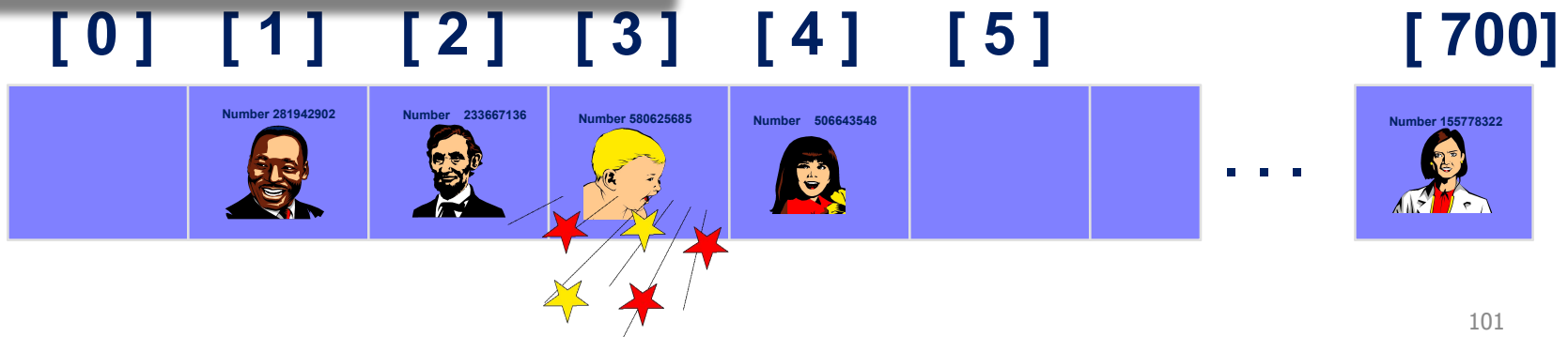
My hash value is [2].



Collisions

- This is called a **collision**, because there is already another valid record at [2].

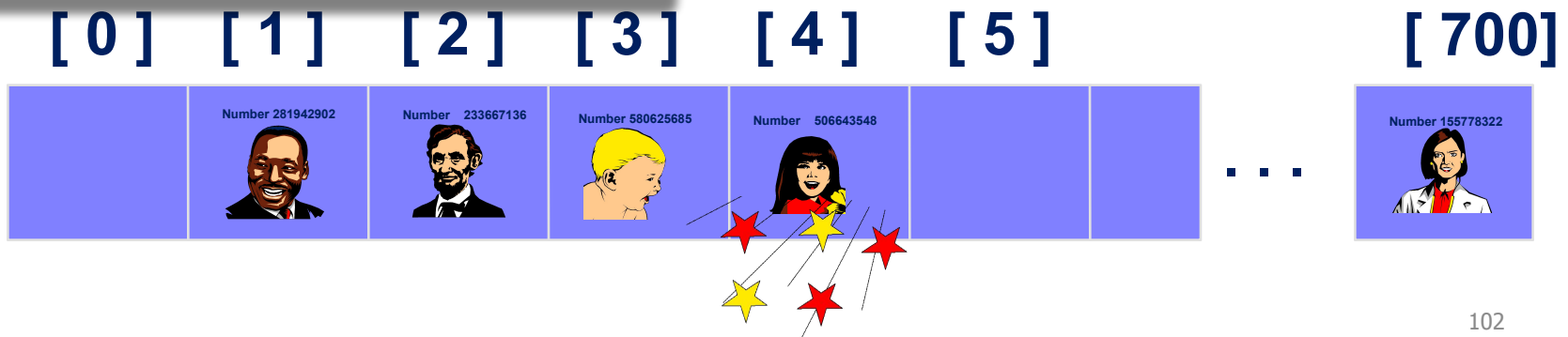
When a collision occurs, move forward until you find an empty spot.



Collisions

- This is called a **collision**, because there is already another valid record at [2].

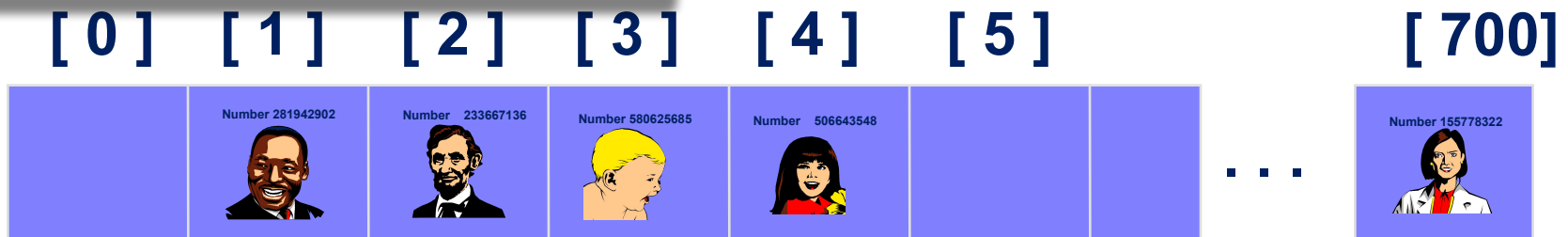
When a collision occurs, move forward until you find an empty spot.



Collisions

- This is called a **collision**, because there is already another valid record at [2].

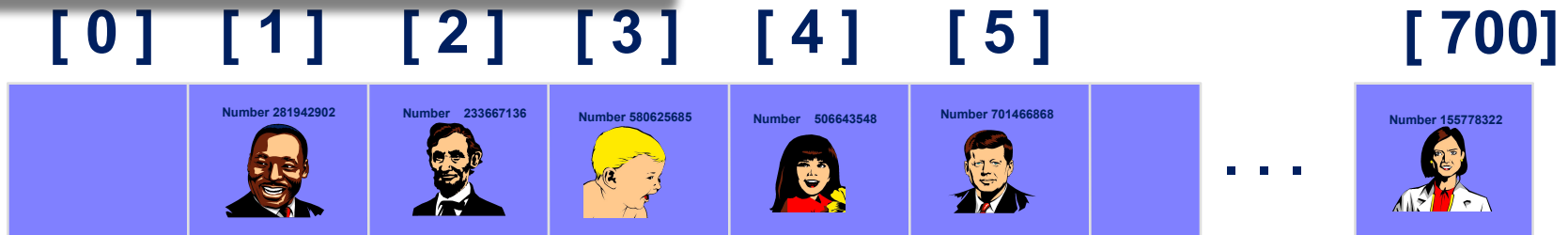
When a collision occurs, move forward until you find an empty spot.



Collisions

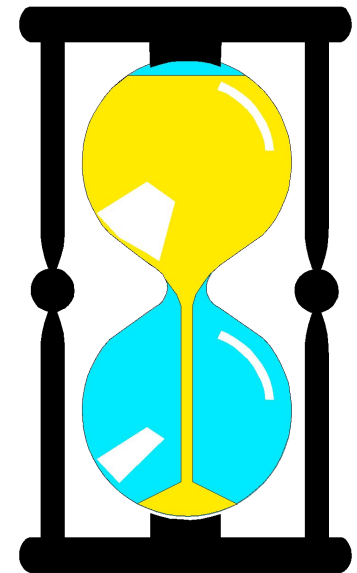
- This is called a **collision**, because there is already another valid record at [2].

The new record goes in the empty spot.



A Quiz

Where would you be placed in this table, if there is no collision? Use your social security number or some other favorite number.



[0]

[1]

[2]

[3]

[4]

[5]

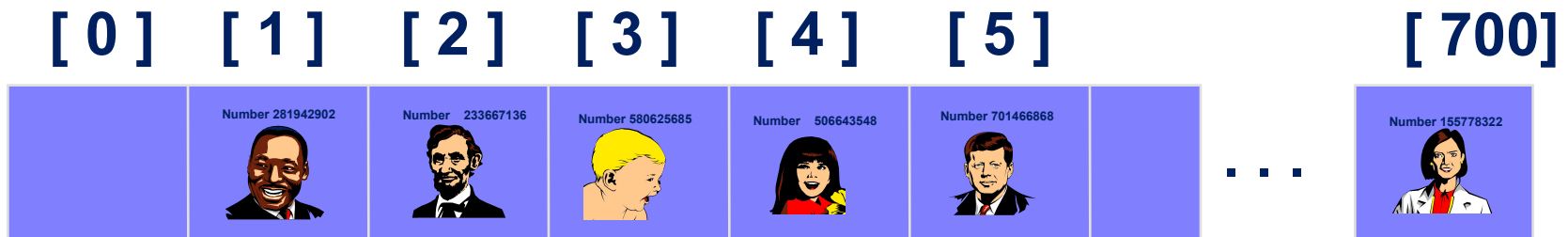
[700]

	Number 281942902 	Number 233667136 	Number 580625685 	Number 506643548 	Number 701466868 	...	Number 155778322 
--	---	---	---	--	---	-----	---

Searching for a Key

- The data that's attached to a key can be found fairly quickly.

Number 701466868



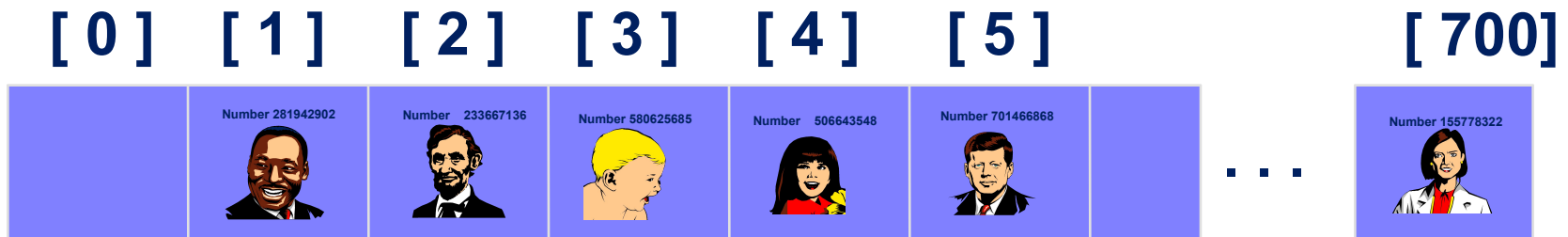
Searching for a Key

- Calculate the hash value.
- Check that location of the array for the key.

Number 701466868

My hash value is [2].

Not me.



Searching for a Key

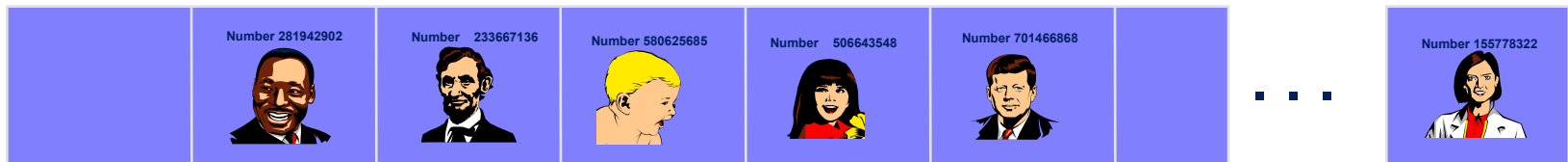
- Keep moving forward until you find the key, or you reach an empty spot.

Number 701466868

My hash value is [2].

Not me.

[0] [1] [2] [3] [4] [5] ... [700]



Searching for a Key

- Keep moving forward until you find the key, or you reach an empty spot.

Number 701466868

My hash value is [2].

Not me.

[0] [1] [2] [3] [4] [5] ... [700]

	Number 281942902 	Number 233667136 	Number 580625685 	Number 506643548 	Number 701466868 	...	Number 155778322 
--	---	---	---	--	---	-----	---

Searching for a Key

- Keep moving forward until you find the key, or you reach an empty spot.

Number 701466868

My hash value is [2].

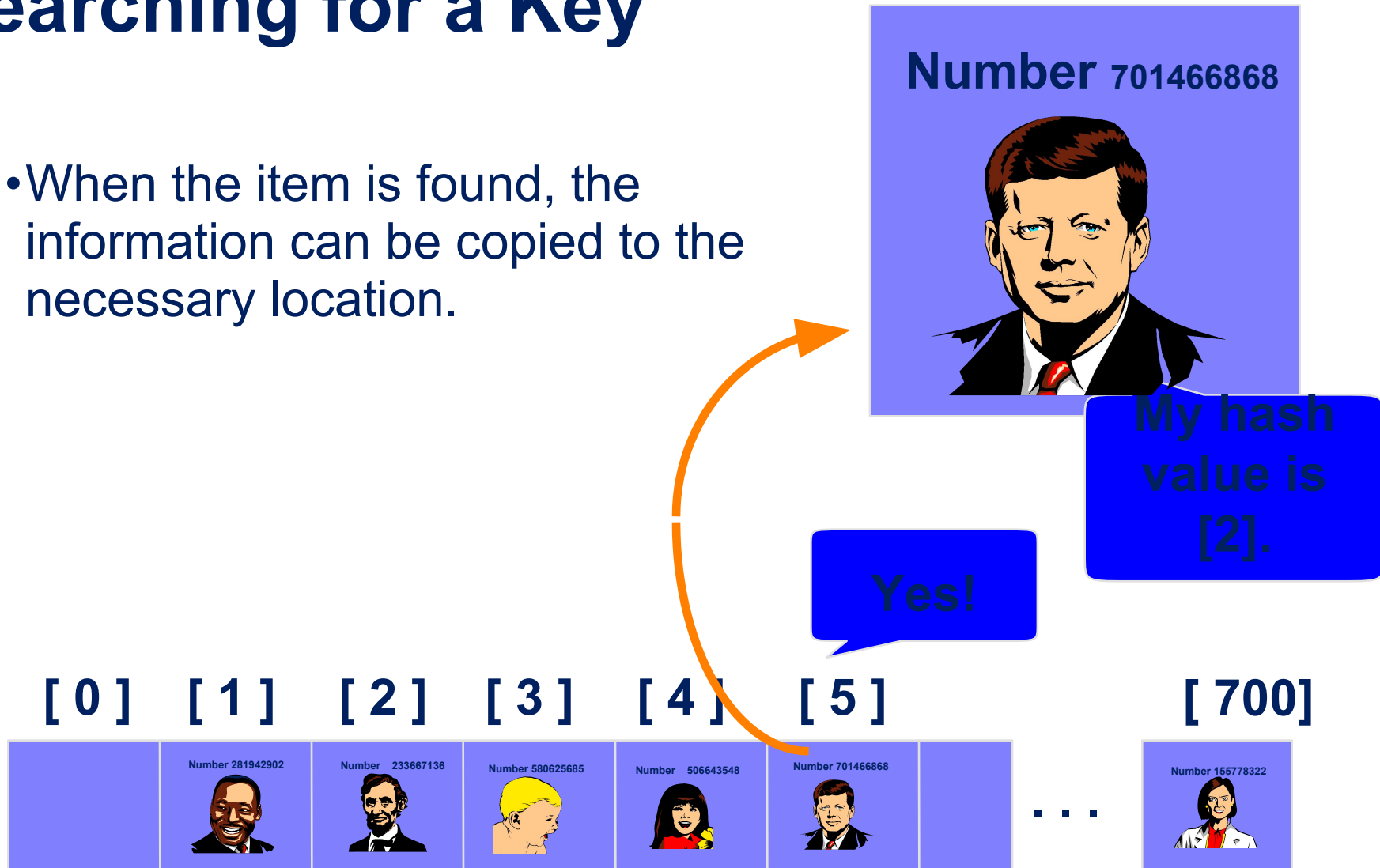
Yes!

[0] [1] [2] [3] [4] [5] ... [700]

	Number 281942902 	Number 233667136 	Number 580625685 	Number 506643548 	Number 701466868 	...	Number 155778322 
--	---	---	---	--	---	-----	---

Searching for a Key

- When the item is found, the information can be copied to the necessary location.



Deleting a Record

- Records may also be deleted from a hash table.



Deleting a Record

- Records may also be deleted from a hash table.
- But the location must not be left as an ordinary "empty spot" since that could interfere with searches.



Deleting a Record

- Records may also be deleted from a hash table.
- But the location must not be left as an ordinary "empty spot" since that could interfere with searches.
- The location must be marked in some special way so that a search can tell that the spot used to have something in it.



Summary

- Hash tables store a collection of records with keys.
- The location of a record depends on the hash value of the record's key.
- When a collision occurs, the next available location is used.
- Searching for a particular key is generally quick.
- When an item is deleted, the location must be marked in a special way, so that the searches know that the spot used to be used.

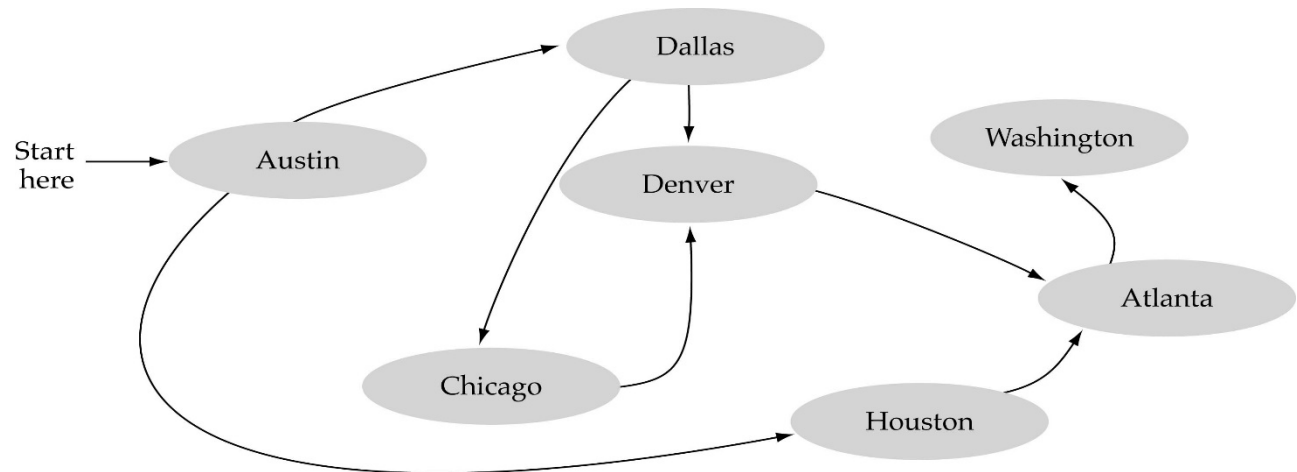
Graphs

Graphs

- Definition of Graphs and Related Concepts
- Representation of Graphs
- The Graph Class
- Graph Traversal
- Graph Applications

Definition of Graphs

- A graph is a finite set of nodes with edges between nodes
 - Formally, a graph G is a structure (V, E) consisting of
 - a finite set V called the set of nodes, and
 - a set E that is a subset of $V \times V$. That is, E is a set of pairs of the form (x, y) where x and y are nodes in V
- (OR)
- A data structure that consists of a set of nodes (vertices) and a set of edges that relate the nodes to each other
 - The set of edges describes relationships among the vertices

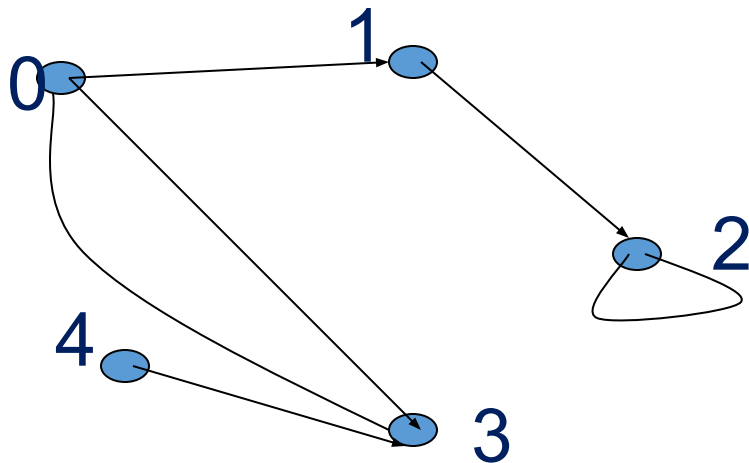


Formal definition of graphs

- A graph G is defined as follows:
 - $G=(V,E)$
 - $V(G)$: a finite, nonempty set of vertices
 - $E(G)$: a set of edges (pairs of vertices)

Examples of Graphs

- $V = \{0, 1, 2, 3, 4\}$
- $E = \{(0, 1), (1, 2), (0, 3), (3, 0), (2, 2), (4, 3)\}$



When (x, y) is an edge, we say that x is *adjacent to* y , and y is *adjacent from* x .

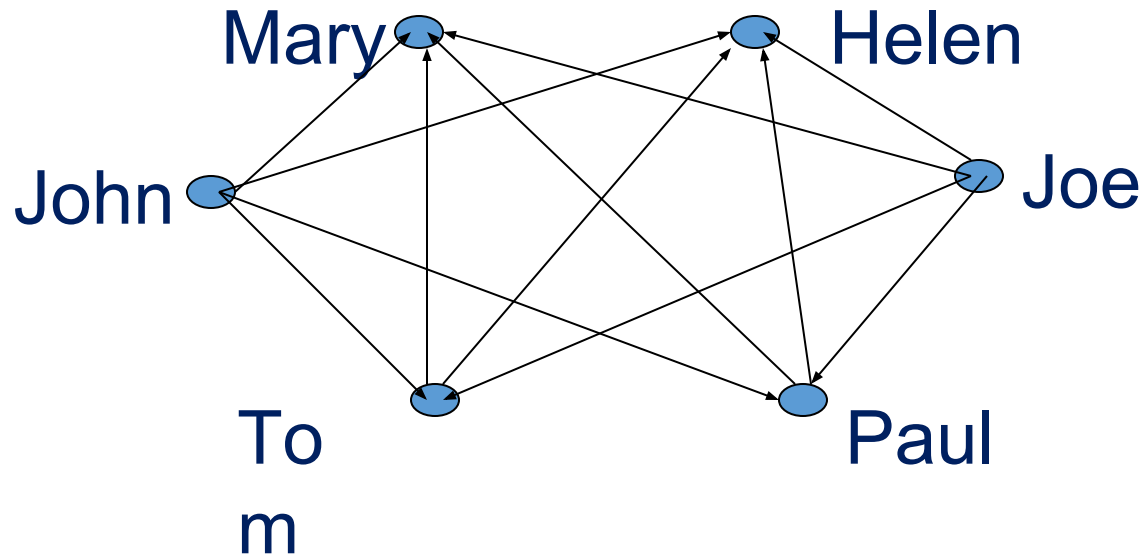
0 is adjacent to 1.

1 is not adjacent to 0.

2 is adjacent from 1.

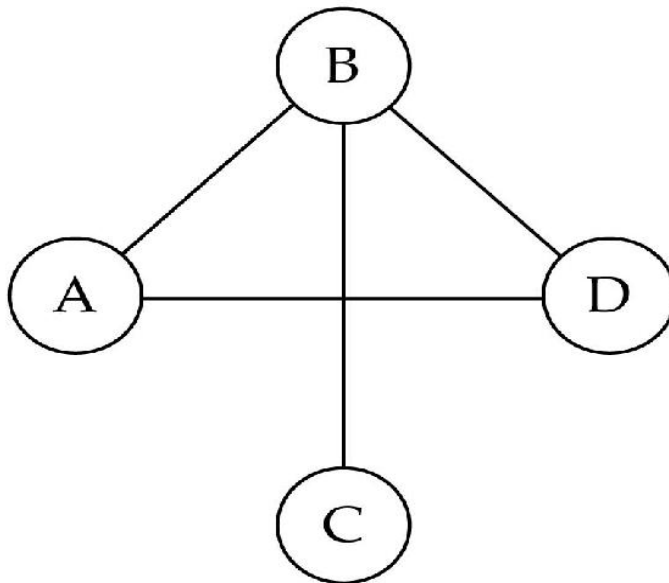
A “Real-life” Example of a Graph

- V = set of 6 people: John, Mary, Joe, Helen, Tom, and Paul, of ages 12, 15, 12, 15, 13, and 13, respectively.
- $E = \{(x,y) \mid \text{if } x \text{ is younger than } y\}$



Directed vs. undirected graphs

- When the edges in a graph have no direction, the graph is called undirected



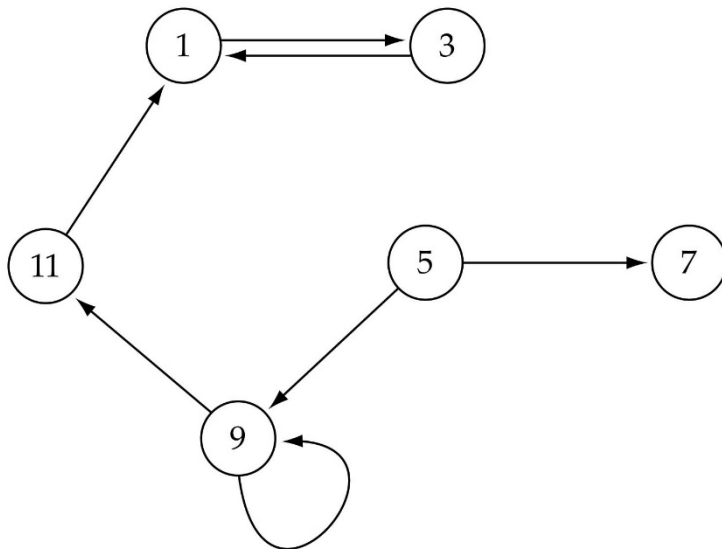
$V(\text{Graph1}) = \{ A, B, C, D \}$

$E(\text{Graph1}) = \{ (A, B), (A, D), (B, C), (B, D) \}$

Directed vs. undirected graphs (cont.)

- When the edges in a graph have a direction, the graph is called directed (or digraph)

(b) Graph2 is a directed graph.



$V(\text{Graph2}) = \{ 1, 3, 5, 7, 9, 11 \}$

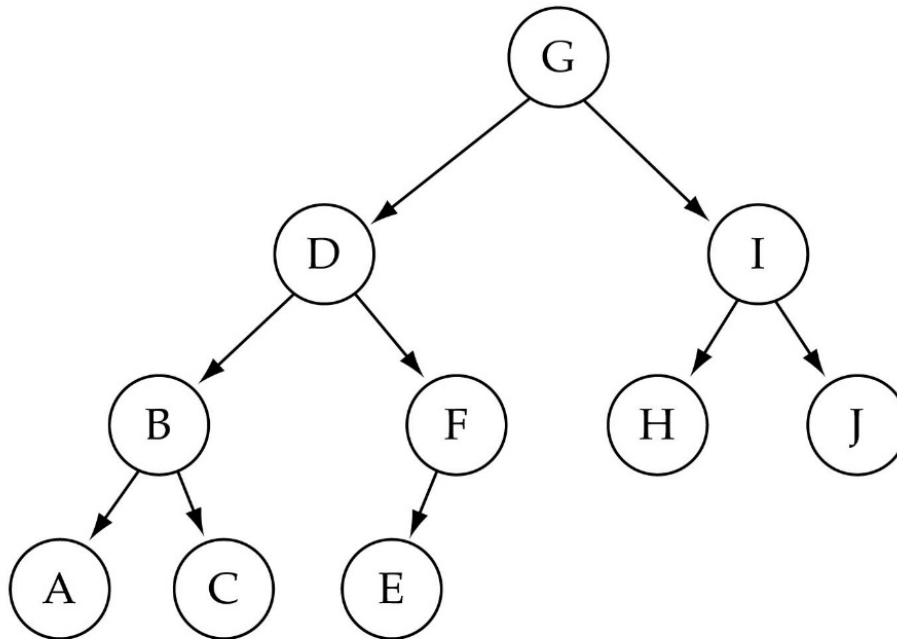
$E(\text{Graph2}) = \{(1,3) (3,1) (5,9) (9,11) (5,7), (9, 9), (11, 1) \}$

Warning: if the graph is directed, the order of the vertices in each edge is important !!

Trees vs graphs

- Trees are special cases of graphs!!

(c) Graph3 is a directed graph.

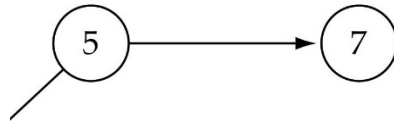


$V(\text{Graph3}) = \{ A, B, C, D, E, F, G, H, I, J \}$

$E(\text{Graph3}) = \{ (G, D), (G, I), (D, B), (D, F), (I, H), (I, J), (B, A), (B, C), (F, E) \}$

Graph terminology

- **Adjacent nodes:** two nodes are adjacent if they are connected by an edge



5 is adjacent to 7
7 is adjacent from 5

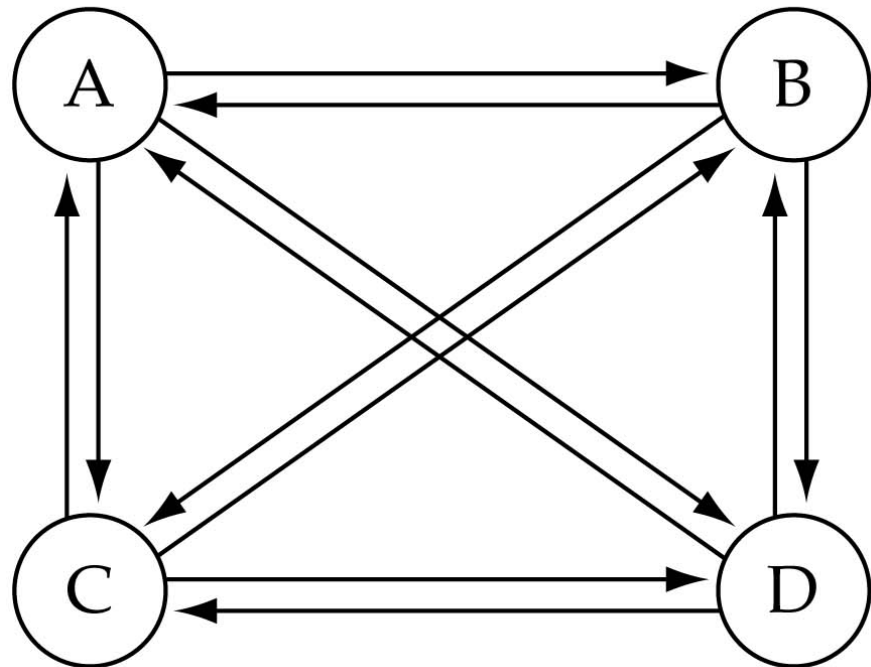
- **Path:** a sequence of vertices that connect two nodes in a graph
- **Complete graph:** a graph in which every vertex is directly connected to every other vertex

Graph terminology (cont.)

- What is the number of edges in a complete directed graph with N vertices?

$$N * (N-1)$$

$$O(N^2)$$



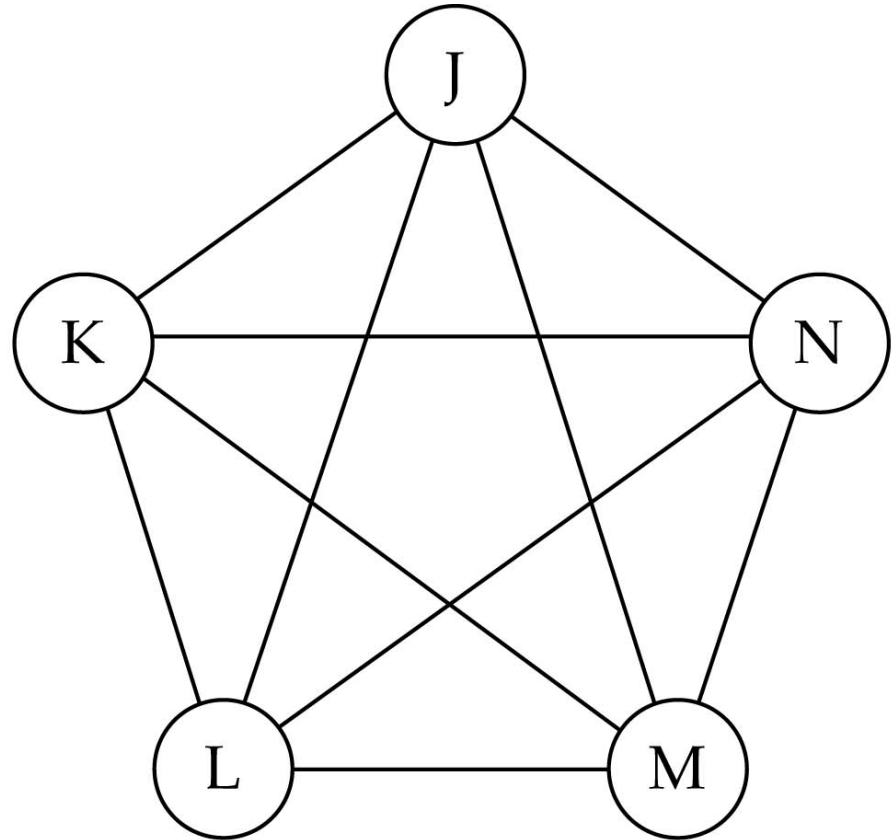
(a) Complete directed graph.

Graph terminology (cont.)

- What is the number of edges in a complete undirected graph with N vertices?

$$N * (N-1) / 2$$

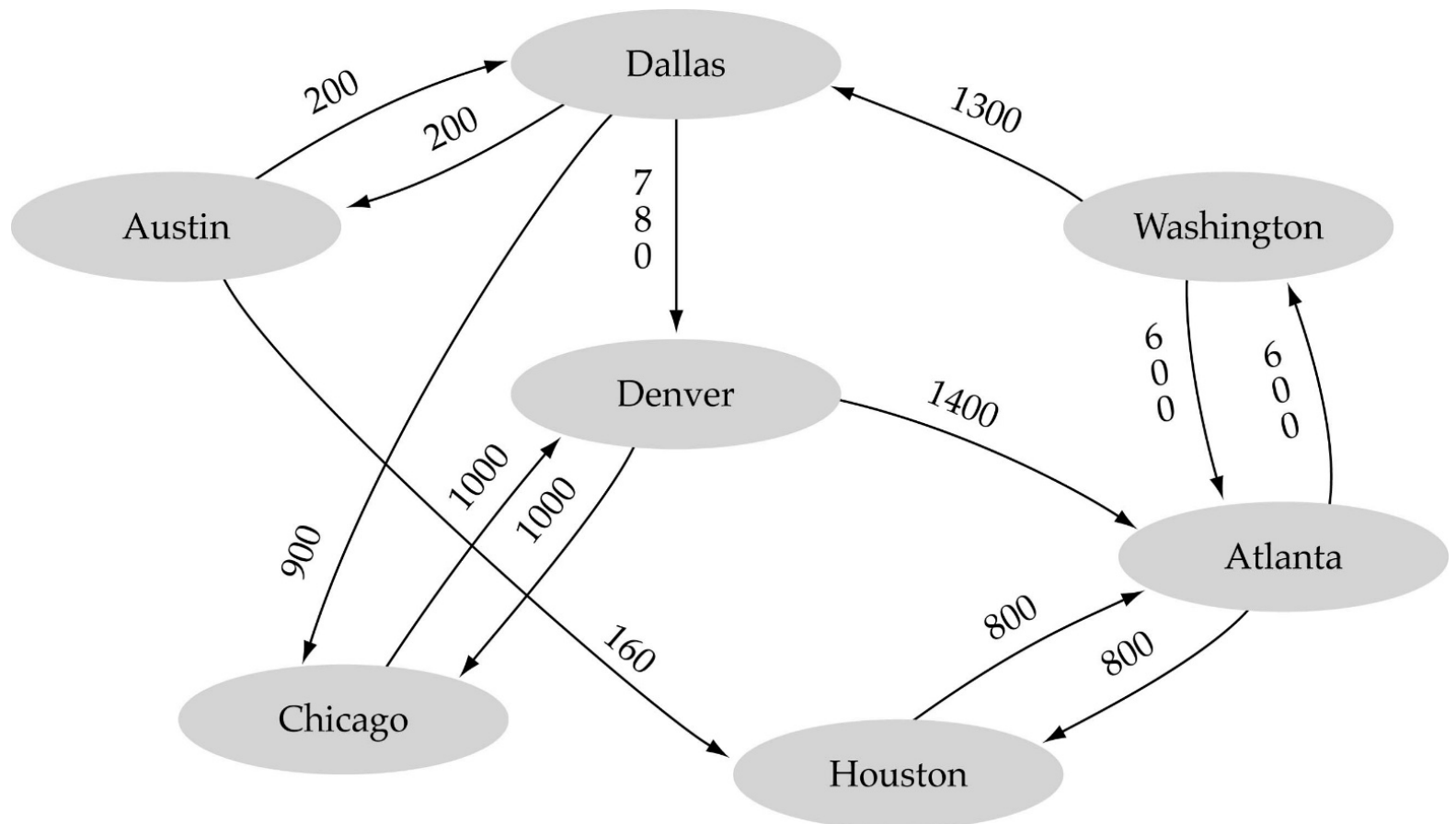
$$O(N^2)$$



(b) Complete undirected graph.

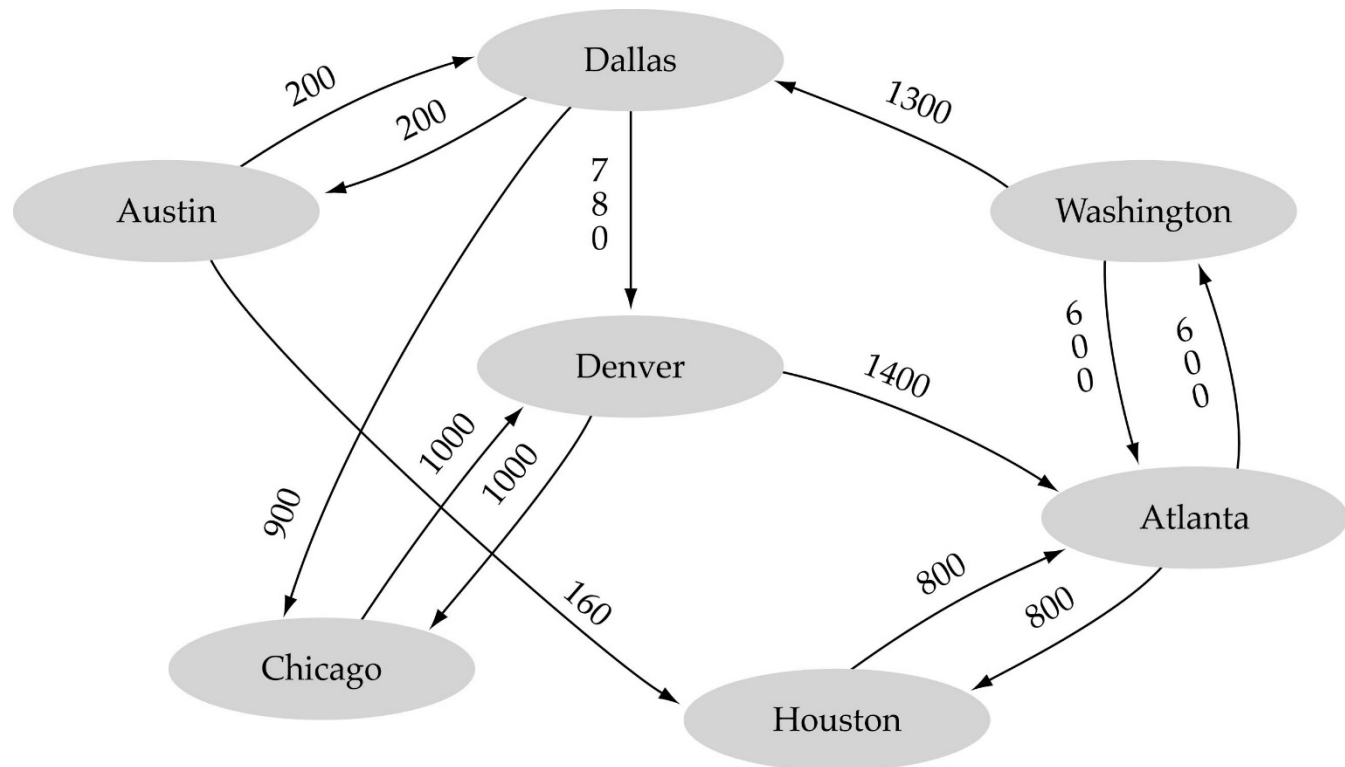
Graph terminology (cont.)

- Weighted graph: a graph in which each edge carries a value



Graph implementation

- **Array-based implementation**
- A 1D array is used to represent the vertices
- A 2D array (adjacency matrix) is used to represent the edges



graph

.numVertices 7

.vertices

[0]	"Atlanta"	"
[1]	"Austin"	"
[2]	"Chicago"	"
[3]	"Dallas"	"
[4]	"Denver"	"
[5]	"Houston"	"
[6]	"Washington"	"
[7]		
[8]		
[9]		

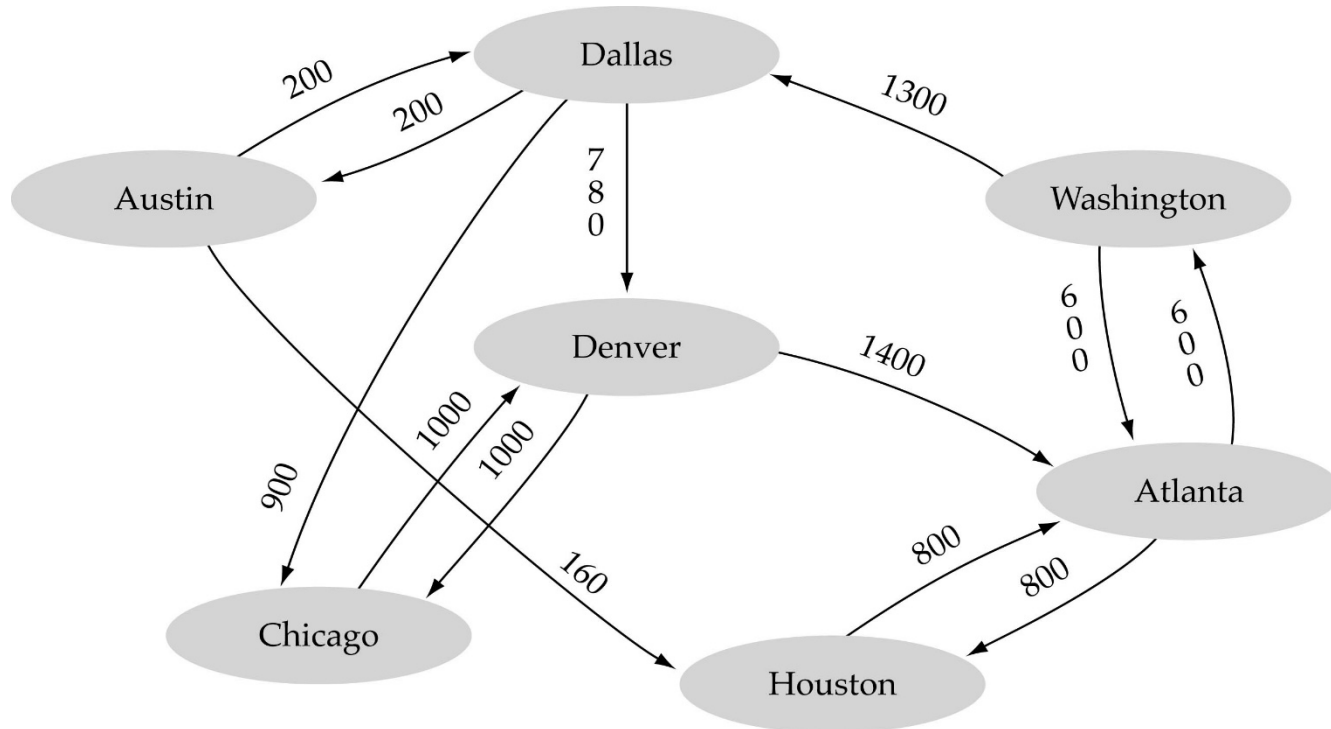
.edges

[0]	0	0	0	0	0	800	600	•	•	•
[1]	0	0	0	200	0	160	0	•	•	•
[2]	0	0	0	0	1000	0	0	•	•	•
[3]	0	200	900	0	780	0	0	•	•	•
[4]	1400	0	1000	0	0	0	0	•	•	•
[5]	800	0	0	0	0	0	0	•	•	•
[6]	600	0	0	1300	0	0	0	•	•	•
[7]	•	•	•	•	•	•	•	•	•	•
[8]	•	•	•	•	•	•	•	•	•	•
[9]	•	•	•	•	•	•	•	•	•	•
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

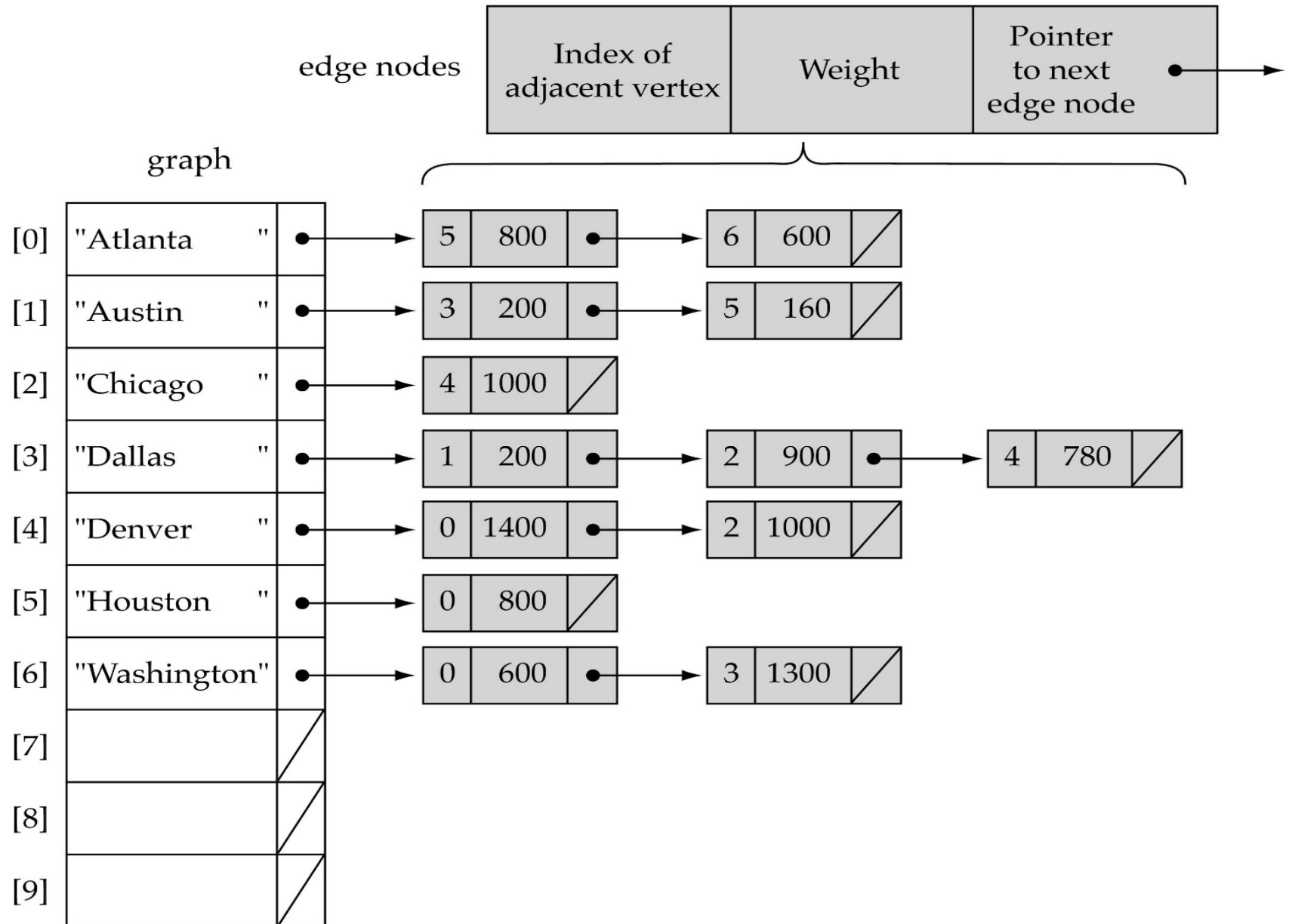
(Array positions marked '•' are undefined)

Graph implementation (cont.)

- **Linked-list implementation**
- A 1D array is used to represent the vertices
- A list is used for each vertex v which contains the vertices which are adjacent from v (adjacency list)



(a)



Graph Representation

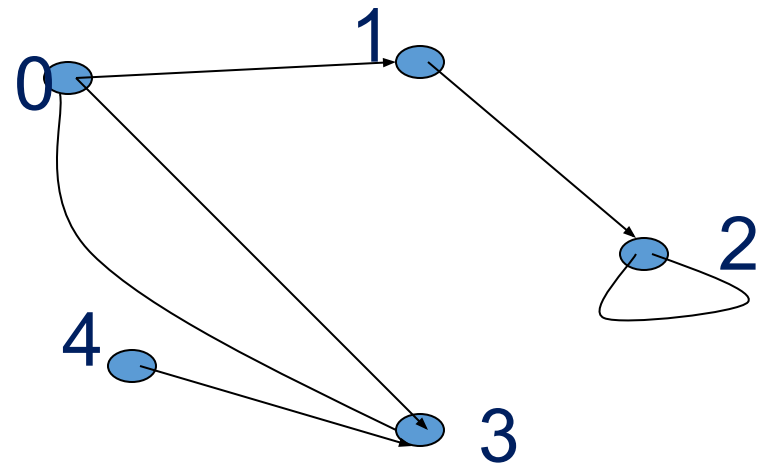
- For graphs to be computationally useful, they have to be conveniently represented in programs
- There are two computer representations of graphs:
 - Adjacency matrix representation
 - Adjacency lists representation

Adjacency Matrix Representation

- In this representation, each graph of n nodes is represented by an $n \times n$ matrix A , that is, a two-dimensional array A
- The nodes are (re)-labeled $1, 2, \dots, n$
- $A[i][j] = 1$ if (i, j) is an edge
- $A[i][j] = 0$ if (i, j) is not an edge

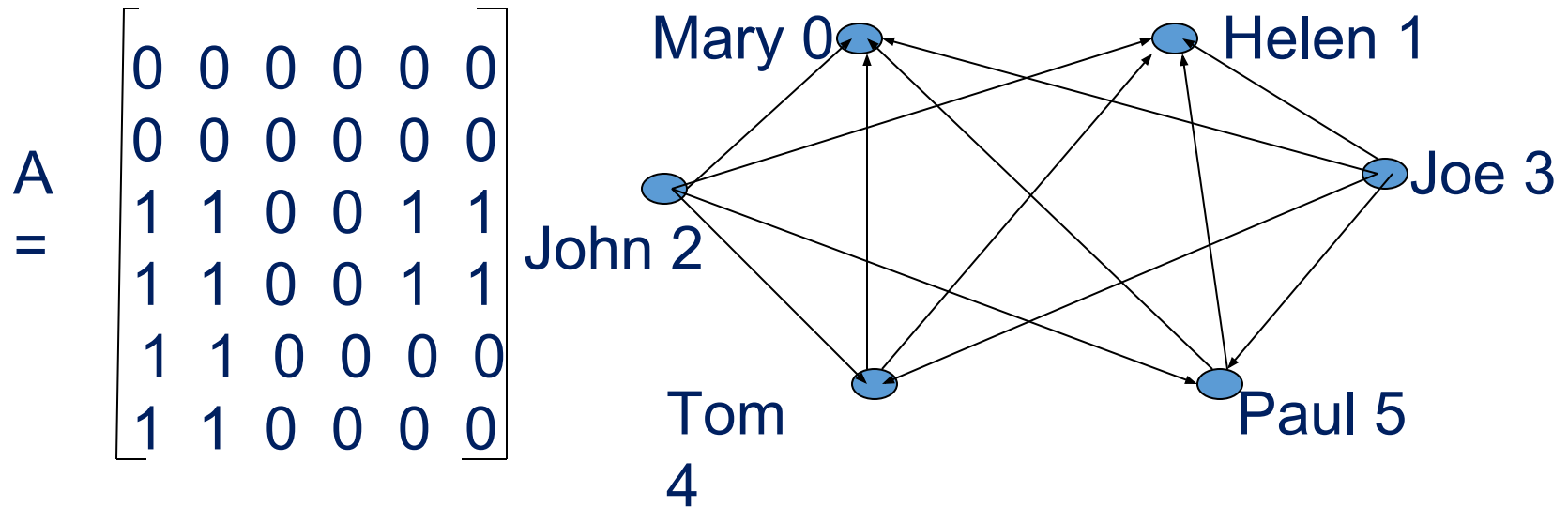
Example of Adjacency Matrix

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$



Another Example of Adj. Matrix

- Re-label the nodes with numerical labels



Pros and Cons of Adjacency Matrices

- Pros:**

- Simple to implement
- Easy and fast to tell if a pair (i,j) is an edge: simply check if $A[i][j]$ is 1 or 0

- Cons:**

- No matter how few edges the graph has, the matrix takes $O(n^2)$ in memory

Adjacency Lists Representation

- A graph of n nodes is represented by a one-dimensional array L of linked lists, where
 - $L[i]$ is the linked list containing all the nodes adjacent from node i .
 - The nodes in the list $L[i]$ are in no particular order

Example of Linked Representation

L[0]: empty

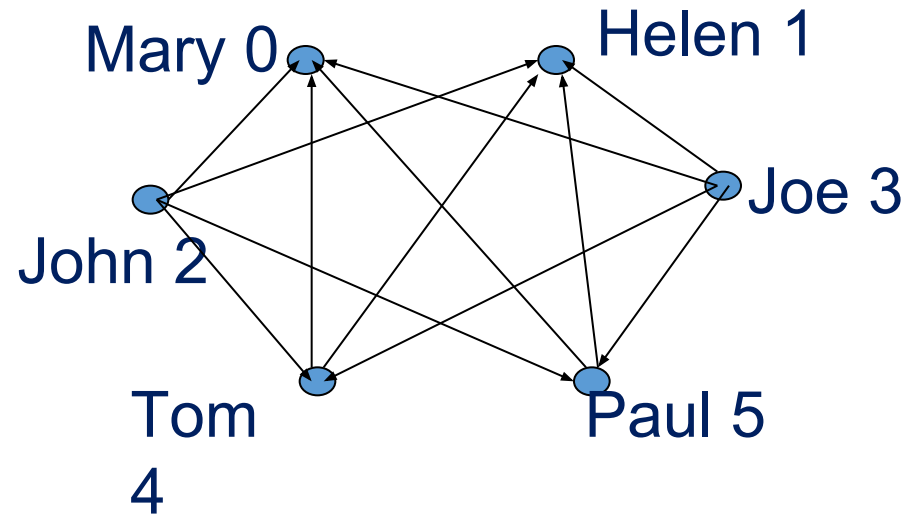
L[1]: empty

L[2]: 0, 1, 4, 5

L[3]: 0, 1, 4, 5

L[4]: 0, 1

L[5]: 0, 1



Pros and Cons of Adjacency Lists

- Pros:**

- Saves on space (memory): the representation takes as many memory words as there are nodes and edge.

- Cons:**

- It can take up to $O(n)$ time to determine if a pair of nodes (i,j) is an edge: one would have to search the linked list $L[i]$, which takes time proportional to the length of $L[i]$.

The Graph Class

```
class Graph {  
    public:  
        typedef int datatype;  
        typedef datatype * datatypeptr;  
        Graph( int n=0); // creates a graph of n nodes and no edges  
        bool isEdge( int i, int j);  
        void setEdge( int i, int j, datatype x);  
        int getNumberOfNodes(){return numberOfNodes;};  
    private:  
        datatypeptr *p; //a 2-D array, i.e., an adjacency matrix  
        int numberOfNodes;  
};
```

Graph Class Implementation

```
Graph::Graph( int n){
    assert(n>=0);
    numberOfNodes=n;
    if (n==0) p=NULL;
    else{
        p = new datatypeptr[n];
        for (int i=0;i<n;i++){
            p[i] = new datatype[n];
            for (int j=0;j<n;j++)
                p[i][j]=0;
        }
    }
};
```

```
bool Graph::isEdge(int i, int j){
    assert(i>=0 && j>=0);
    return p[i][j] != 0;
};
```

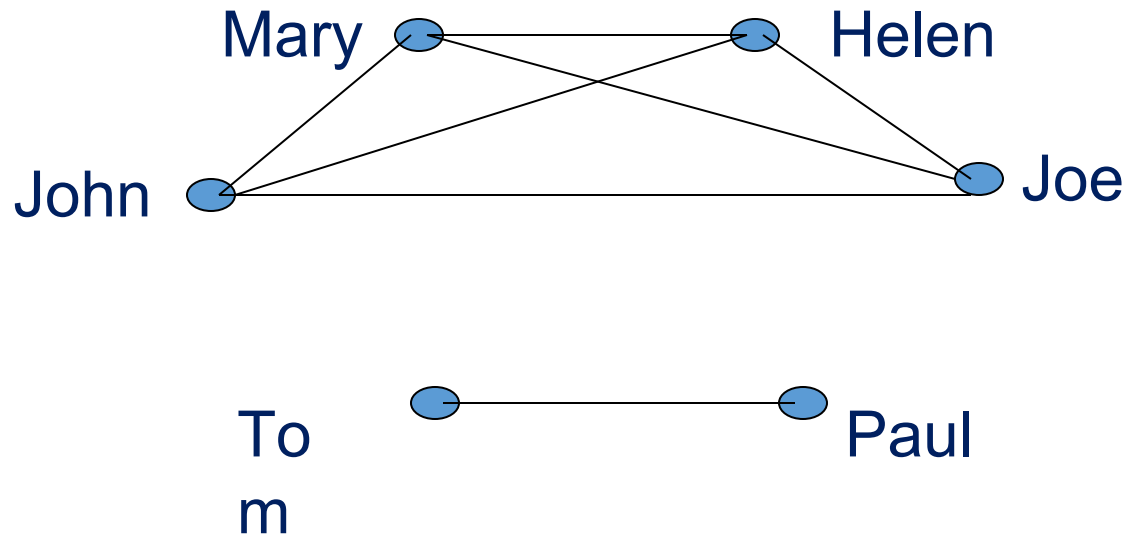
```
void Graph::setEdge(int i,
                    int j, datatype x){
    assert(i>=0 && j>=0);
    p[i][j]=x;
};
```

Directed vs. Undirected Graphs

- If the directions of the edges matter, then we show the edge directions, and the graph is called a *directed graph* (or a *digraph*)
- The previous two examples are digraphs
- If the relationships represented by the edges are symmetric (such as (x,y) is edge if and only if x is a sibling of y), then we don't show the directions of the edges, and the graph is called an *undirected graph*.

Examples of Undirected Graphs

- V = set of 6 people: John, Mary, Joe, Helen, Tom, and Paul, where the first 4 are siblings, and the last two are siblings
- $E = \{(x,y) \mid x \text{ and } y \text{ are siblings}\}$



if (x,y) is an edge:
we say that x is
adjacent to y , &
 y adjacent to x .
We also say that
 x and y are
neighbors

Representations of Undirected Graphs

- The same two representations for directed graphs can be used for undirected graphs
- Adjacency matrix A :
 - $A[i][j]=1$ if (i,j) is an edge; 0 otherwise
- Adjacency Lists:
 - $L[i]$ is the linked list containing all the neighbors of i

Example of Representations

Linked Lists:

L[0]: 1, 2, 3

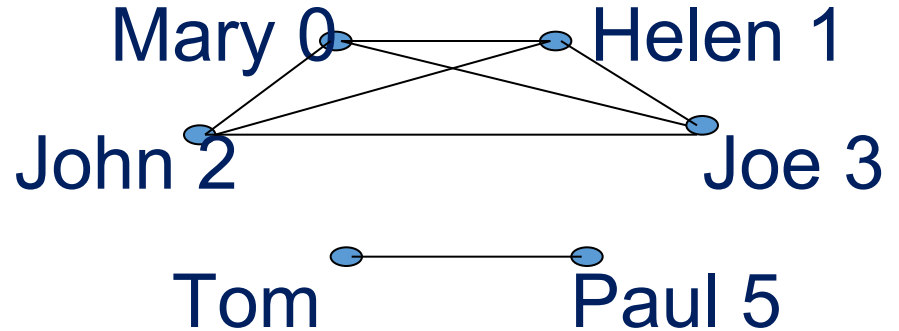
L[1]: 0, 2, 3

L[2]: 0, 1, 3

L[3]: 0, 1, 2

L[4]: 5

L[5]: 4



Adjacency Matrix:

A
=

0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	1	0	0
1	1	1	0	0	0
0	0	0	0	0	1
0	0	0	0	1	0

Definition of Some Graph Related Concepts

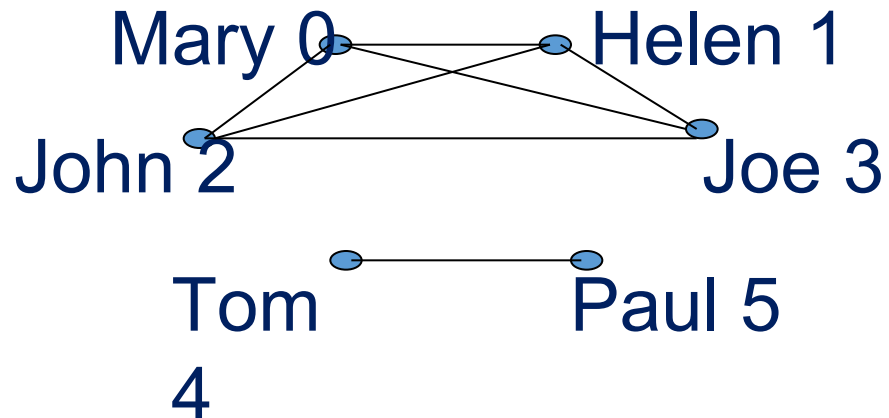
- Let G be a directed graph
 - The *indegree* of a node x in G is the number of edges coming to x
 - The *outdegree* of x is the number of edges leaving x .
- Let G be an undirected graph
 - The *degree* of a node x is the number of edges that have x as one of their end nodes
 - The *neighbors* of x are the nodes adjacent to x

Things for You To Do

- Add a member function to the class `graph`, called `getIndegree( int x)`, which returns the indegree of node `x`
- Add a member function to the class `graph`, called `getOutdegree( int x)`, which returns the outdegree of node `x`

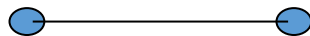
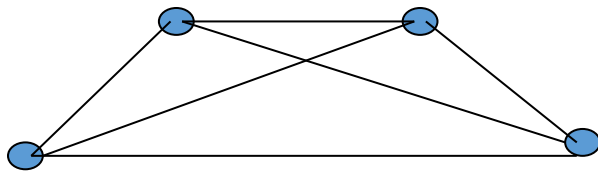
Paths

- A path in a graph G is a sequence of nodes $x_1, x_2, \dots, x_k$, such that there is an edge from each node to the next one in the sequence
- For example, in the first example graph, the sequence 3, 0, 1, 2 is a path, but the sequence 0, 3, 4 is not a path because (0,3) is not an edge
- In the “sibling-of” graph, the sequence **John, Mary, Joe, Helen** is a path, but the sequence **Helen, Tom, Paul** is not a path

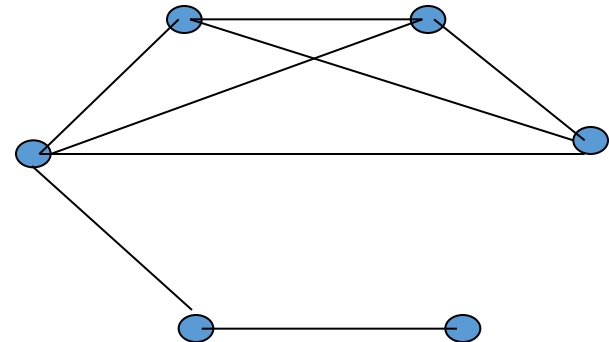


Graph Connectivity

- An undirected graph is said to be *connected* if there is a path between every pair of nodes. Otherwise, the graph is *disconnected*
- Informally, an undirected graph is connected if it hangs in one piece



Disconnected



Connected

Connected Components

- If an undirected graph is not connected, then each “piece” is called a connected component.
 - A piece in itself is connected, but if you bring any other node to it from the graph, it is no longer connected
- If the graph is connected, then the whole graph is one single connected component
- Of Interest: Given any undirected graph G ,
 - Is G connected?
 - If not, find its connected components.

Graph Traversal Techniques

- The previous connectivity problem, as well as many other graph problems, can be solved using graph traversal techniques
- There are two standard graph traversal techniques:
 - *Depth-First Search* (DFS)
 - *Breadth-First Search* (BFS)

Graph Traversal (Contd.)

- In both DFS and BFS, the nodes of the undirected graph are visited in a systematic manner so that every node is visited exactly one.
- Both BFS and DFS give rise to a tree:
 - When a node x is visited, it is labeled as visited, and it is added to the tree
 - If the traversal got to node x from node y , y is viewed as the parent of x , and x a child of y

Graph searching

1. Problem: find a path between two nodes of the graph (e.g., Austin and Washington)
2. Methods: Depth-First-Search (DFS) or Breadth-First-Search (BFS)

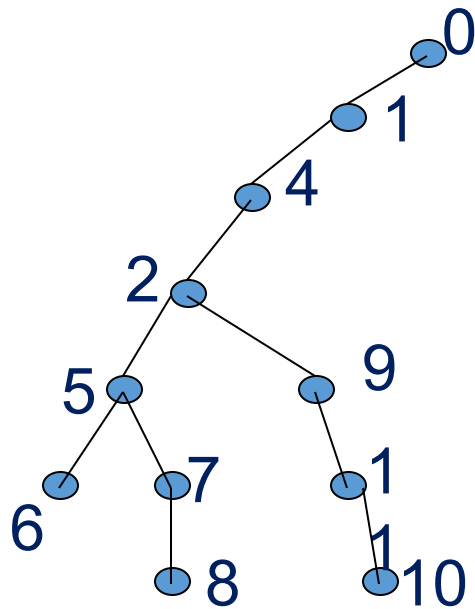
Depth-First-Search (DFS)

1. What is the idea behind DFS?
2. Travel as far as you can down a path
3. Back up as little as possible when you reach a "dead end" (i.e., next vertex has been "marked" or there is no next vertex)
4. DFS can be implemented efficiently using a stack

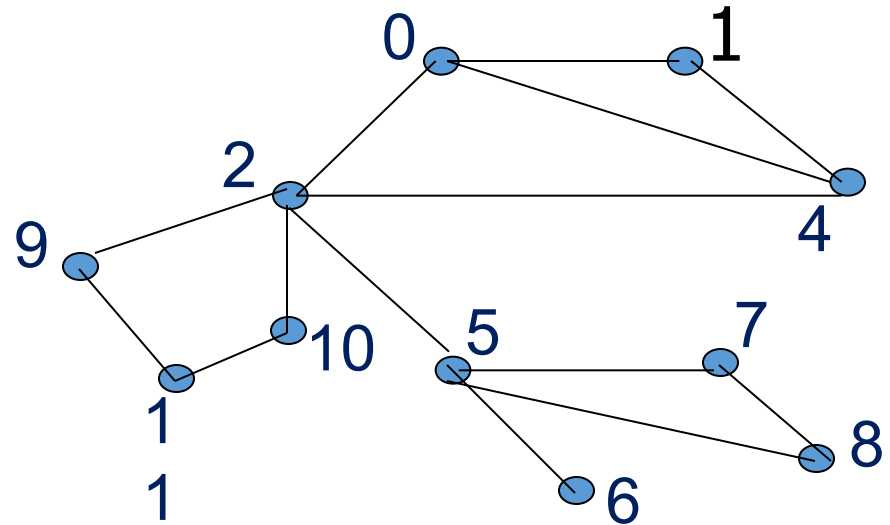
Depth-First Search

- DFS follows the following rules:
 1. Select an unvisited node x , visit it, and treat as the current node
 2. Find an unvisited neighbor of the current node, visit it, and make it the new current node;
 3. If the current node has no unvisited neighbors, backtrack to the its parent, and make that parent the new current node;
 4. Repeat steps 3 and 4 until no more nodes can be visited.
 5. If there are still unvisited nodes, repeat from step 1.

Illustration of DFS



DFS Tree



Graph G

Implementation of DFS

- **Observations:**

- the last node visited is the first node from which to proceed.
- Also, the backtracking proceeds on the basis of "last visited, first to backtrack too".
- This suggests that a stack is the proper data structure to remember the current node and how to backtrack.

Illustrate DFS with a Stack

- We will redo the DFS on the previous graph, but this time with stacks in Class

DFS (Pseudo Code)

```
DFS(input: Graph G) {  
    Stack S; Integer x, t;  
    while (G has an unvisited node x){  
        visit(x); push(x,S);  
        while (S is not empty){  
            t := peek(S);  
            if (t has an unvisited neighbor y){  
                visit(y); push(y,S); }  
            else  
                pop(S);  
        }  
    }  
}
```


C++ Code for DFS

```
int * dfs(Graph G){ // returns a parent array representing the DFS
tree
    int n=G.getNumberOfNodes();
    int * parent = new int[n];
    Stack S(n); bool visited[n];
    for ( int i=0; i<n; i++)    visited[i]=false;
    int x=0;    // begin DFS from node 0
    int numOfConnectedComponents=0;
    while (x<n){    // begin a new DFS from x
        numOfConnectedComponents++;
        visited[x]=true; S.push(x); parent[x] = -1; // x is root
        while(!S.isEmpty())    // traverse the current piece
            // insert here the yellow box from the next slide
            x= getNextUnvisited(visited,n,x);
    }
    cout<<"Graph has "<< numOfConnectedComponents<<
        " connected components\n";
    return p;
}
```

```

{
    int t=S.peek( );
    int y=getNextUnvisitedNeighbor(
        t,G,visited,n);
    if (y<n){
        visited[y]=true;
        S.push(y);
        parent[y]=t;
    }
    else    S.pop( );
}

```

```

// Put this before dfs(...)
// returns the leftmost unvisited
// neighbor of node t. If none
// remains, returns n.
int getNextUnvisitedNeighbor(int t,
    graph G, bool visited[],int n){
    for (int j=0;j<n;j++)
        if (G.isEdge(t,j) && !visited[j])
            return j;
    // if no unvisited neighbors left:
    return n;
}

```

```

//Put this before dfs(...). This returns the next unvisited node, or n otherwise
int getNextUnvisited(bool visited[],int n, int lastVisited){
    int j=lastVisited+1;
    while (visited[j] && j<n)    j++;
    return j;
}

```

Breadth-First-Searching (BFS)

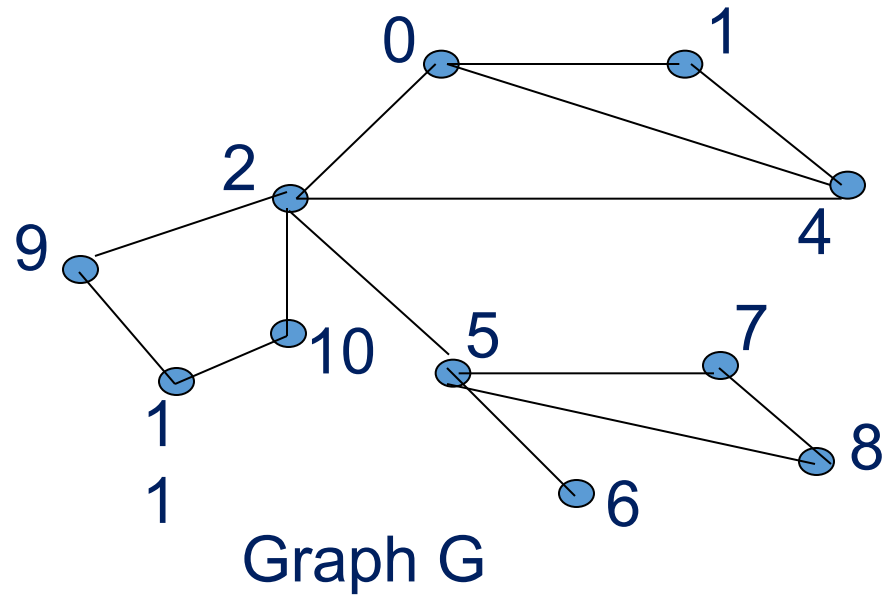
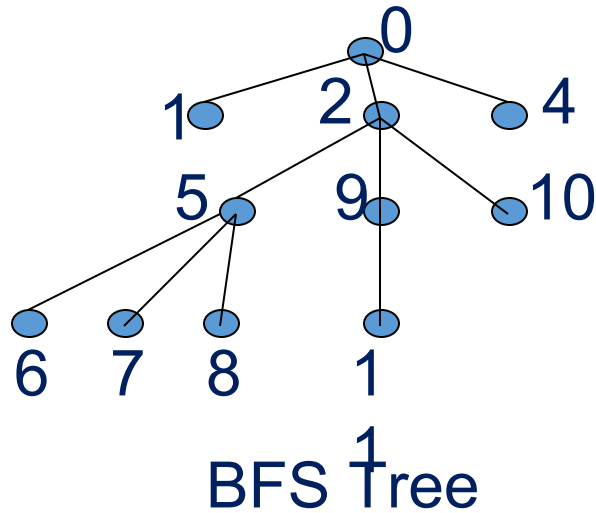
What is the idea behind BFS?

1. Look at all possible paths at the same depth before you go at a deeper level
2. Back up as far as possible when you reach a "dead end" (i.e., next vertex has been "marked" or there is no next vertex)

Breadth-First Search

- BFS follows the following rules:
 1. Select an unvisited node x , visit it, have it be the root in a BFS tree being formed. Its level is called the current level.
 2. From each node z in the current level, in the order in which the level nodes were visited, visit all the unvisited neighbors of z . The newly visited nodes from this level form a new level that becomes the next current level.
 3. Repeat step 2 until no more nodes can be visited.
 4. If there are still unvisited nodes, repeat from Step 1.

Illustration of BFS



Implementation of DFS

- Observations:
 - the first node visited in each level is the first node from which to proceed to visit new nodes.
- This suggests that a queue is the proper data structure to remember the order of the steps.

Illustrate BFS with a Queue

- We will redo the BFS on the previous graph, but this time with queues in Class

BFS (Pseudo Code)

```
BFS(input: graph G) {  
    Queue Q;    Integer x, z, y;  
    while (G has an unvisited node x) {  
        visit(x); Enqueue(x,Q);  
        while (Q is not empty){  
            z := Dequeue(Q);  
            for all (unvisited neighbor y of z){  
                visit(y); Enqueue(y,Q);  
            }  
        }  
    }  
}
```


Things for you to Do

- Give a C++ implementation of BFS, using the pseudo code as your guide
- Use BFS as a way to determine if the input graph G is connected, and if **not**, to output the number of connected components
- Modify BFS so that the level of each node in the BFS tree is computed.
- Give another class for graph, this time using a linked lists representation.

END